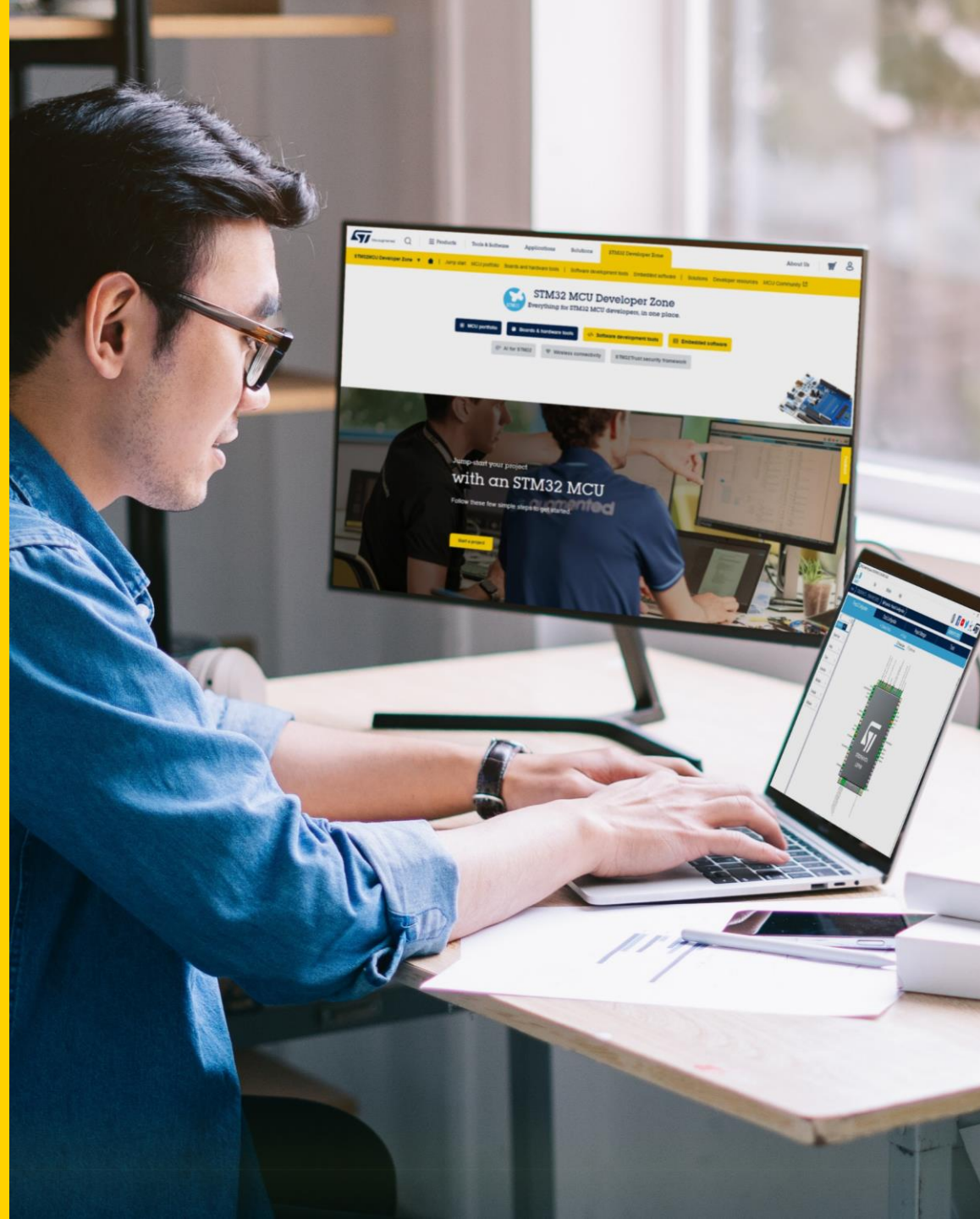




STM32CubeIDE for Visual Studio Code

意法半导体 中国区

微控制器、数字IC与射频产品部 (MDRF)

STM32团队

Agenda

1 L1-硬件环境与软件安装

2 L2-STMicro32CubeMX创建工程

3 L2-工程配置

4 L2-编辑/编译/调试

5 L3-进阶主题

6 L3-丰富生态

7 问与答

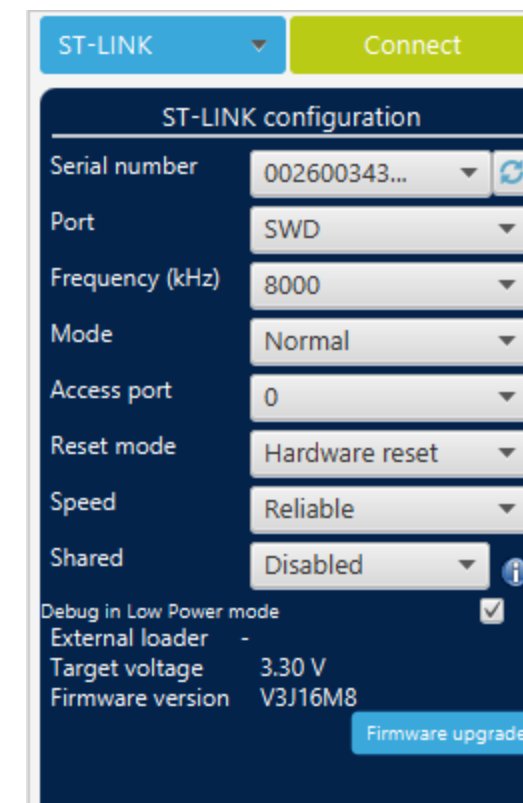
硬件环境与软件安装

- 硬件
 - NUCLEO-U385-RG-Q (NUCLEO-H533RE/NUCLEO-H755ZI-Q)
 - Type-C USB线
- 工具
 - STM32CubeMX 6.17.0
 - STM32CubeProgrammer v2.22.0
 - VS Code(最新版 V1.111.0)
 - STM32CubeIDE for VS Code extension pack V3.8.0

注意： 不限于这些配置

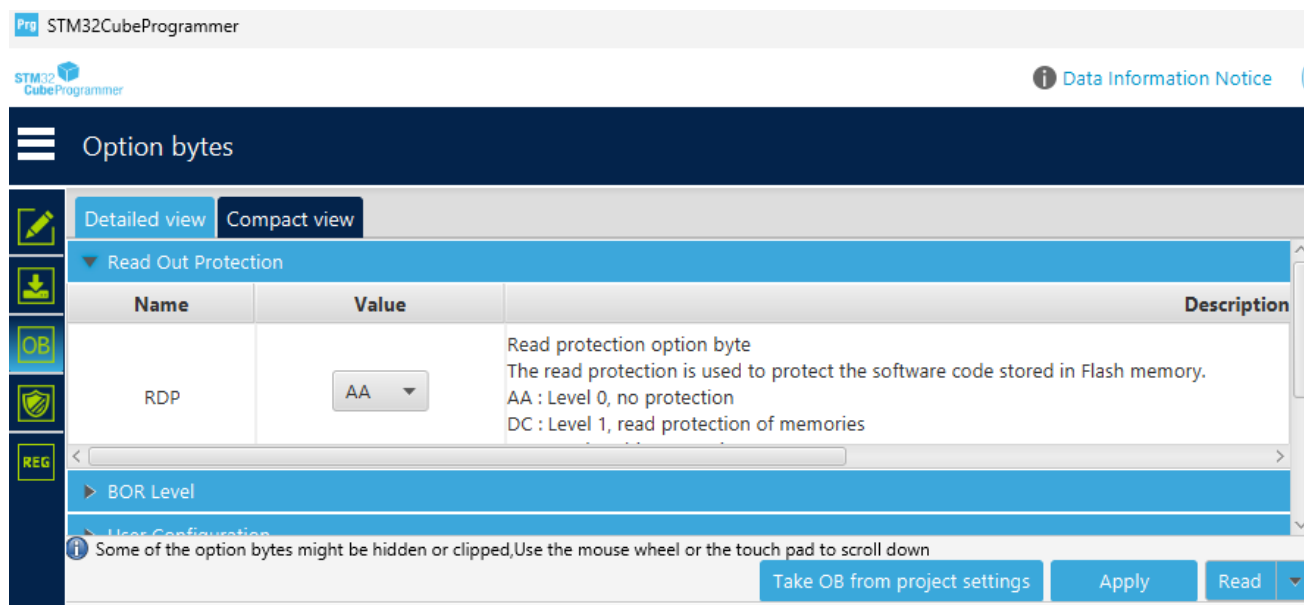
硬件环境检查

- 使用STM32CubeProgrammer检查环境
 - 点击Connect连接开发板
 - 如果提示，请升级ST-LINK版本
 - 确认能够连接成功



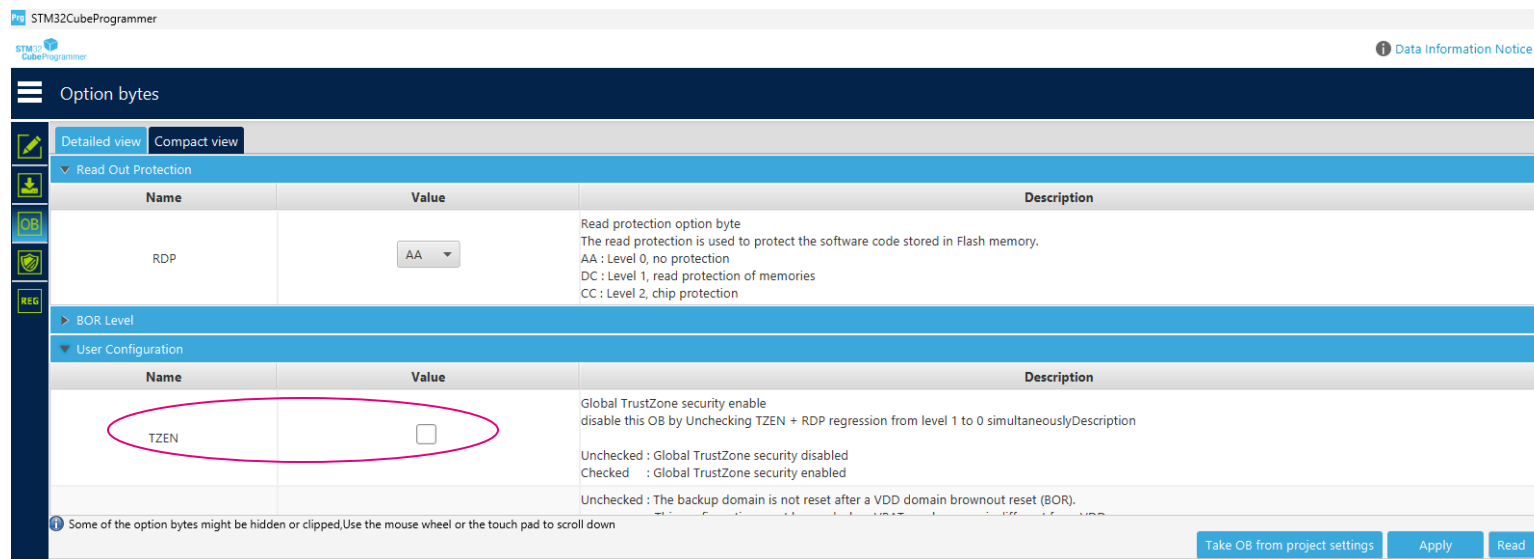
硬件环境检查

- 检查Option bytes
 - 确认当前状态位于Open



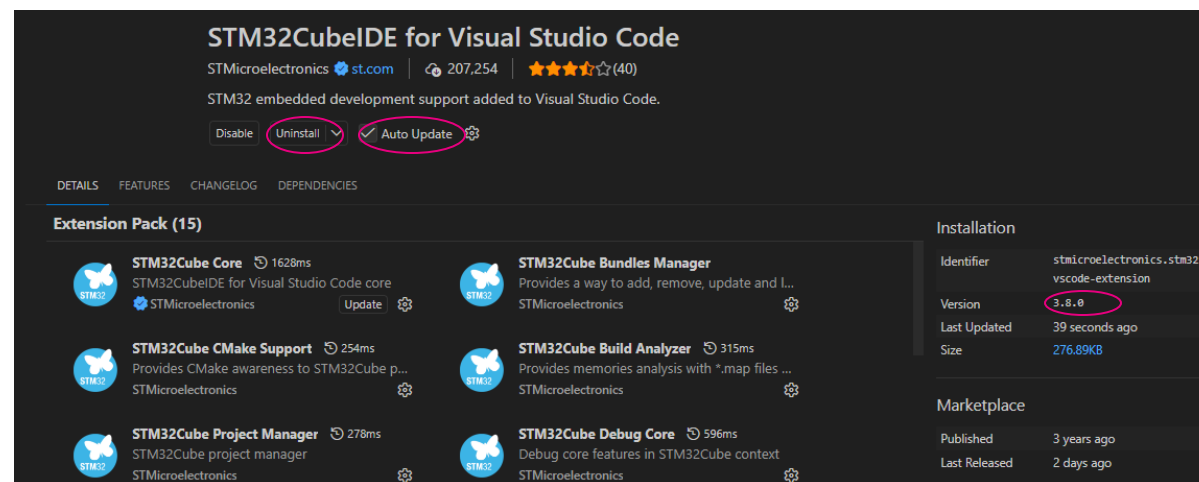
硬件环境检查

- 检查Option bytes
 - 确认Trustzone在关闭状态



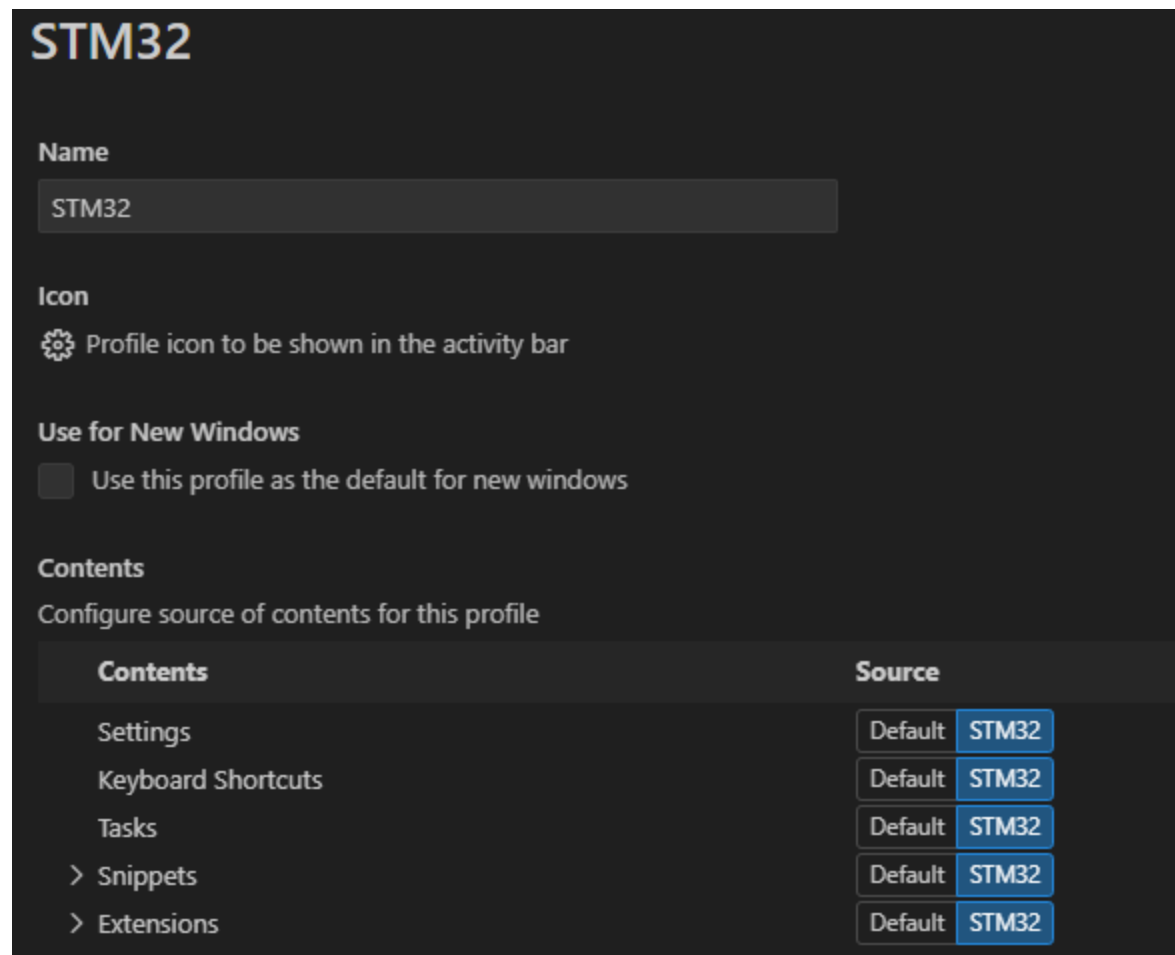
STM32CubeIDE For VSCode安装

- VSCode安装选择最新版本
- 安装STM32CubeIDE For VS Code
 - 请选择最新版本V3.8.0。如果之前安装了pre-release或者其他版本，请卸载之前版本后再安装。
 - 只需要选择该组件，其他组件会被自动安装
 - 请关闭VSCode的当前工作目录，避免网络问题
 - 可关闭组件的【Auto update】
- VSCode的路径建议不带空格或者中文字符
- 不建议安装到D:或者其他非C:
- 不需要提前安装任何依赖组件(可以看到提示为INSTALLED 0)



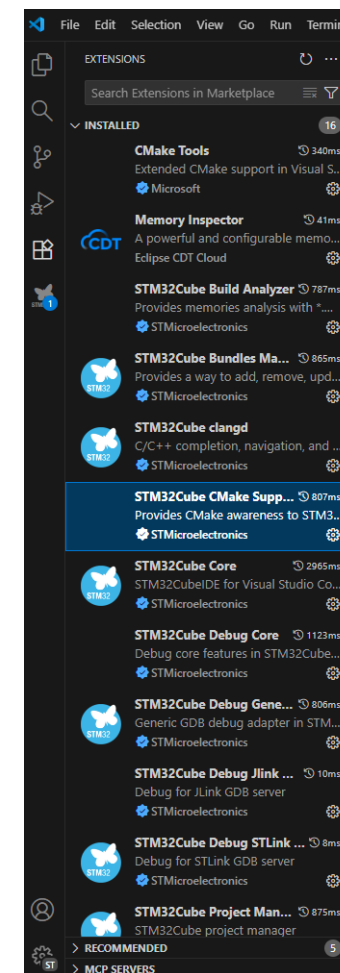
STM32CubeIDE for VS Code安装技巧

- 建议使用单独的Profile[非必须]
 - 使用【File】->【Preference】->【Profile】->【Profiles】->【New Profile】创建新Profile
 - 例如：可以将新的Profile的名字命名为 STM32
 - 使用【File】->【Preference】->【STM32】切换到新Profile
- 建议路径不要有中文，不要有空格
- 在线安装



安装效果图

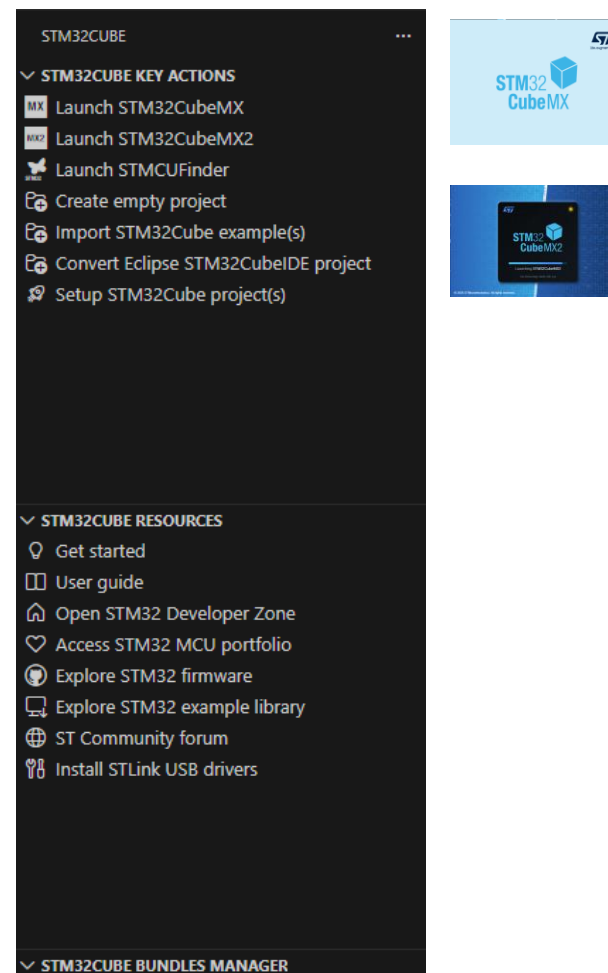
- 安装成功后可以看到左边的STM32蝴蝶图标
- 在组件图标（窗格）内可以看到自动安装的其他STM32Cube组件和CMake



在VSCode的STM32Cube界面

- **【Create empty project】**
 - 接下来会用到，用来验证环境和熟悉流程
- **【Setup STM32Cube project(s)】**
 - 接下来会用到，用来自动配置从STM32CubeMX生成的工程

建议：阅读在线帮助【Get Started】和【User guide】



- 网络在任何情况下不能访问，检查
 - C:\Users\用户名\AppData\Roaming\Code\User\settings.json
- 版本问题
 - C:\Users\用户名\AppData\Local\stm32cube\bundles
 - C:\Users\用户名\AppData\Local\stm32cube\packs
- 其他Error
 - 如果VSCode弹出的Error不含有STM32字样，则不用处理

创建空工程

- 使用扩展中的【Create empty project】可以创建一个空工程，名字为demo，以验证安装是否完成

Create empty project 1

Enter project name | Create project (1/4)

demo 2

Project name (Press 'Enter' to confirm or 'Escape' to cancel)

Select hardware | Create project (2/4)

Device (4911 available)
Start from microcontroller or microprocessor

Board (206 available) 3
Start from evaluation board



Select board | Create project (2/4)

u3 4

NUCLEO-U031R8
NUCLEO-U083RC
NUCLEO-U385RG-Q

Select project directory | Create project (3/4) | Create project (4/4)

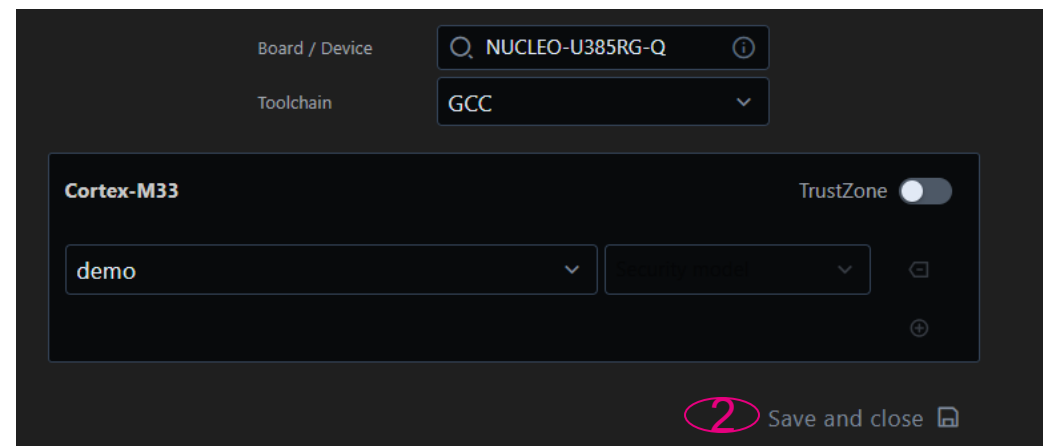
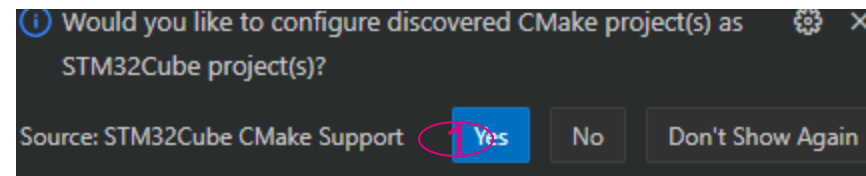
This PC > OSDisk (C:) > MCU 5

Create project 6 Actions

What do you want to do?

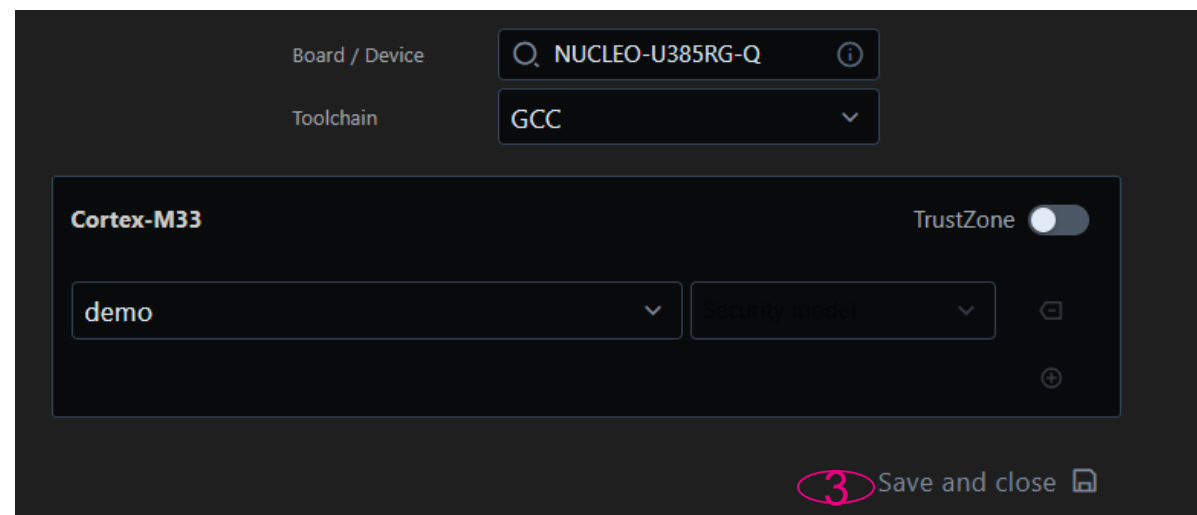
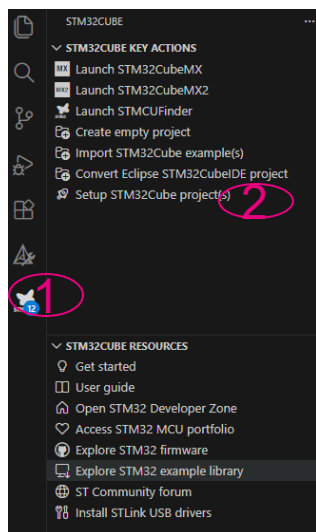
Sour... 7 Open in this window Open new window Add to workspace

- 使用它自动弹出的对话框进行配置
- 选择【CONFIGURE】
- 选择【Save and close】

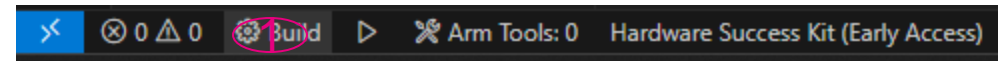
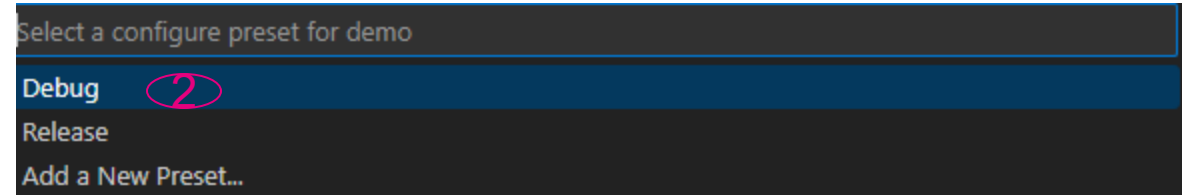


再次配置

- 如果对话框消失，也可以通过之前提到的STM32Cube界面选择【Setup STM32Cube project(s)】
- 选择【CONFIGURE】
- 选择【Save and close】

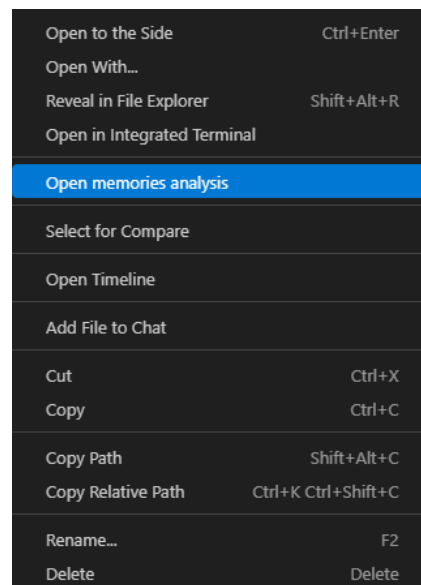
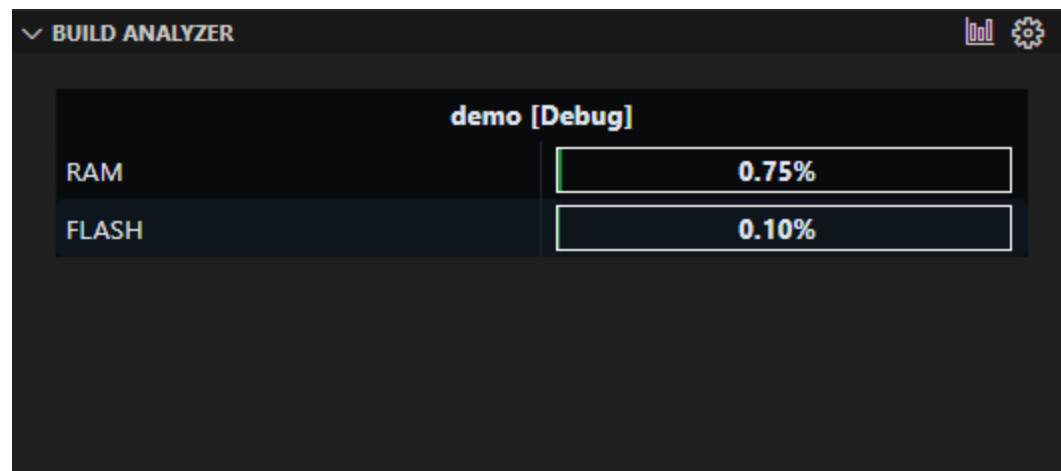


- 在配置过程中已经触发编译，则请选择【Debug】让其编译
- 如果需要再次编译
 - 在界面底部，选择【Build】
 - 如果弹出配置选项，可选择【Debug】



查看编译数据

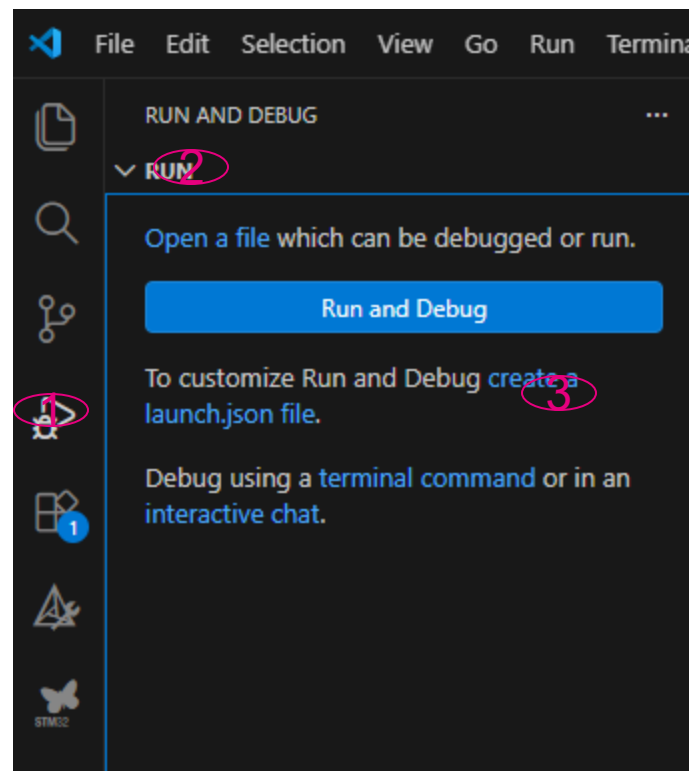
- 从左边栏进入【Cmake】扩展
- 查看【Build ANALYZER】可以得到STM32 Cube的构建分析
 - RAM
 - Flash



如果【Build ANALYZER】不能正确显示，则可以到在VSCode源文件界面找到build相应子目录下的map文件，右键打开【open memories analysis】

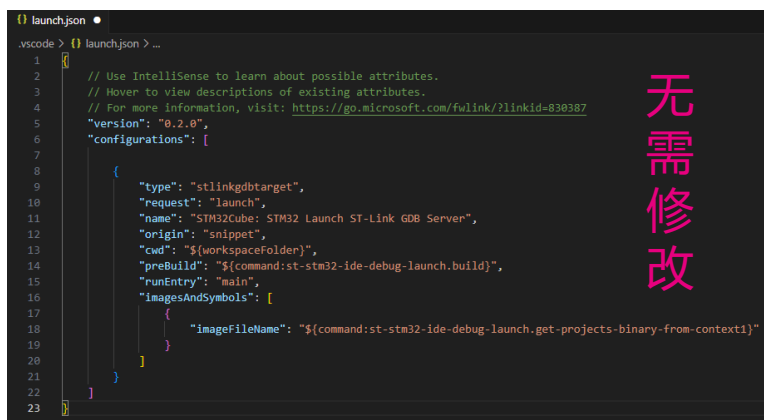
启动调试

- 启动调试请使用图标【Run and Debug(Ctrl + Shift +D)】
- 选择【RUN】
- 点击【create a launch.json file】
 - 如果不想保存调试配置，也可以直接使用【Run and Debug】



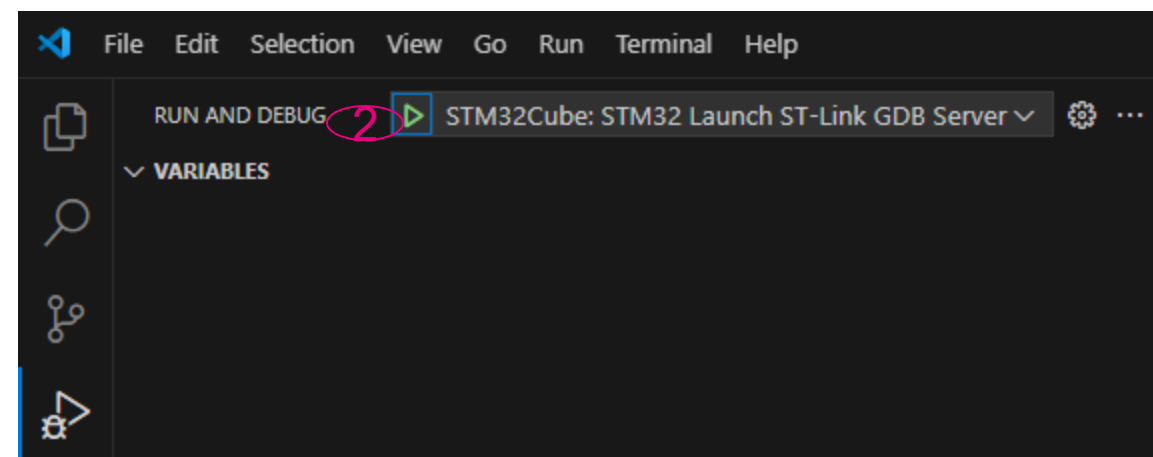
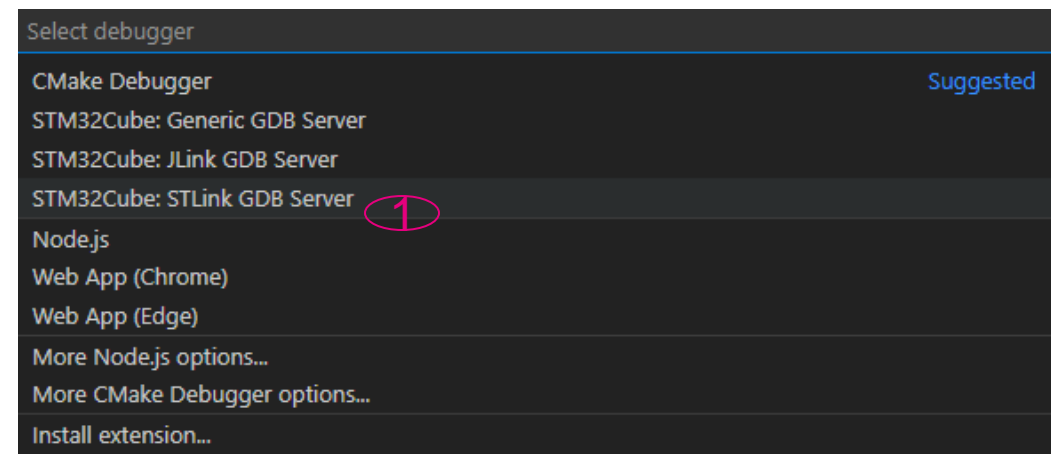
生成配置文件

- 选择【STM32Cube:STLink GDB Server】
- 对于弹出的【launch.json】文件，对于这一步无需修改
- 点击【STM32Cube: STM32 Launch ST-Link GDB Server】启动调试



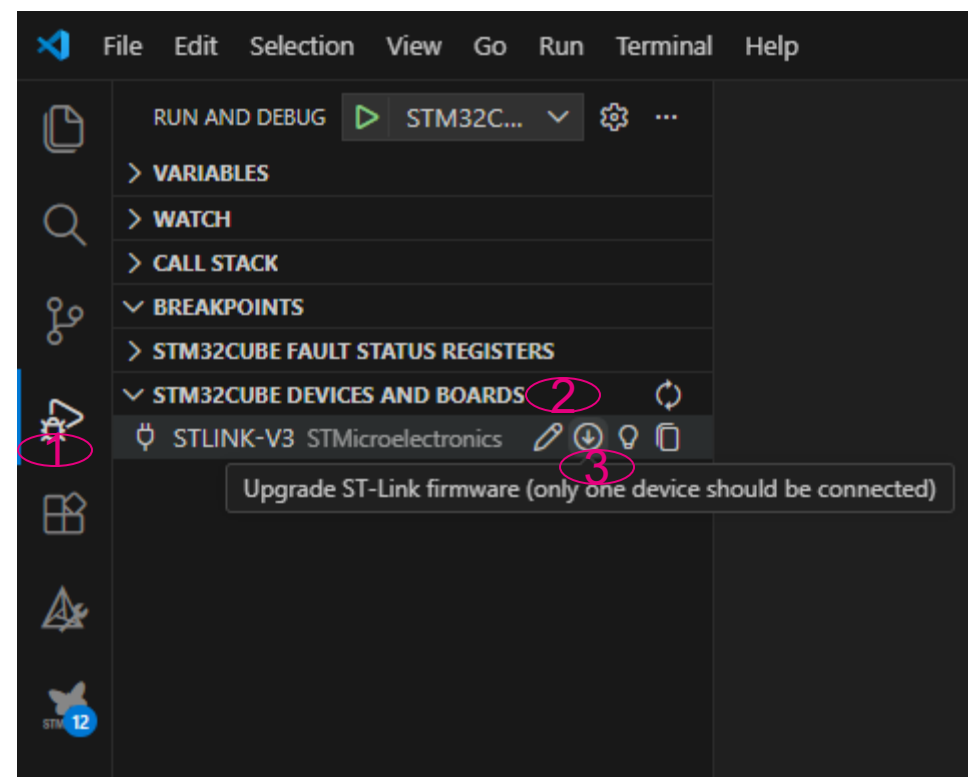
```
.vscode > launch.json > ...
1 // Use IntelliSense to learn about possible attributes.
2 // Hover to view descriptions of existing attributes.
3 // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
4 "version": "0.2.0",
5 "configurations": [
6   {
7     "type": "stlinkdbtarget",
8     "request": "launch",
9     "name": "STM32Cube: STM32 Launch ST-Link GDB Server",
10    "origin": "snippet",
11    "cwd": "${workspaceFolder}",
12    "preBuild": "${command:st-stm32-ide-debug-launch.build}",
13    "runEntry": "main",
14    "imagesAndSymbols": [
15      {
16        "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
17      }
18    ]
19  }
20 ]
21
22
23
```

无需修改



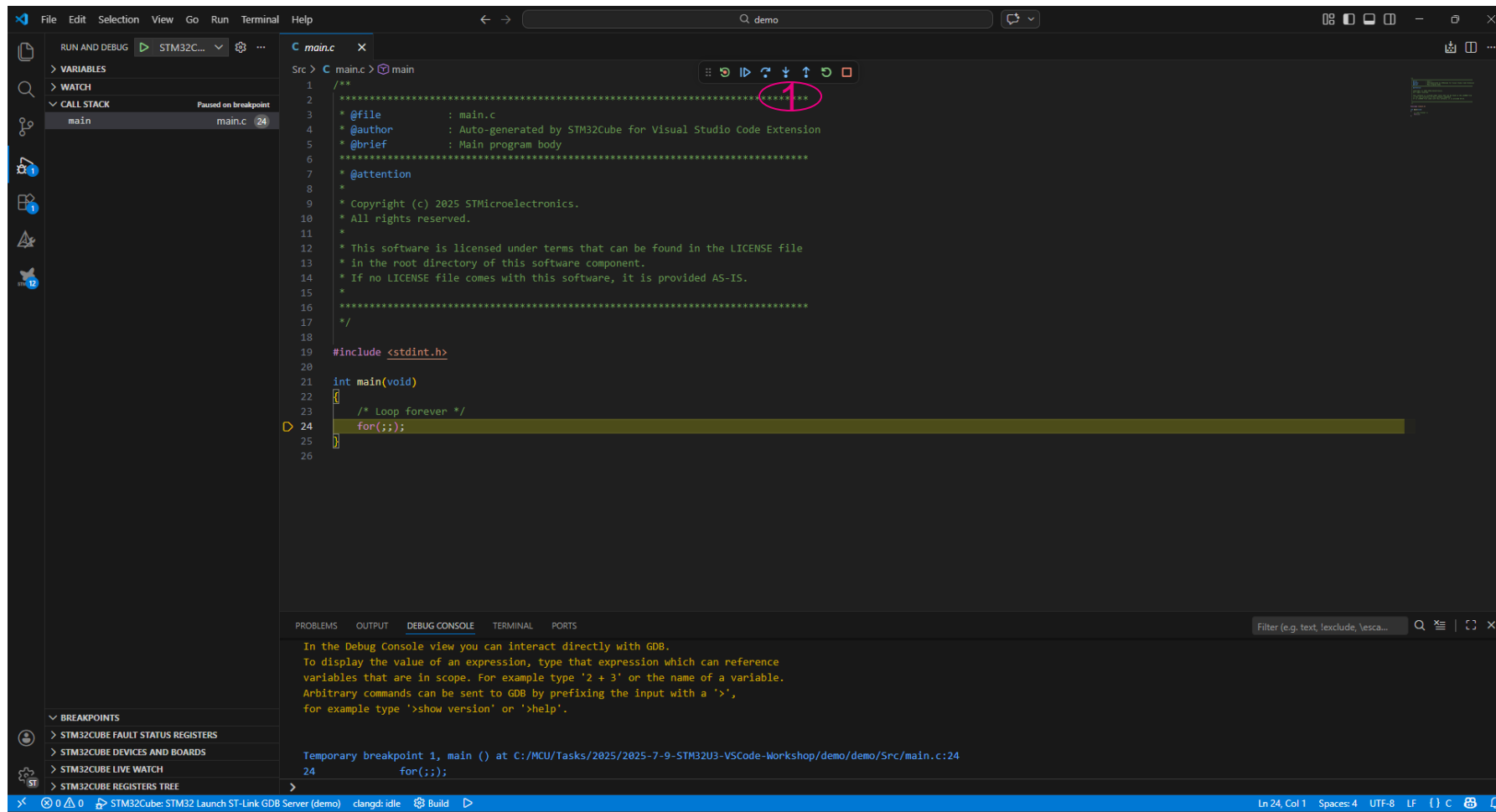
更新固件

- 点击图标【Run and Debug(Ctrl + Shift +D)】
- 选择【STM32CUBE DEVICES AND BOARDS】
- 选择【Upgrade ST-LINK Firmware】



在配置，编译完成后，系统会自动安装了其他依赖组件。例如，可以看到最左边菜单栏多了新的图标，其中包括之后需要使用Cmake

调试界面



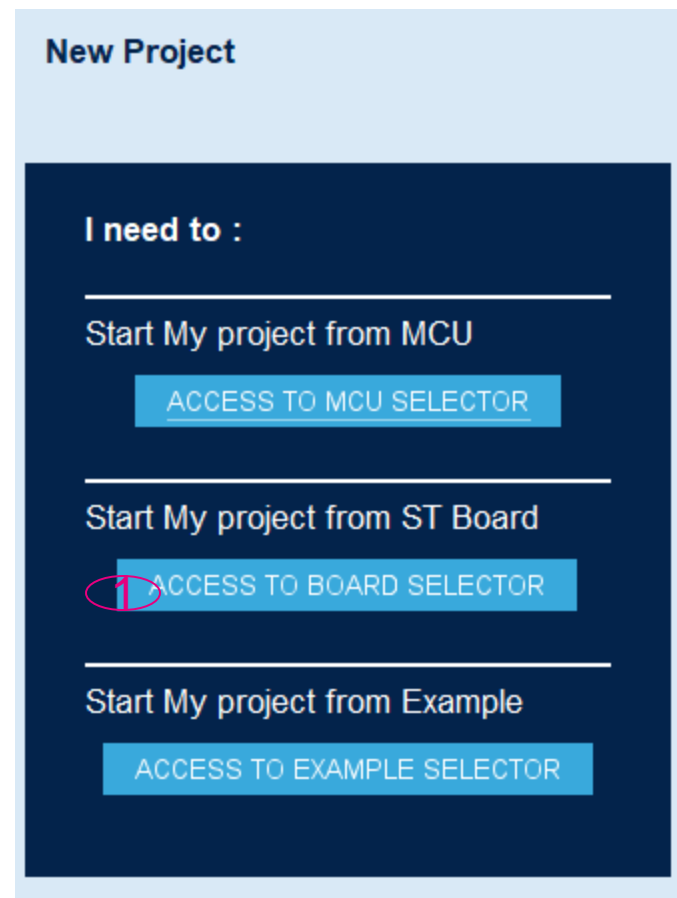
新功能：支持快速复位调试（System Reset）

STM32CubeMX创建工程



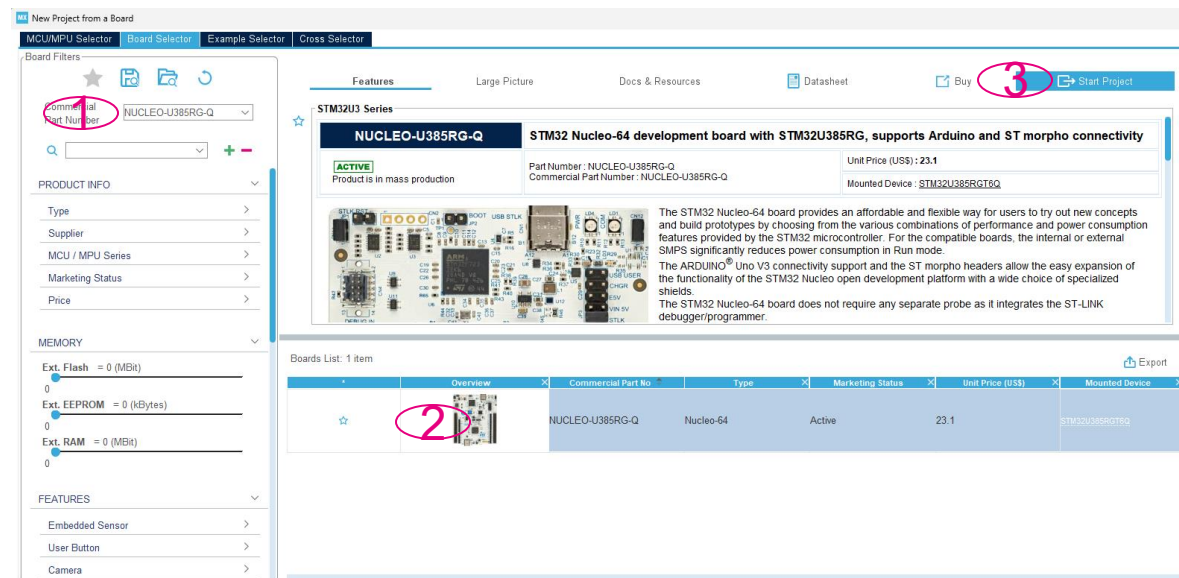
启动STM32CubeMX

- 从开发板创建工程
 - 选择【ACCESS TO BOARD SELECTOR】
- 从芯片创建工程亦可
 - 选择【ACCESS TO MCU SELECTOR】

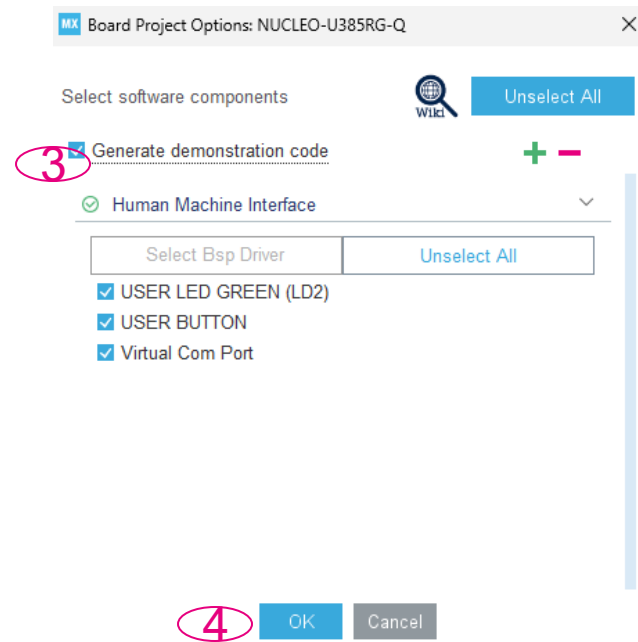
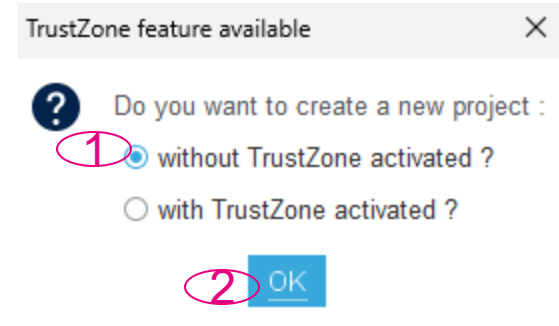


选择开发板

- 输入NUCLEO-U385RG-Q
 - 不需要完整输入，仅输入U3然后选择就可以了
- 选中该栏
- 点击【Start Project】

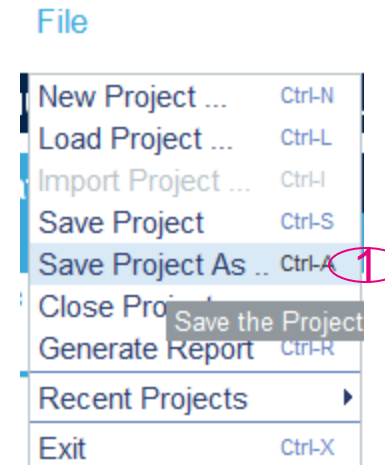


- 不选Trustzone
 - 为简化起见, 可选择不带Trustzone
 - 点击【OK】
- 选择演示代码
 - 选择生成演示代码【Generate demonstration code】
 - 点击【OK】



保存工程

- 保存工程
 - 【File】 -> 【Save Project As】
- 对于工程文件名，可以选择名字 demo_mx，表示该工程使用 STM32CubeMX



生成Cmake工程

- 选择CMake

- 【Project Manager】 -> 【Code Generator】 -> 【Toolchain/IDE】
- 使用默认编译器GCC

- 保存工程

- 【File】 -> 【Save Project】

- 生成代码

- 【Generate Code】

- 关于Cache

- 为简化这里不配置Cache。对于真实场景，一般需要打开这项功能

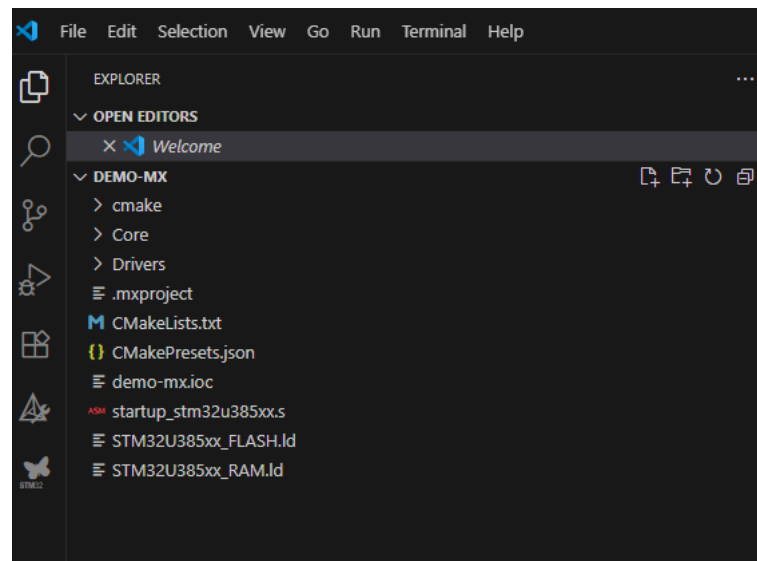
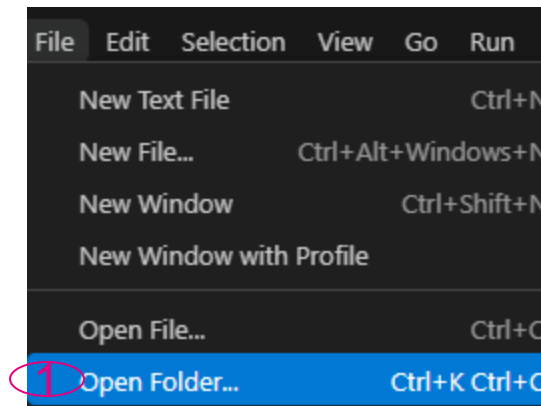
The screenshot illustrates the process of generating a CMake project in STM32CubeIDE. The interface is divided into several sections:

- Project Manager:** A blue bar at the top with a circled '1' next to the text 'Project Manager'.
- Project Tab:** A blue bar on the left with a circled '2' next to the text 'Project'.
- Code Generator:** A blue bar on the left with the text 'Code Generator'.
- Advanced Settings:** A blue bar on the left with the text 'Advanced Settings'.
- Toolchain / IDE:** A dropdown menu on the right with 'CMake' selected, circled '3'.
- Default Compiler/Linker:** A dropdown menu on the right with 'GCC' selected.
- Generate Under Root:** A checkbox on the right with the text 'Generate Under Root'.
- Generate Code:** A blue button at the top right with a circled '4' and the text 'GENERATE CODE'.
- Warning Dialog:** A dialog box at the bottom with a yellow warning icon and the text 'Warning: Code Generation'. It contains a 'WARNINGS:' section with two bullet points: '- The instruction cache (ICACHE) must be enabled to reach the maximum performance. The ICACHE can be enabled from the Pinout tab under ICACHE.' and '- SMPS must be enabled to reach the best consumption figure. To enable SMPS: 1- Go to the "Pinout and Configuration" tab. 2- Click on "RCC." 3- Under "Configuration" expand the "Power Parameters" section. 4- Select PWR_SMPS_SUPPLY from the SupplySource menu.' Below the warnings is the question 'Do you still want to generate code ?' and a circled '5' next to the 'Yes' button.

配置工程

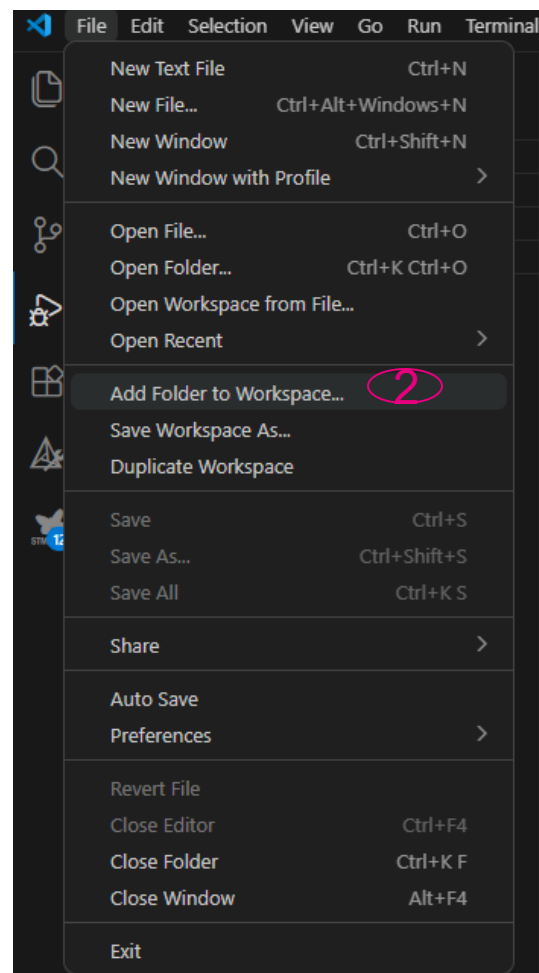
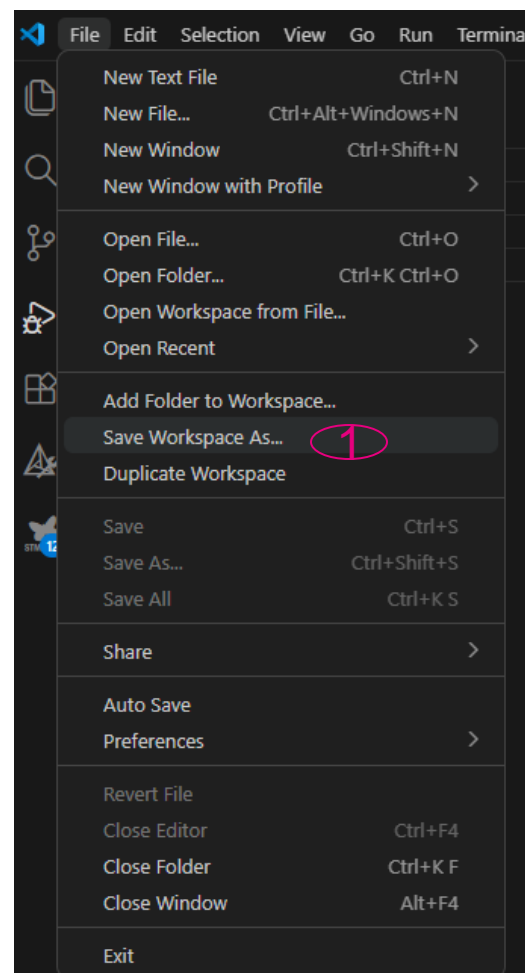
打开工程

- 使用VSCode打开工程文件夹
 - 【File】 -> 【Open Folder】
 - 选择文件目录【demo_mx】

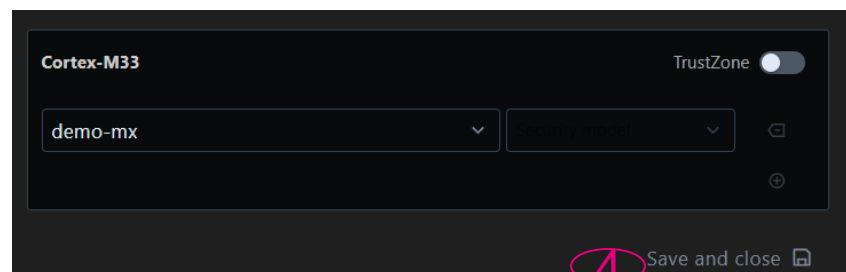
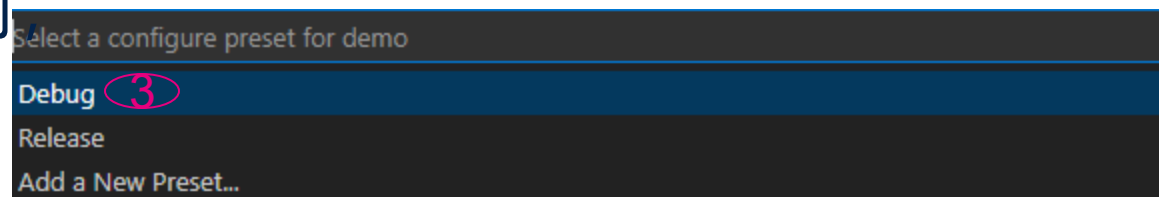
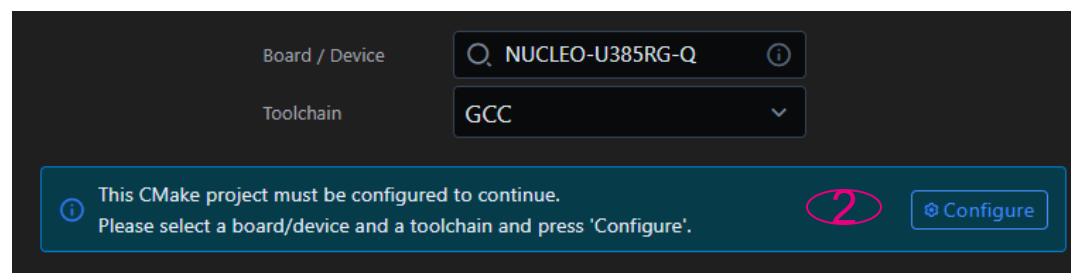
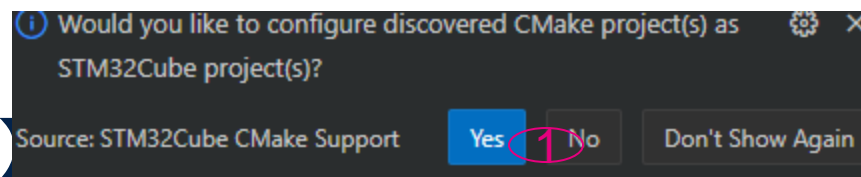


多工程管理技巧 (非必须)

- 【File】 -> 【Save Workspace As...】
保存一个Workspace, 例如:
demo.code-workspace
- 【File】 -> 【Add Folder to Workspace...】 加入另外一个工程
- 去Cmake界面设置当前活动工程
- 去Cmake界面一次编译所有工程

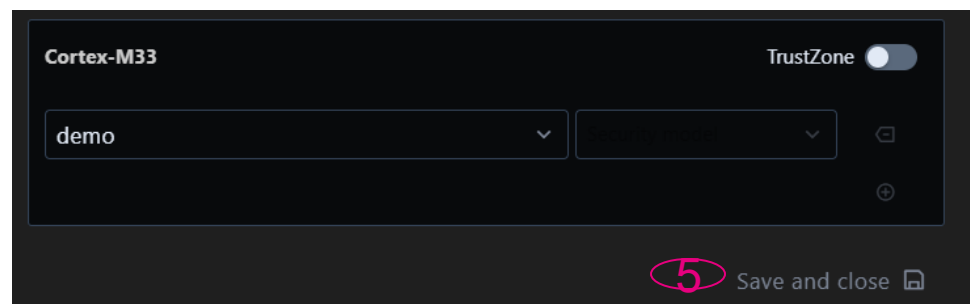
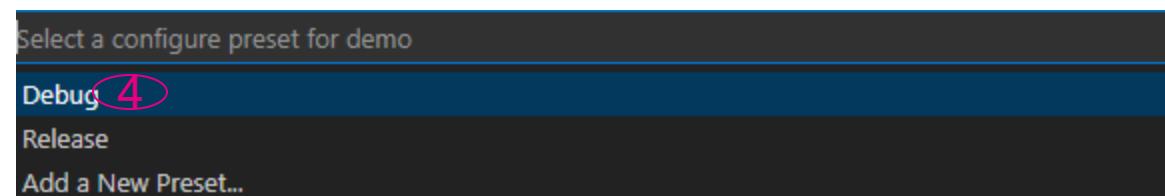
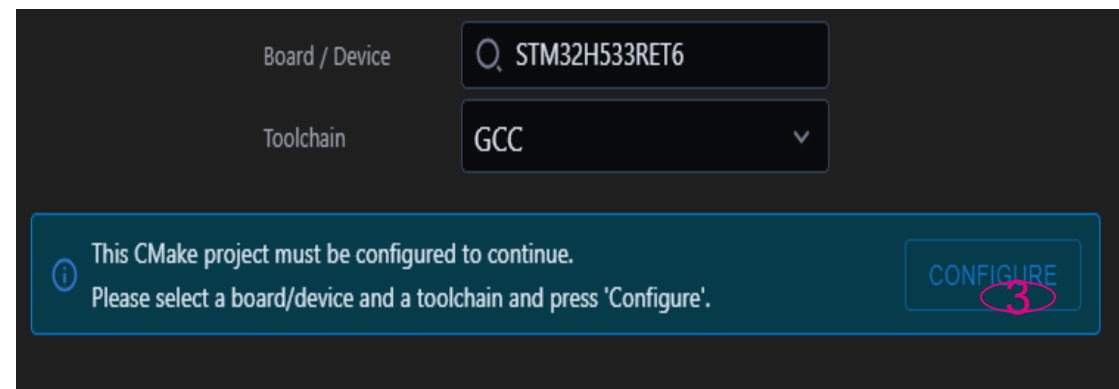
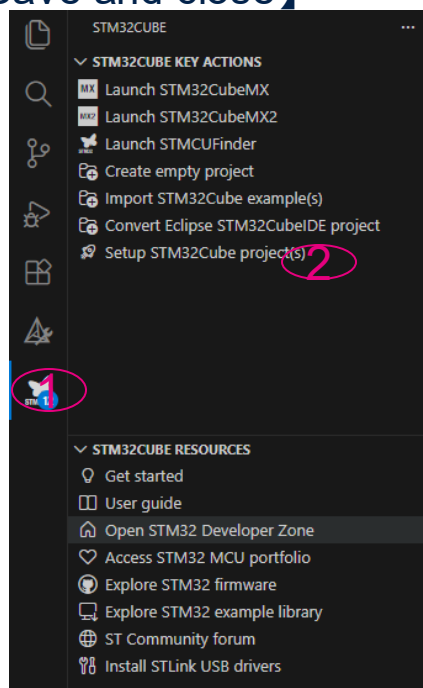


- 使用它自动弹出的对话框进行配置
- 输入U3, 选择【NUCLEO-U385RG-Q】
- 选择【CONFIGURE】
- 如果弹出【Debug】/【Release】选择时选择【Debug】
- 在弹出工程名字【demo-mx】对话框时选择【Save and close】



再次配置

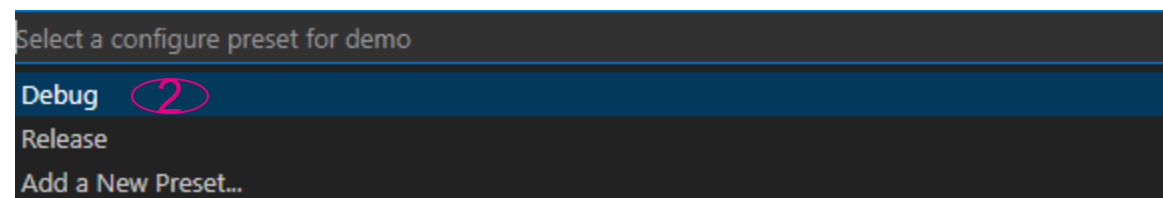
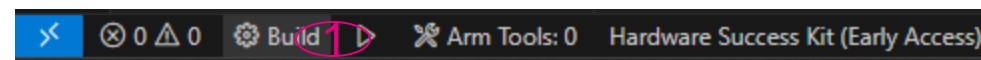
- 如果对话框消失，也可以通过之前提到的STM32Cube界面选择【Setup STM32Cube project(s)】
- 选择【CONFIGURE】
- 如果弹出【Debug】 / 【Release】选择时选择【Debug】
- 选择【Save and close】



编辑/编译/调试

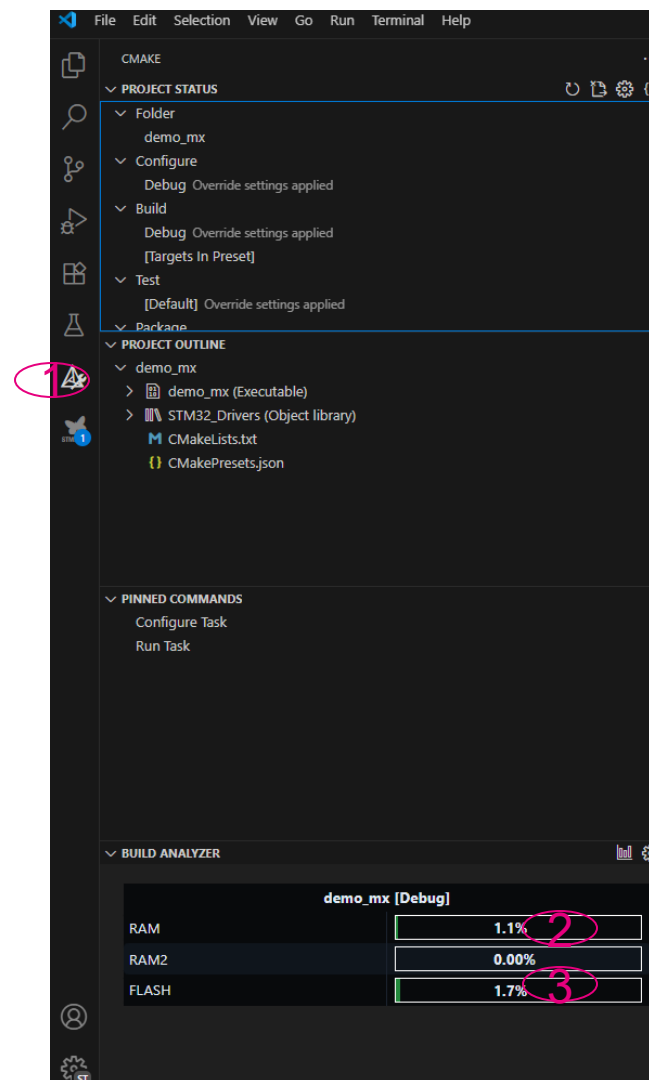
编译-Debug

- 在配置过程中已经触发编译，如果需要再次编译
 - 在界面底部，选择【Build】
 - 如果弹出配置选项，先选择【Debug】



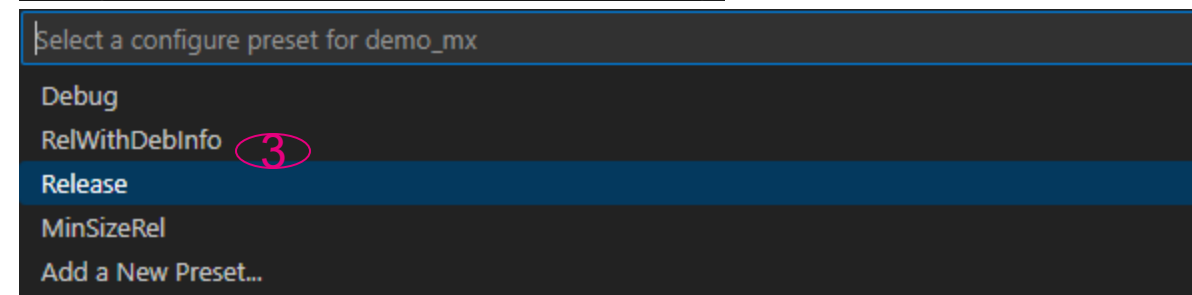
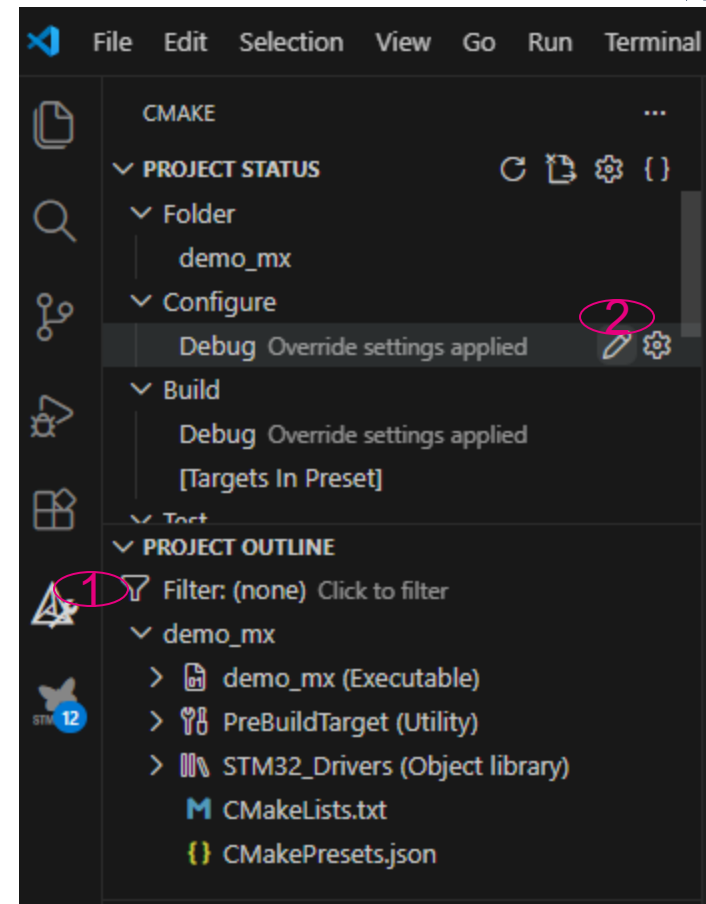
查看编译数据

- 从左边栏进入【Cmake】扩展
- 观察【Debug】下的RAM和FLASH占用分别为1.1%和1.7%



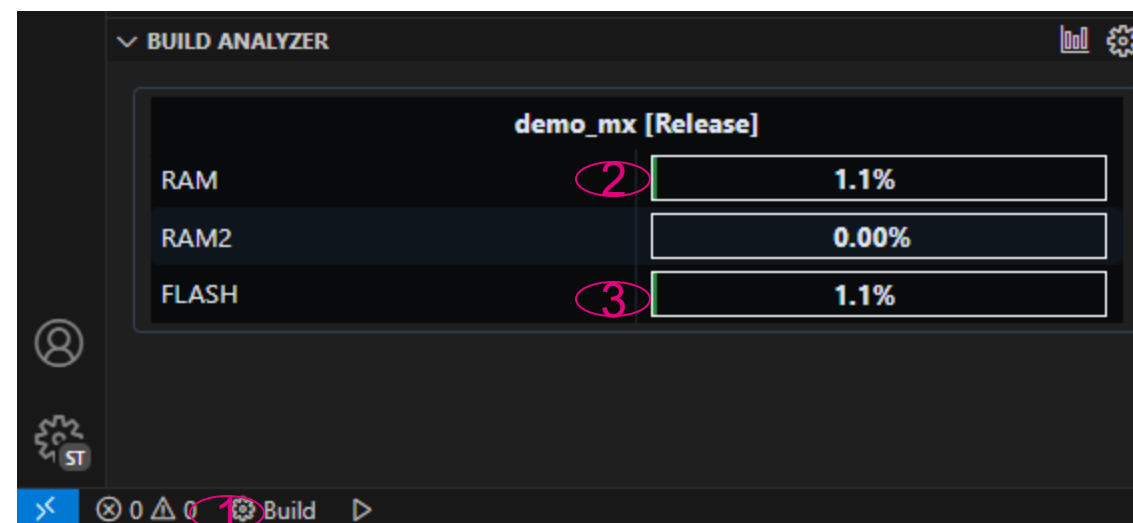
编译-Release

- 从左边栏进入【Cmake】扩展
- 选择【PROJECT STATUS】
- 选择【Configure】 -> 【Change Build Preset】
- 在弹出的配置选项中选择【Release】



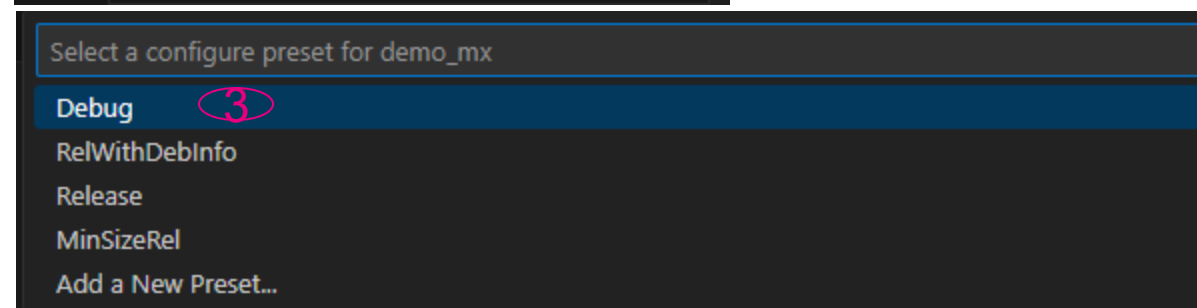
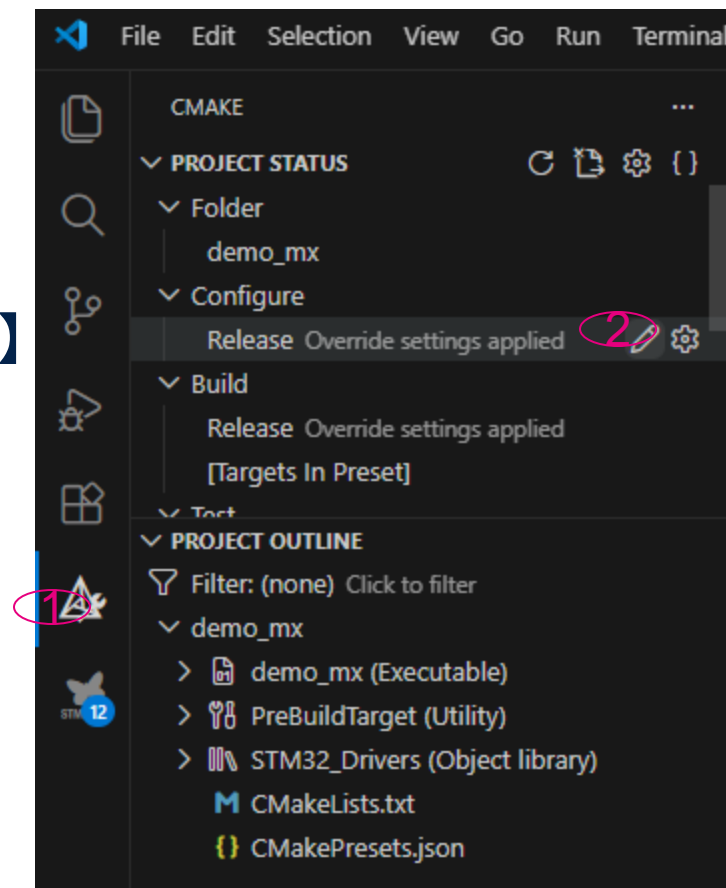
查看编译数据

- 点击底部的【Build】
- 观察【Build ANALYZER】的数据可以看到【Release】版本的RAM和FLASH占用分别为1.1%和1.1%，也就是说Release版本FLASH占用会变小
 - RAM
 - Flash



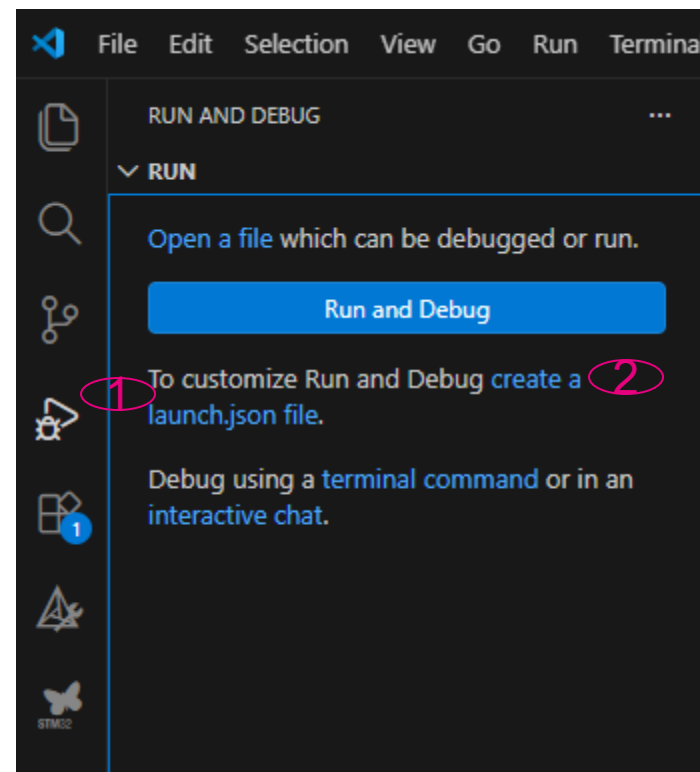
切回Debug

- 直接切换至Debug配置
 - 【CMake】 -> 【Project Status】 -> 【Configure】 -> 【Change Configure Preset】
- 在弹出菜单里选择【Debug】



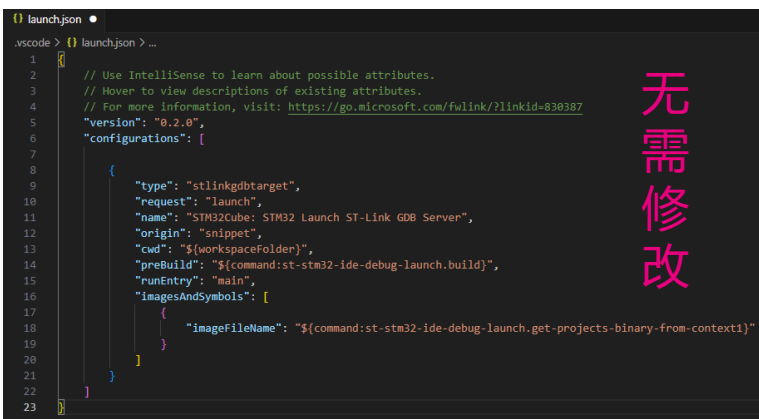
启动调试

- 启动调试请使用图标【Run and Debug(Ctrl + Shift +D)】
 - 点击【create a launch.json file】
 - 如果不想保存调试配置，也可以直接使用【Run and Debug】



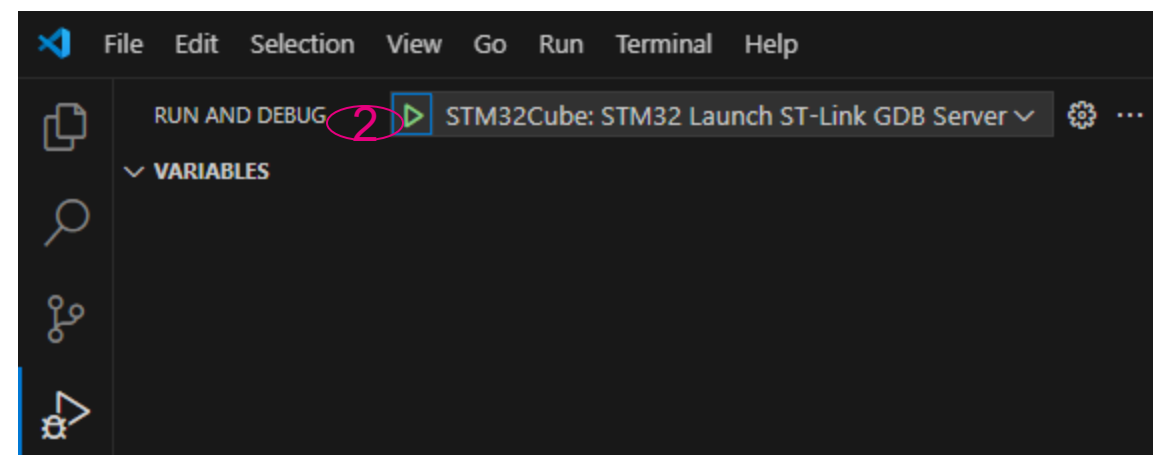
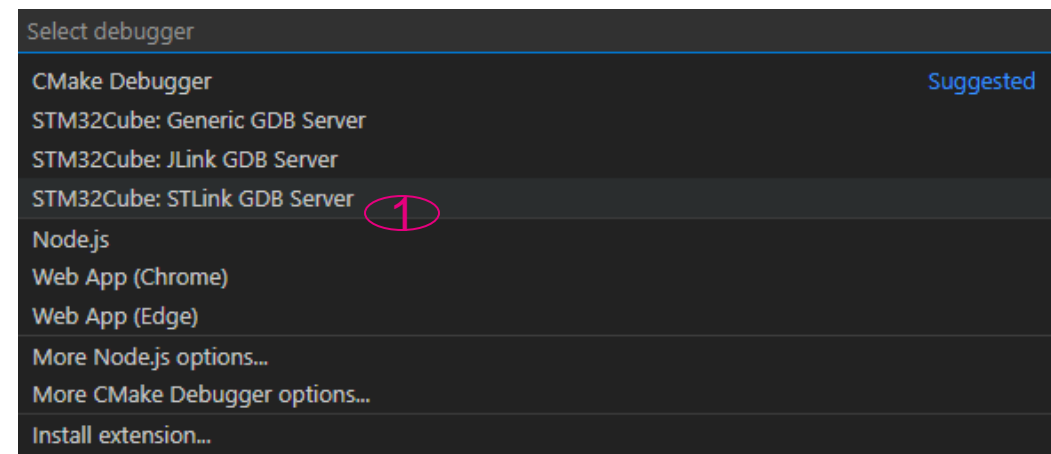
生成配置文件

- 选择【STM32Cube:STLink GDB Server】
- 对于弹出的【launch.json】文件，对于这一步无需修改
- 点击【STM32Cube: STM32 Launch ST-Link GDB Server】启动调试

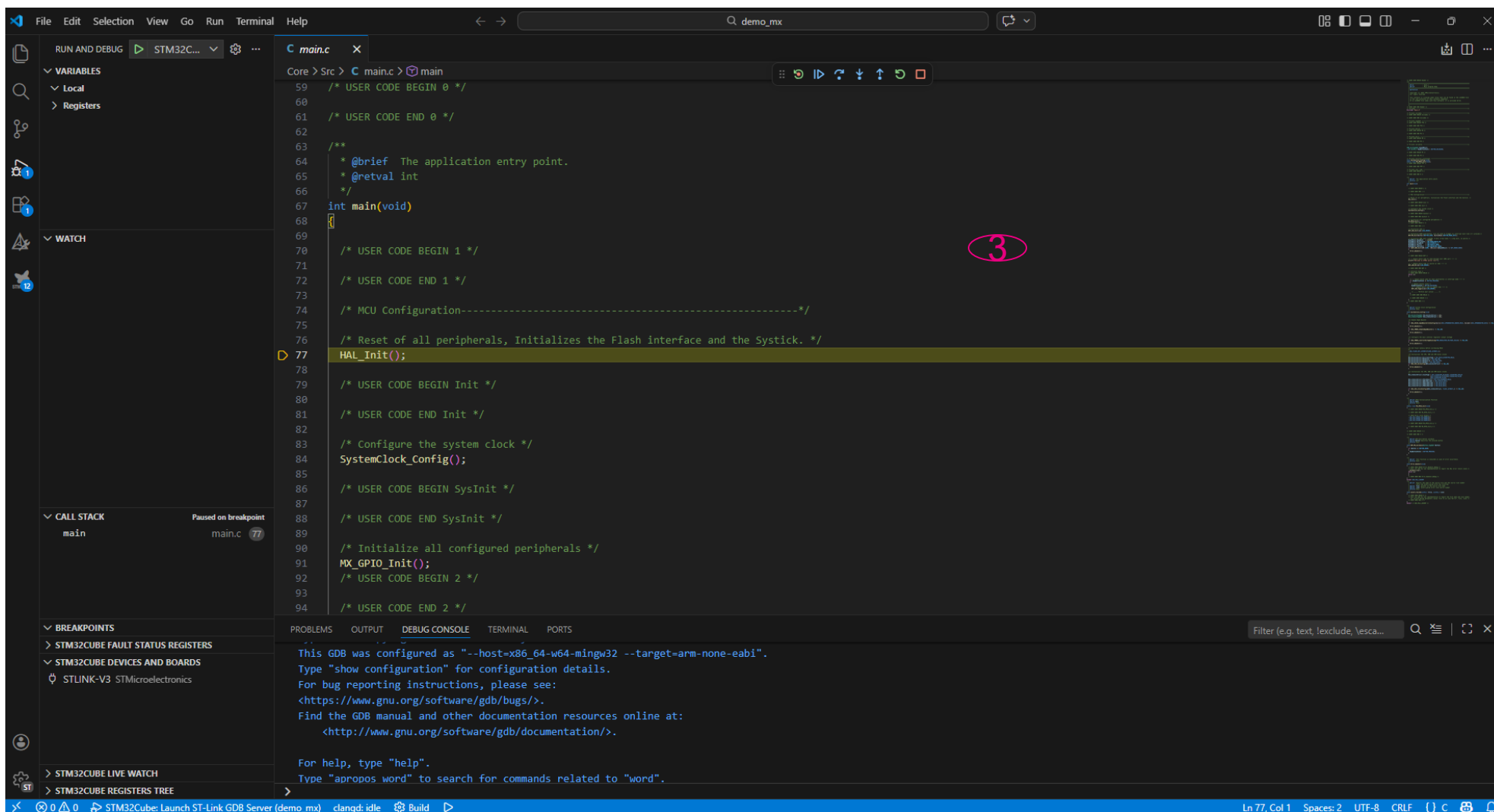


```
1 {
2   // Use IntelliSense to learn about possible attributes.
3   // Hover to view descriptions of existing attributes.
4   // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5   "version": "0.2.0",
6   "configurations": [
7
8     {
9       "type": "stlinkgdbtarget",
10      "request": "launch",
11      "name": "STM32Cube: STM32 Launch ST-Link GDB Server",
12      "origin": "onipset",
13      "cwd": "${workspaceFolder}",
14      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
15      "runEntry": "main",
16      "imagesAndSymbols": [
17        {
18          "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
19        }
20      ]
21    }
22  ]
23 }
```

无需修改

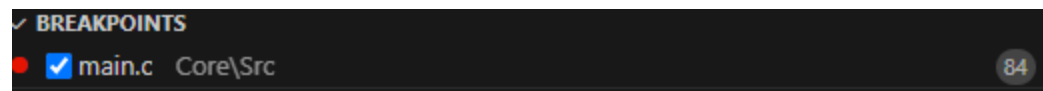
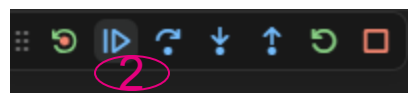
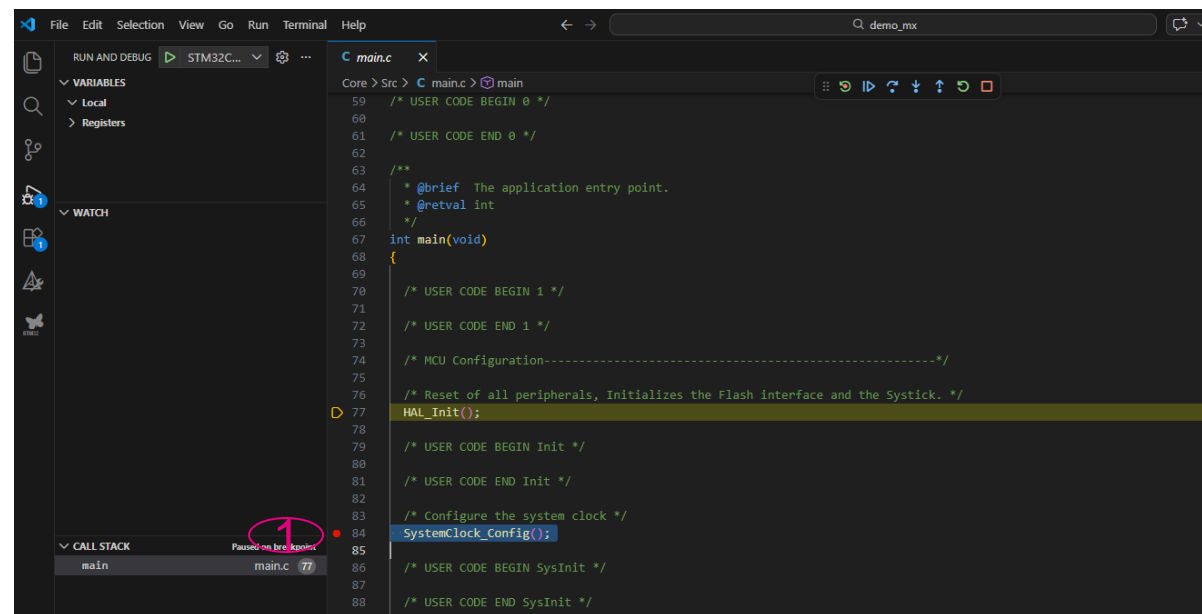


调试界面



断点与单步调试

- 在SystemClock_Config行设置断点
- 单步调试
- 进入SystemClock_Config



查看局部变量

The screenshot shows an IDE with a C program named `main.c`. The program is a system clock configuration for an STM32 microcontroller. A red circle highlights the variable `HSICalibrationValue` in the `RCC_OscInitStruct` initialization. The value of this variable is shown in the `VARIABLES` window on the left, where it is listed as `HSICalibrationValue = 0`. The code in the center shows the `main` function and the `SystemClock_Config` function. The `main` function calls `SystemClock_Config` and then enters a loop. The `SystemClock_Config` function initializes the `RCC_OscInitStruct` and `RCC_ClkInitStruct` and then calls `HAL_RCCEx_EpodbBoosterClkConfig` and `HAL_PWREx_EnableEpodbBooster`. The `main` function also calls `HAL_PWREx_ControlVoltageScaling`.

```
Core > Src > C main.c > SystemClock_Config
67 int main(void)
134     BSP_LED_Toggle(Led: LED_GREEN);
135
136     /* ..... Perform your action ..... */
137 }
138 /* USER CODE END WHILE */
139
140 /* USER CODE BEGIN 3 */
141
142 /* USER CODE END 3 */
143 }
144
145 /**
146  * @brief System Clock Configuration
147  * @retval None
148  */
149 void SystemClock_Config(void)
150 {
151     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
152     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
153
154     /* Enable Epod Booster
155     */
156     if (HAL_RCCEx_EpodbBoosterClkConfig(Source: RCC_EP0DB00STER_SOURCE_MSIS, Divider: RCC_EP0DB00STER_DIV1) != HAL_OK)
157     {
158         Error_Handler();
159     }
160     if (HAL_PWREx_EnableEpodbBooster() != HAL_OK)
161     {
162         Error_Handler();
163     }
164
165     /* Configure the main internal regulator output voltage
166     */
167     if (HAL_PWREx_ControlVoltageScaling(VoltageScaling: PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
168     {
```

VARIABLES

- Local
 - `RCC_OscInitStruct = {...}`
 - `OscillatorType = 0`
 - `HSEState = 0`
 - `HSIState = 0`
 - `HSICalibrationValue = 0`
 - `HSI48State = 0`
 - `LSEState = 0`
 - `LSIState = 0`
 - `LSIDiv = 0`
 - `MSISState = 0`
 - `MSISSource = 0`
 - `MSISDiv = 0`
 - `MSIKState = 0`
 - `MSIKSource = 0`
 - `MSIKDiv = 0`
 - `RCC_ClkInitStruct = {...}`
- Registers

WATCH

- `CALL STACK` (Paused on step)
- `BREAKPOINTS`
- `STM32CUBE FAULT STATUS REGISTERS`
- `STM32CUBE DEVICES AND BOARDS`
- `STM32CUBE LIVE WATCH`
- `STM32CUBE REGISTERS TREE`

PROBLEMS

Filter (e.g. text, exclude, Vesca...)

This GDB was configured as "--host=x86_64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<<https://www.gnu.org/software/gdb/bugs/>>.
Find the GDB manual and other documentation resources online at:
<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".
Type "apropos word" to search for commands related to "word".

Ln 156, Col 1 Spaces: 2 UTF-8 CRLF {} C

查看寄存器

The screenshot displays a development environment for STM32Cube, showing the GDB interface. The left sidebar contains the 'Registers' panel, which lists the current state of various registers. A red circle highlights register **r1**, which contains the value **0x0**. The main editor shows the **main.c** file, specifically the **SystemClock_Config** function. The code is written in C and includes comments in Chinese. The bottom status bar indicates the current line and column: **Ln 182, Col 1**.

Registers:

- r0 = 0x0
- r1 = 0x0
- r2 = 0x40022000
- r3 = 0x0
- r4 = 0x0
- r5 = 0x0
- r6 = 0x0
- r7 = 0x2002ffa0
- r8 = 0xffffffff
- r9 = 0xffffffff
- r10 = 0xffffffff
- r11 = 0xffffffff
- r12 = 0xffffffff
- sp = 0x2002ffa0
- lr = 0x800329f
- pc = 0x80032c8
- xpsr = 0x61000000
- d0 = 0x0
- d1 = 0x0
- d2 = 0x0
- d3 = 0x0
- d4 = 0x0
- d5 = 0x0
- d6 = 0x0
- d7 = 0x0
- d8 = 0x0
- d9 = 0x0
- d10 = 0x0
- d11 = 0x0
- d12 = 0x0
- d13 = 0x0

main.c:

```
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};

    /** Enable Epod Booster
     */
    if (HAL_RCCEx_EpodBoosterClkConfig(Source: RCC_EPODBOOSTER_SOURCE_MSIS, Divider: RCC_EPODBOOSTER_DIV1) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_PWREx_EnableEpodBooster() != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure the main internal regulator output voltage
     */
    if (HAL_PWREx_ControlVoltageScaling(VoltageScaling: PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Set Flash latency before increasing MSIS
     */
    __HAL_FLASH_SET_LATENCY(FLASH_LATENCY_2);

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSIS;
    RCC_OscInitStruct.MSISState = RCC_MSI_ON;
    RCC_OscInitStruct.MSISSource = RCC_MSI_RC0;
    RCC_OscInitStruct.MSISDiv = RCC_MSI_DIV1;
    if (HAL_RCC_OscConfig(RCC_OscInitStruct, &RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}
```

查看调用栈

The screenshot shows an IDE interface with a call stack on the left and C code in the center. The call stack, titled 'CALL STACK', shows a 'Paused on step' state with two entries: 'SystemClock_Config' at line 182 in 'main.c' and 'main' at line 184 in 'main.c'. The 'main.c' entry is highlighted with a red circle. The main code editor displays the 'SystemClock_Config' function, which includes comments and code for enabling the Epod Booster, configuring the main internal regulator output voltage, setting Flash latency, and initializing the CPU, AHB, and APB buses clocks. The function is currently paused at line 182, which is highlighted in yellow. The bottom status bar indicates the current line is 182, column 1, with 2 spaces, UTF-8 encoding, and CRLF line endings.

```
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};

    /** Enable Epod Booster
    */
    if (HAL_RCCEx_EpodBoosterClkConfig(Source: RCC_EP0DB00STER_SOURCE_MSIS, Divider: RCC_EP0DB00STER_DIV1) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_PWREx_EnableEpodBooster() != HAL_OK)
    {
        Error_Handler();
    }

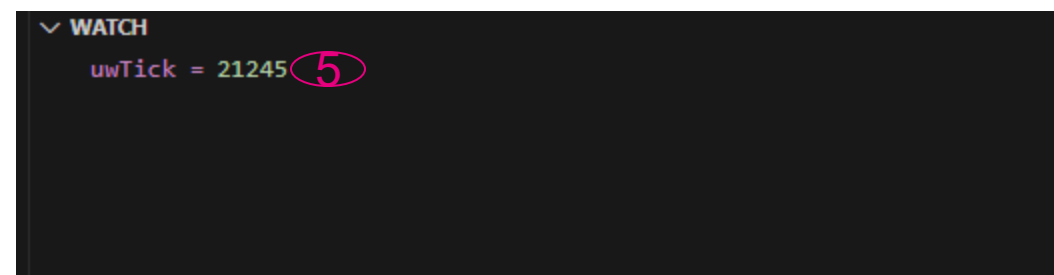
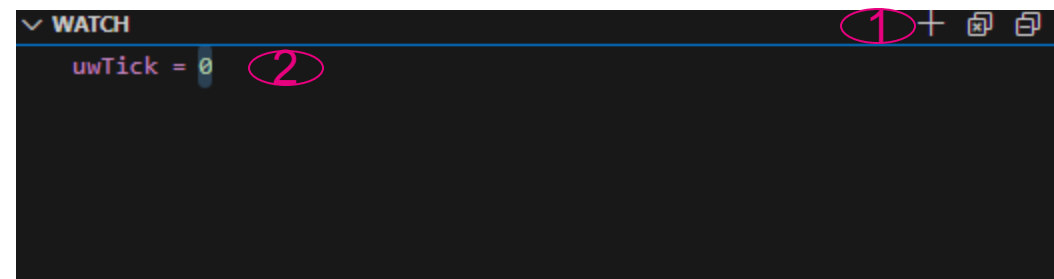
    /** Configure the main internal regulator output voltage
    */
    if (HAL_PWREx_ControlVoltageScaling(VoltageScaling: PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Set Flash latency before increasing MSIS
    */
    __HAL_FLASH_SET_LATENCY(FLASH_LATENCY_2);

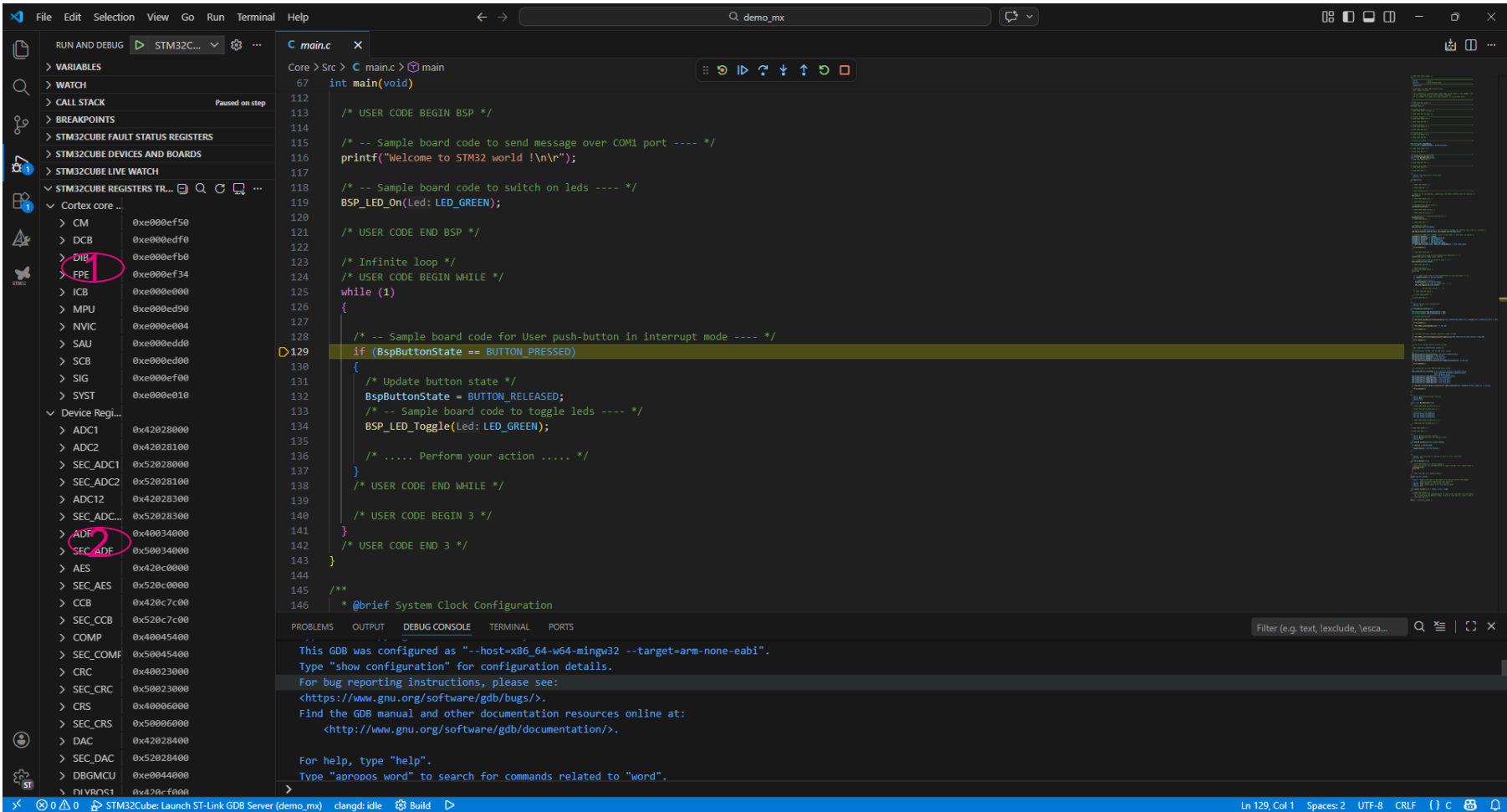
    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSIS;
    RCC_OscInitStruct.MSISState = RCC_MSI_ON;
    RCC_OscInitStruct.MSISSource = RCC_MSI_RC0;
    RCC_OscInitStruct.MSISDiv = RCC_MSI_DIV1;
    if (HAL_RCC_OscConfig(RCC_OscInitStruct: &RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}
```

查看全局变量

- 点击【WATCH】->【+】添加全局变量uwTick
- 继续执行
- 中断
- 可以看到uwTick的当前值



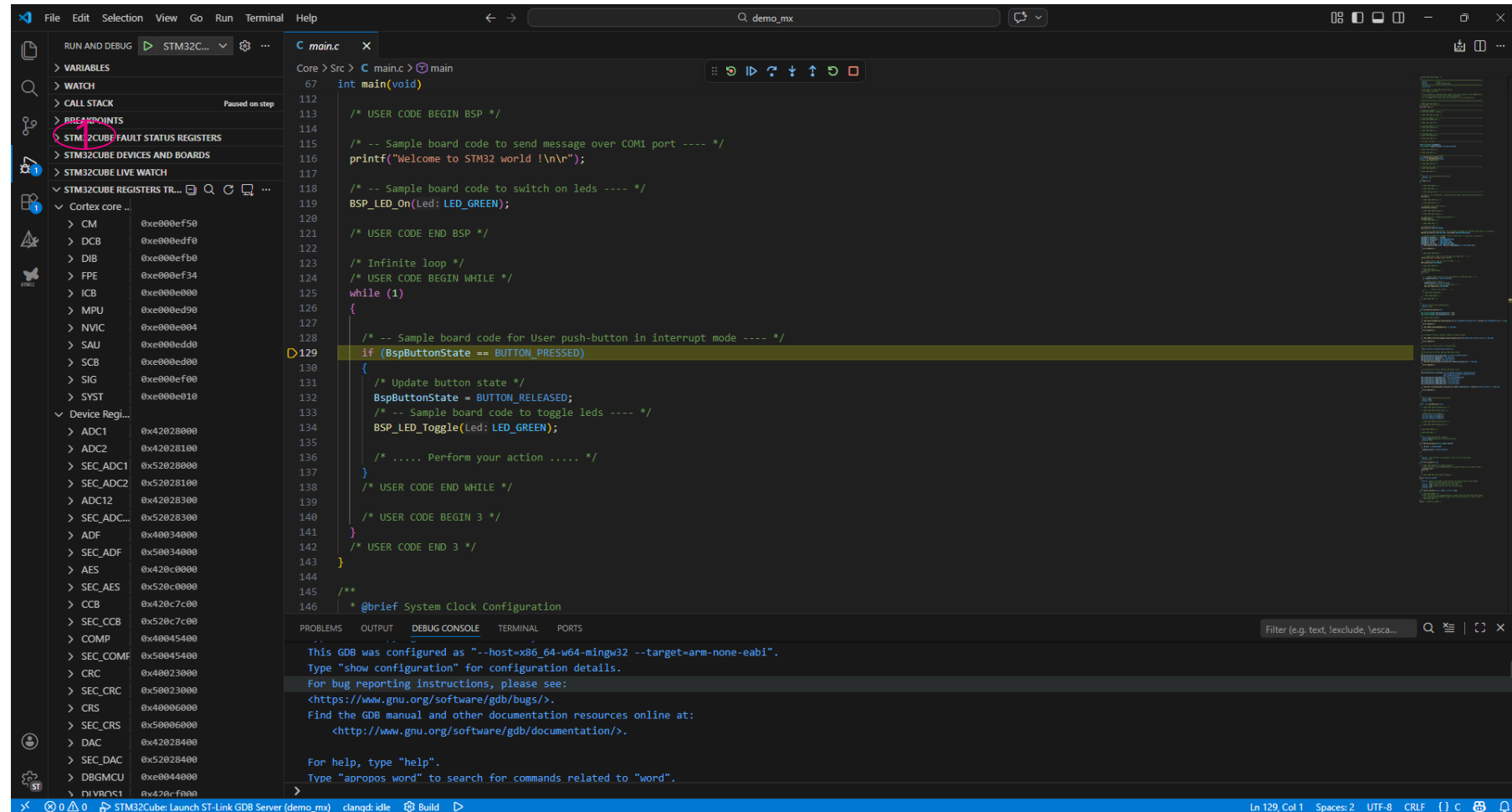
查看外设寄存器



新功能：查看核心寄存器（Cortex Core Register）

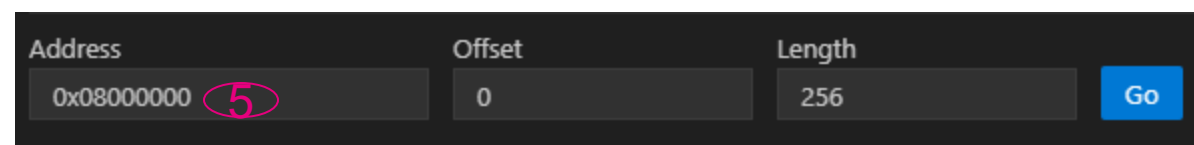
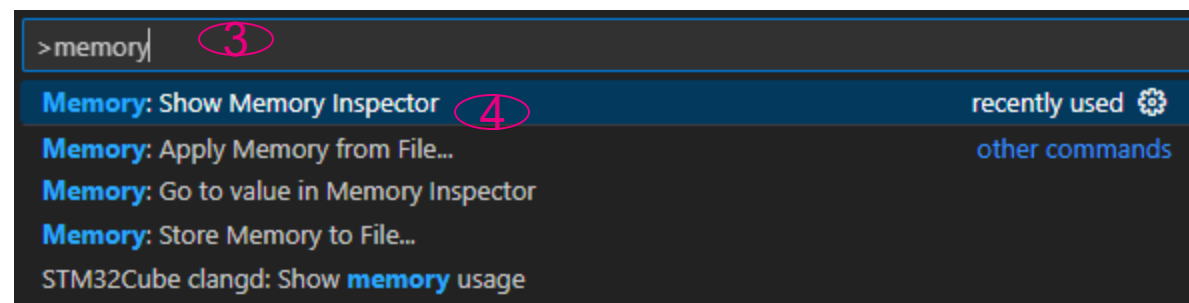
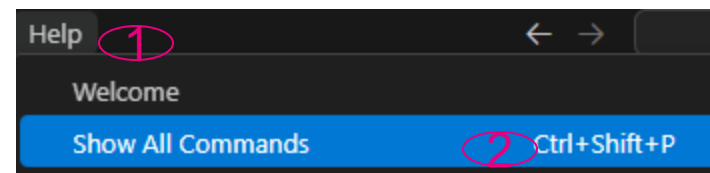


Fault 分析



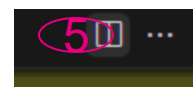
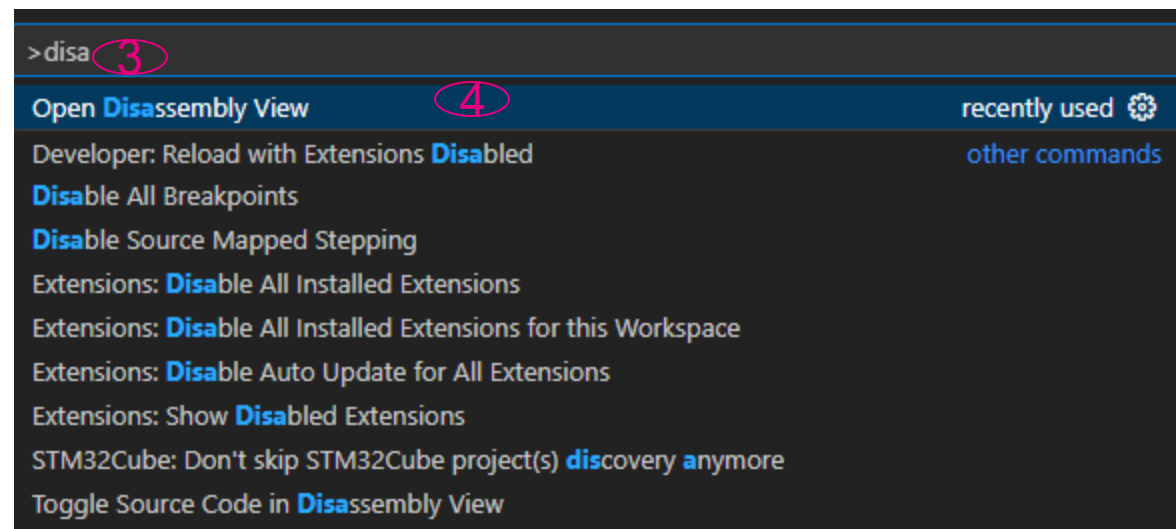
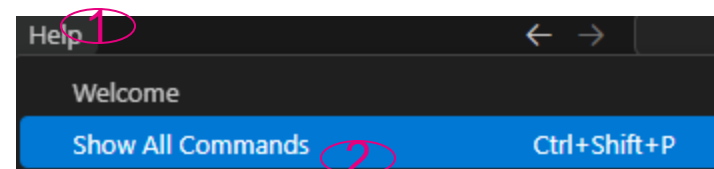
查看内存

- 显示命令菜单
 - 【Help】->【Show All commands】
- 输入【memory】
- 选择【Memory:Show Memory Inspector】
- 在【Address】里输入0x0800 0000



Address	Data
0x8000000	00 00 03 20
0x8000004	a1 35 00 08
0x8000008	b9 33 00 08
0x800000c	c1 33 00 08

- 显示命令菜单
 - 【Help】->【Show All commands】
- 输入【disa】
- 选择【Open Disassembly View】
- 在屏幕右上点击【split】
- 关闭多余窗口



汇编调试效果

The screenshot displays the STM32Cube IDE interface with the following components:

- Left Panel:** Contains 'VARIABLES' (Local, Registers), 'WATCH', 'CALL STACK' (Paused on SIGINT, main, main.c 129), 'BREAKPOINTS', 'STM32CUBE FAULT STATUS REGISTERS', 'STM32CUBE DEVICES AND BOARDS' (STLINK-V3 STMicrelectronics), 'STM32CUBE LIVE WATCH', and 'STM32CUBE REGISTERS TREE'.
- Source Editor:** Shows the C file 'main.c' with the following code:

```
67 int main(void)
112
113 /* USER CODE BEGIN BSP */
114
115 /* -- Sample board code to send message over COM1 port ---- */
116 printf("Welcome to STM32 world !\n\r");
117
118 /* -- Sample board code to switch on leds ---- */
119 BSP_LED_On(Led: LED_GREEN);
120
121 /* USER CODE END BSP */
122
123 /* Infinite loop */
124 /* USER CODE BEGIN WHILE */
125 while (1)
126 {
127
128 /* -- Sample board code for User push-button in interrupt mode ---- */
129 if (BspButtonState == BUTTON_PRESSED)
130 {
131 /* Update button state */
132 BspButtonState = BUTTON_RELEASED;
133 /* -- Sample board code to toggle leds ---- */
134 BSP_LED_Toggle(Led: LED_GREEN);
135
136 /* ..... Perform your action ..... */
137 }
138 /* USER CODE END WHILE */
139
140 /* USER CODE BEGIN 3 */
141 }
142 /* USER CODE END 3 */
143
144 /**
145 * @brief System Clock Configuration
146 */
```

The line number '129' is circled in red in the original image.
- Disassembly View:** Shows the assembly code corresponding to the C code, with the instruction at address 0x08003230 highlighted:

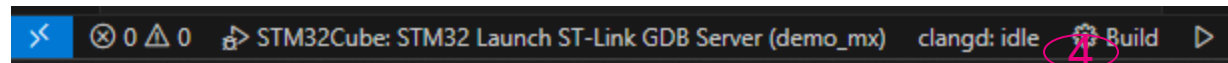
```
0x08003230 07 4b ldr r3, [pc, #28] @ (0x8003250 <main+...>
0x08003232 1b 68 ldr r3, [r3, #0]
0x08003234 01 2b cmp r3, #1
0x08003236 fb d1 bne.n 0x8003230 <main+92>
132: BspButtonState = BUTTON_RELEASED;
0x08003238 05 4b ldr r3, [pc, #20] @ (0x8003250 <main+...>
0x0800323a 00 22 movs r2, #0
0x0800323c 1a 60 str r2, [r3, #0]
134: BSP_LED_Toggle(LED_GREEN);
0x0800323e 00 20 movs r0, #0
0x08003240 ff f7 3c fe bl 0x8002ebc <BSP_LED_Toggle>
129: if (BspButtonState == BUTTON_PRESSED)
0x08003244 f4 e7 b.n 0x8003230 <main+92>
0x08003246 00 bf nop
0x08003248 40 01 lsls r0, r0, #5
0x0800324a 00 20 movs r0, #0
0x0800324c 8c 44 add r12, r1
0x0800324e 00 08 lsr r0, r0, #32
0x08003250 00 00 ldr r0, #0
```
- Bottom Panel:** Contains 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'DEBUG CONSOLE' shows the GDB configuration and help text.

- 可以使用B站查看vscode技巧
- 可以使用微软网站查看vscode技巧
- 点击调试停止按钮
- 输入”pr”选择printf加入语句
 - printf("LED is toggled");
- 可以看到函数自动完成
- 重新编译



```
128  /* -- Sample board code for User push-button in interrupt mode ---- */
129  if (BspButtonState == BUTTON_PRESSED)
130  {
131      /* Update button state */
132      BspButtonState = BUTTON_RELEASED;
133      /* -- Sample board code to toggle leds ---- */
134      BSP_LED_Toggle(Led: LED_GREEN);
135      pr
136      /* printf(const char *restrict, ...)
```

```
3 printf("LED is toggled");
```



函数浏览

- 右键点击函数BSP_LED_Toggle
- 使用【Go to Definition】跳到实现
- 使用【Go to Declaration】跳到头文件
- 使用【Find All References】得到调用信息

```
while (1)
{
    /* -- Sample board code for User push-button */
    if (BspButtonState == BUTTON_PRESSED)
    {
        /* Update button state */
        BspButtonState = BUTTON_RELEASED;
        /* -- Sample board code to toggle leds -- */
        BSP_LED_Toggle(Led: LED_GREEN);
        printf("LED is toggled");
        /* ..... Perform your action ..... */
    }
}
```

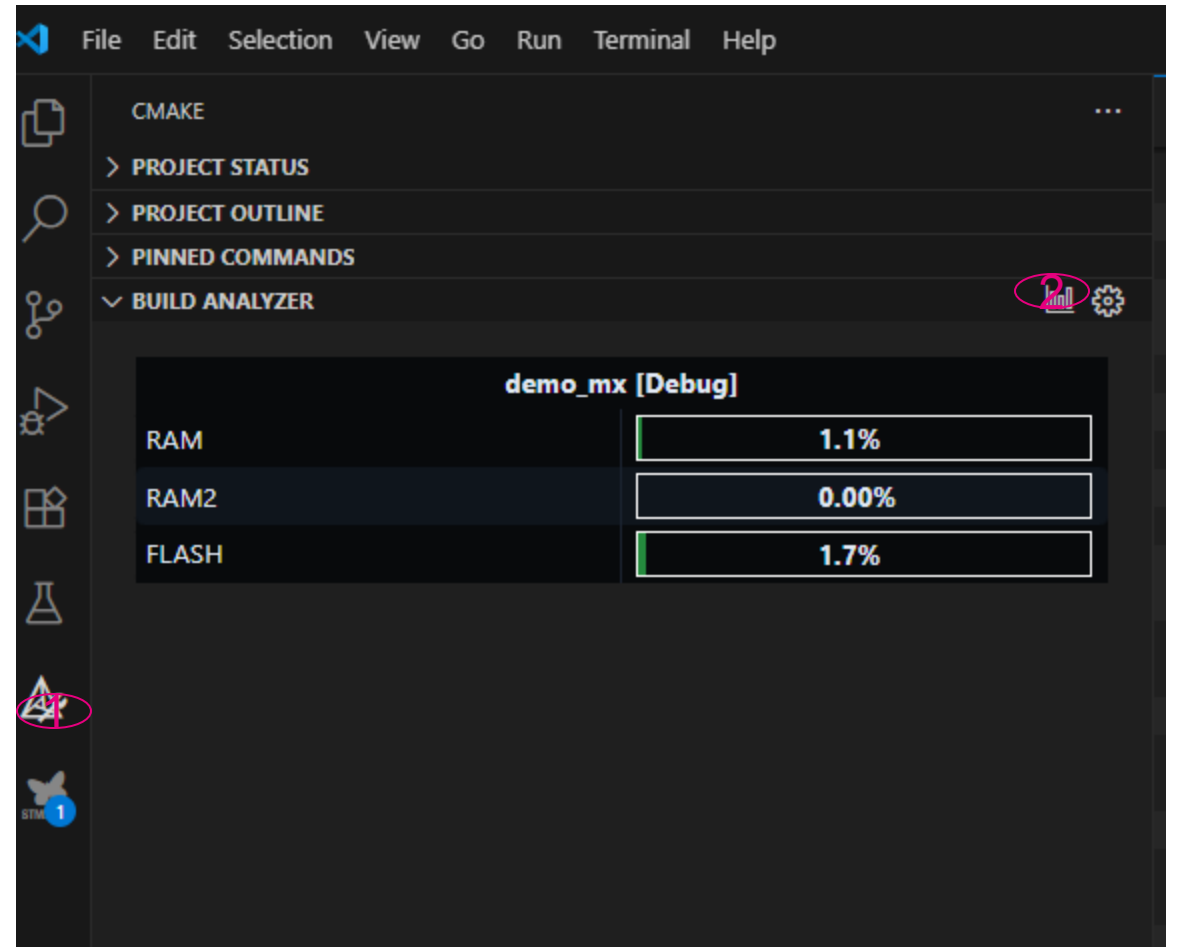
Go to Definition	F12
Go to Declaration	
Go to Type Definition	
Go to Implementations	Ctrl+F12
Go to References	Shift+F12
Peek	>
Find All References	Shift+Alt+F12
Find All Implementations	
Show Call Hierarchy	Shift+Alt+H
Show Type Hierarchy	
Switch Between Source/Header	Alt+O
Rename Symbol	F2
Change All Occurrences	Ctrl+F2
Format Document	Shift+Alt+F
Format Document With...	
Format Selection	Ctrl+K Ctrl+F
Refactor...	Ctrl+Shift+R
Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Command Palette...	Ctrl+Shift+P
Show AST	

高级主题

Build分析

找出占用Flash最大的函数

- 通过双击【CMake】->【BUILD ANALYZER】找到代码中memory的详细信息
- 通过点击按钮可对数据进行升序与降序排序，以找出最大最小值



找出占用Flash最大的函数

demo_mx [Debug] X

Build analysis:

Build description: demo_mx [Debug]
File path: ... emo/demo_mx/build/Debug/demo_mx.map
File date: Thu Mar 12 2026 10:19:21 GMT+0800 (China Standard Time)
Tool chain: ARM-GCC gnu-tools-for-stm32 13.3.1+st.9

Memory regions:

Regions	Start address	End address	% used	Alert	Status	Used size	Free size	Max size
RAM	0x20000000	0x20030000	1.1%	✓	% used < 75%	2216	194392	192 KB
RAM2	0x20030000	0x20040000	0.00%	✓	% used < 75%		65536	64 KB
FLASH	0x08000000	0x08100000	1.7%	✓	% used < 75%	17696	1030880	1 MB

Elements display type: ☒ Tree ☐ Array ☐ Graph

Memory element tree: > Expand all < Collapse all

Name	Type	Object	Library	Address	Size
FLASH	region			0x08000000	1048576
.text	section			0x08000234	16984
HAL_RCCEx_GetPeriphCLKFreq	function	stm32u3xx_hal_rcc_exc.obj	... rs.dir/_/_/Drivers/STM32U3xx_HAL_Driver/Src	0x08001588	2484
HAL_RCC_OscConfig	function	stm32u3xx_hal_rcc.obj	... rs.dir/_/_/Drivers/STM32U3xx_HAL_Driver/Src	0x08000b5c	1708
__udivmoddi4	function	_udivmoddi4.o	libgcc.a	0x080002a4	840
HAL_GPIO_Init	function	stm32u3xx_hal_gpio.obj	... rs.dir/_/_/Drivers/STM32U3xx_HAL_Driver/Src	0x08001fc4	788
UART_SetConfig	function	stm32u3xx_hal_uart.obj	... rs.dir/_/_/Drivers/STM32U3xx_HAL_Driver/Src	0x080026e8	744
_printf_i	function	libc_a-nano-vfprintf_i.o	libg_nano.a	0x08003e74	572
vfprintf_r	function	libc_a-nano-vfprintf.o	libg_nano.a	0x08003b60	560

双击进入相应的函数

demo_mx [Debug] X C stm32u3xx_hal_rcc_exc

Build analysis:

Build description: demo_mx [Debug]
File path: ...emo/demo_mx/build/Debug/demo_mx.map
File date: Thu Mar 12 2026 10:19:21 GMT+0800 (China Standard Time)
Tool chain: ARM-GCC gnu-tools-for-stm32 13.3.1+st9

Memory regions:

Regions	Start address	End address	% used	Alert	Status	Used size	Free size	Max size
RAM	0x20000000	0x20030000	1.1%	✓	% used < 75%	2216	194392	192 KB
RAM2	0x20030000	0x20040000	0.00%	✓	% used < 75%	65536	65536	64 KB
FLASH	0x08000000	0x08100000	1.7%	✓	% used < 75%	17696	1030880	1 MB

Elements display type: ☒ Tree ☐ Array ☐ Graph

Memory element tree: > Expand all < Collapse all

Name	Type	Object	Library	Address	Size
FLASH	region			0x08000000	1048576
Text	section			0x08000234	16984
HAL_RCCEx_GetPeriphCLKFreq	function	stm32u3xx_hal_rcc_exc.obj	...rsdrt/_/_Drivers/STM32U3xx_HAL_Driver/Src	0x08001588	2464
HAL_RCCEx_GetPeriphCLKFreq	function	stm32u3xx_hal_rcc_exc.obj	...rsdrt/_/_Drivers/STM32U3xx_HAL_Driver/Src	0x08000b5c	1708
__divmod4	function	_udivmod4.o	libgcc.a	0x080002a4	840
HAL_GPIO_Init	function	stm32u3xx_hal_gpio.obj	...rsdrt/_/_Drivers/STM32U3xx_HAL_Driver/Src	0x080011f4	788
UART_SetConfig	function	stm32u3xx_hal_uart.obj	...rsdrt/_/_Drivers/STM32U3xx_HAL_Driver/Src	0x080026e8	744
_printf_l	function	libc_s-nano-vprintf.o	libg_nano.a	0x08003474	572
__main	function	libc_s-nano-__main.o	libg_nano.a	0x08003474	572

Click to open source file with definition

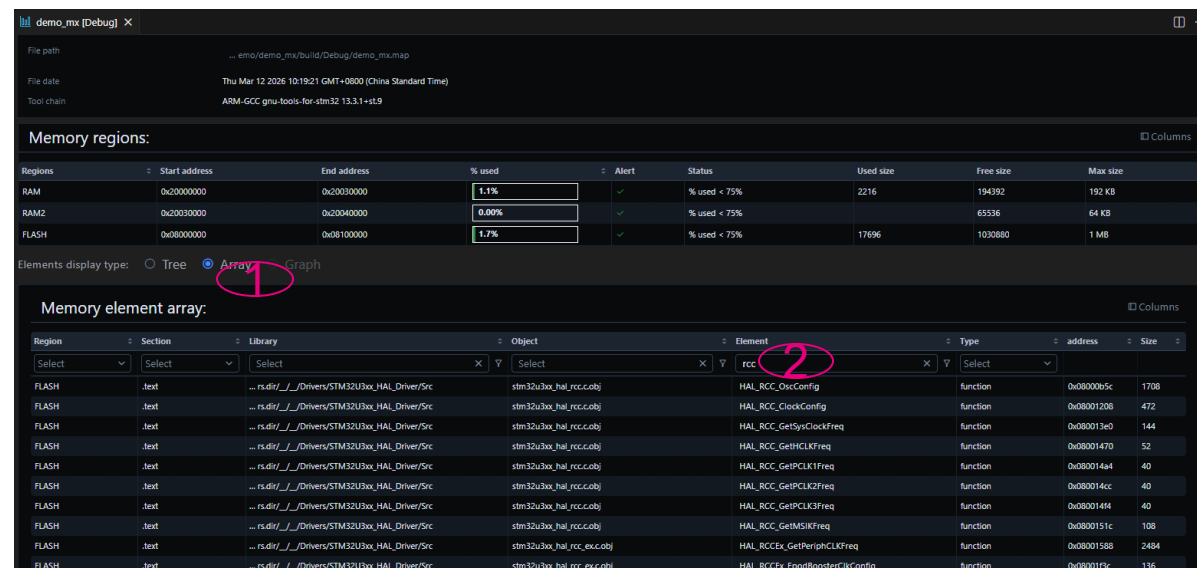
demo_mx [Debug] C stm32u3xx_hal_rcc_exc X

Drivers > STM32U3xx_HAL_Driver > Src > C stm32u3xx_hal_rcc_exc > HAL_RCCEx_GetPeriphCLKFreq

```
672 * @arg @ref RCC_PERIPHCLK_UART5 UART5 peripheral clock
673 * @arg @ref RCC_PERIPHCLK_I2C1 I2C1 peripheral clock
674 * @arg @ref RCC_PERIPHCLK_I2C2 I2C2 peripheral clock
675 * @arg @ref RCC_PERIPHCLK_I2C2 I2C2 peripheral clock (*)
676 * @arg @ref RCC_PERIPHCLK_I3C2 I3C2 peripheral clock (*)
677 * @arg @ref RCC_PERIPHCLK_SPI2 SPI2 peripheral clock
678 * @arg @ref RCC_PERIPHCLK_LPTIM2 LPTIM2 peripheral clock
679 * @arg @ref RCC_PERIPHCLK_SPI1 SPI1 peripheral clock
680 * @arg @ref RCC_PERIPHCLK_SYSTICK SYSTICK peripheral clock
681 * @arg @ref RCC_PERIPHCLK_FDCAN FDCAN peripheral clock (*)
682 * @arg @ref RCC_PERIPHCLK_ICLK ICLK peripheral clock
683 * @arg @ref RCC_PERIPHCLK_USB1 USB1 peripheral clock
684 * @arg @ref RCC_PERIPHCLK_TIMIC TIMIC peripheral clock
685 * @arg @ref RCC_PERIPHCLK_ADF1 ADF1 peripheral clock (*)
686 * @arg @ref RCC_PERIPHCLK_SPI3 SPI3 peripheral clock
687 * @arg @ref RCC_PERIPHCLK_SAI1 SAI1 peripheral clock (*)
688 * @arg @ref RCC_PERIPHCLK_RNG RNG peripheral clock
689 * @arg @ref RCC_PERIPHCLK_ADCDAC ADCDAC peripheral clock
690 * @arg @ref RCC_PERIPHCLK_DAC1SH DAC1SH peripheral clock
691 * @arg @ref RCC_PERIPHCLK_OCTOSP11 OCTOSP11 peripheral clock
692 * @arg @ref RCC_PERIPHCLK_LPUART1 LPUART1 peripheral clock
693 * @arg @ref RCC_PERIPHCLK_I2C3 I2C3 peripheral clock
694 * @arg @ref RCC_PERIPHCLK_LPTIM34 LPTIM34 peripheral clock
695 * @arg @ref RCC_PERIPHCLK_LPTIM1 LPTIM1 peripheral clock
696 * @arg @ref RCC_PERIPHCLK_USART2 USART2 peripheral clock (*)
697 * @arg @ref RCC_PERIPHCLK_I2C4 I2C4 peripheral clock (*)
698 * @arg @ref RCC_PERIPHCLK_SPI4 SPI4 peripheral clock (*)
699 * @arg @ref RCC_PERIPHCLK_RTC RTC peripheral clock
700
701 * (*) value not defined in all devices.
702 * @retval Frequency in Hz
703 */
704 uint32_t HAL_RCCEx_GetPeriphCLKFreq(uint32_t PeriphClk)
705 {
706     uint32_t frequency = 0;
707     uint32_t srcclk;
708
709     /* Check the parameters */
```

查找分析

- 选择【Array】
- 在【Element】中输入【rcc】获得rcc相关的FLASH占用



demo_mx [Debug] X

File path: .../demo_mx/build/Debug/demo_mx.map

File date: Thu Mar 12 2026 10:19:21 GMT+0800 (China Standard Time)

Tool chain: ARM-GCC gnu-tools-for-stm32 13.3.1+st9

Memory regions:

Regions	Start address	End address	% used	Alert	Status	Used size	Free size	Max size
RAM	0x20000000	0x20030000	1.1%		% used < 75%	2216	194392	192 KB
RAM2	0x20030000	0x20040000	0.00%		% used < 75%		65536	64 KB
FLASH	0x08000000	0x08100000	1.7%		% used < 75%	17696	1030880	1 MB

Elements display type: ☐ Tree ☒ Array ☐ Graph

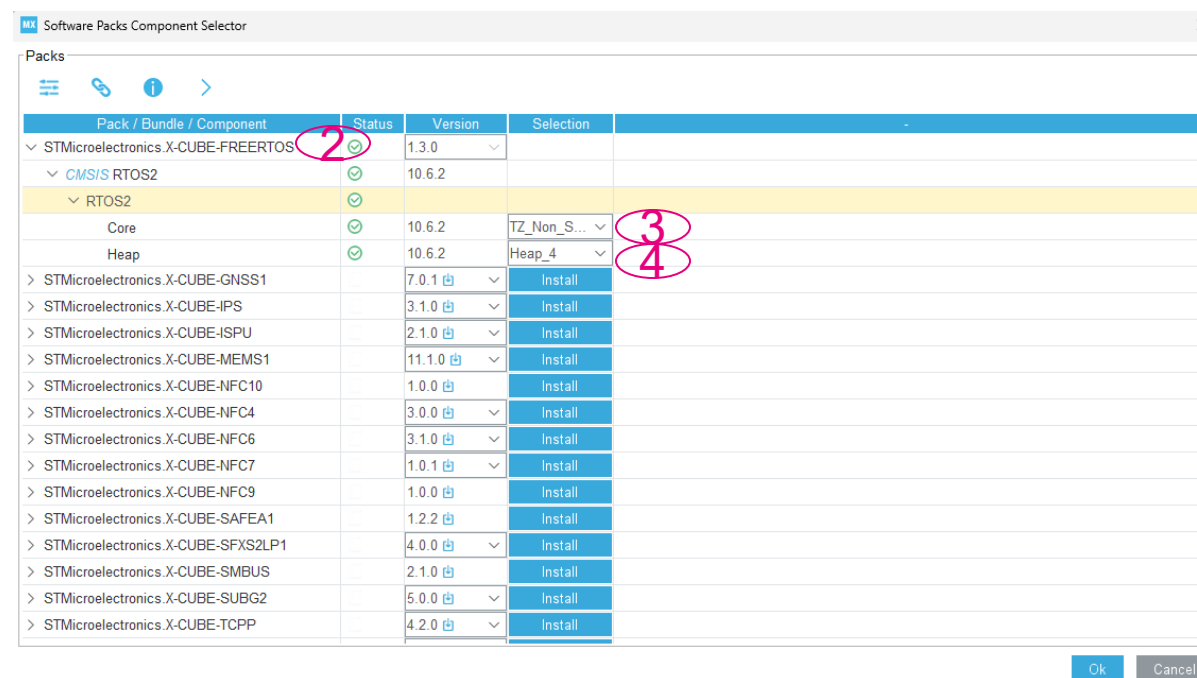
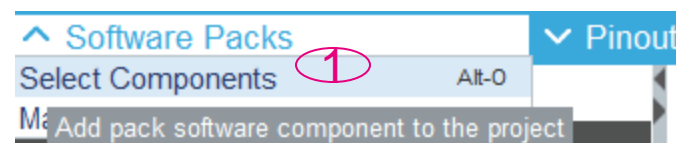
Memory element array:

Region	Section	Library	Object	Element	Type	address	Size
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_OscConfig	function	0x0800005c	1708
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_ClockConfig	function	0x08001208	472
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetSysClockFreq	function	0x080013e0	144
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetHCLKFreq	function	0x08001470	52
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetPCLK1Freq	function	0x080014a4	40
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetPCLK2Freq	function	0x080014cc	40
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetPCLK3Freq	function	0x080014f4	40
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCC_GetMSIHFreq	function	0x0800151c	108
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCCEx_GetPeriphCLKFreq	function	0x08001588	2484
FLASH	.text	.../Drivers/STM32U3xx_HAL_Driver/Src	stm32u3xx_hal_rcc.o	HAL_RCCEx_EpwmBoosterClkConfig	function	0x08001f3c	136

RTOS调试

选择FreeRTOS

- 重新使用STM32CubeMX打开demo_mx.ioc文件
- 选择【Select Components】
- 选择【X-CUBE-FREERTOS】 V1.3.0
- 选择【CMSIS RTOS2】 -> RTOS2
- 选择【Core】 -> TZ_Non_Support
- 选择【Heap】 -> 【Heap_4】



CMSIS RTOS接口

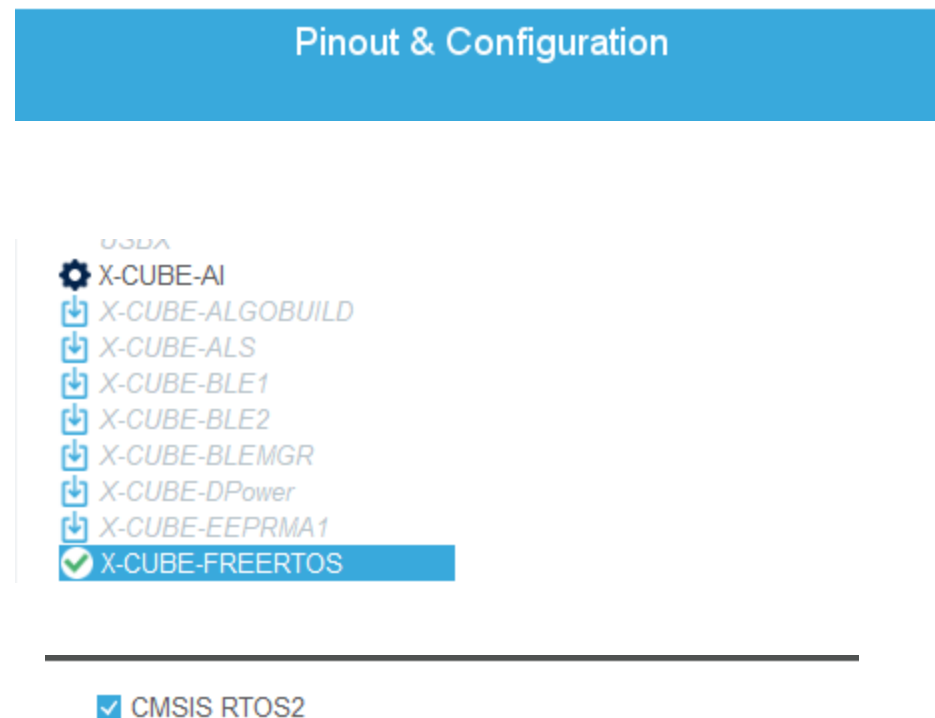
- CMSIS (Cortex Microcontroller Software Interface Standard) 是由ARM公司定义的一套标准接口，用于简化Cortex-M微控制器的开发
- CMSIS_V2相对于CMSIS_V1
 - 支持更多内核架构，例如ARMV8_M
 - 支持更多RTOS
 - 接口变化
 - 并不完全兼容
 - 更多功能

FreeRTOS中的内存分配

- heap_1
 - 特点：最简单的内存分配方式，只支持内存分配，不支持内存释放
 - 适用场景：适用于内存分配后不需要释放的简单应用
- heap_2
 - 特点：支持内存分配和释放，使用最佳适应(best-fit)算法
 - 适用场景：适用于需要动态分配和释放内存的应用，但不支持近邻空闲块合并，可能会产生内存碎片
- heap_3
 - 特点：将 malloc() 和 free() 函数直接映射到标准 C 库的 malloc() 和 free()
 - 适用场景：适用于希望使用标准 C 库内存管理函数的应用
- heap_4
 - 特点：支持内存分配和释放，使用首次适应 (first-fit) 算法，与heap_2不同，它支持近邻空闲块合并
 - 适用场景：适用于需要频繁分配和释放内存且希望减少内存碎片的应用
- heap_5
 - 特点：在heap_4的基础上，支持在跨多块不连续内存区域上进行内存分配和释放
 - 适用场景：适用于需要管理多个内存区域的复杂应用

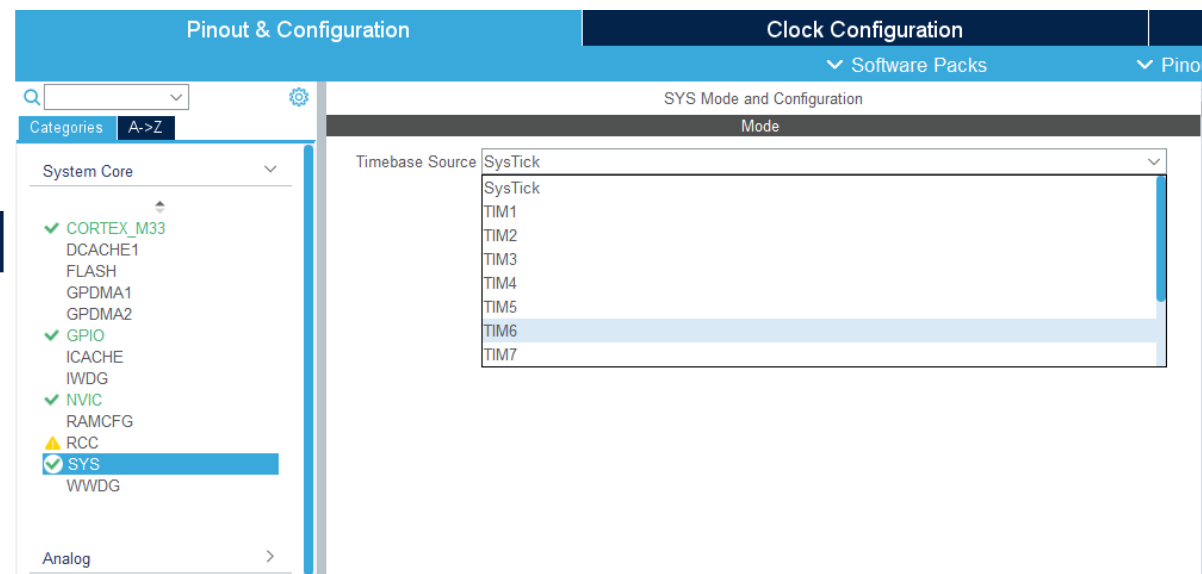
使能X-CUBE-FREERTOS

- 点击【Pinout & Configuration】
- 点击【X-CUBE-FREERTOS】
- 选择【CMSIS RTOS2】



改变HAL的时基

- 选择【Pinout&Configuration】
- 选择【SYS】
- 在【Timebase Source】中选择【TIM6】
- 选择【File】->【Save Project】保存工程
- 使用【GENERATE CODE】生成代码



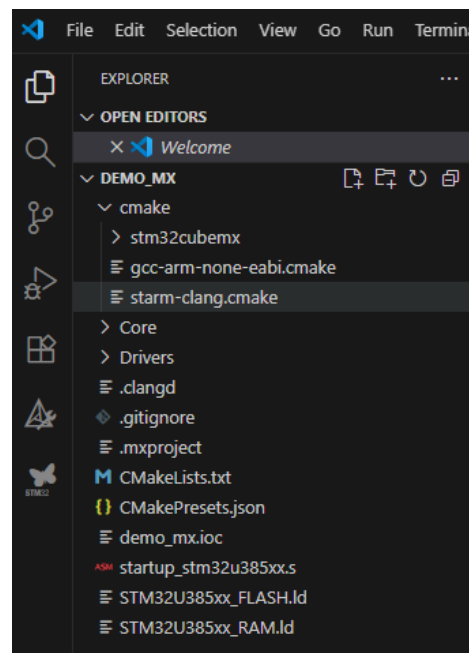
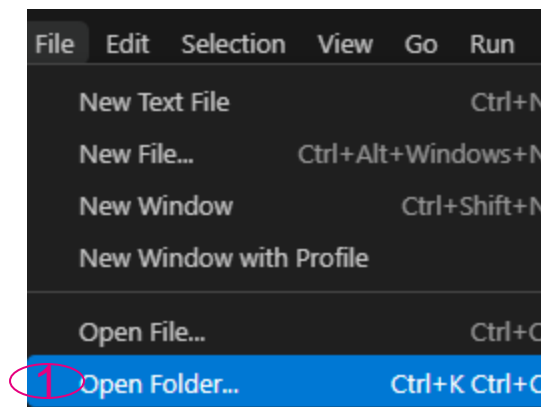
File

- New Project ... Ctrl-N
- Load Project ... Ctrl-L
- Import Project ... Ctrl-I
- Save Project Ctrl-S

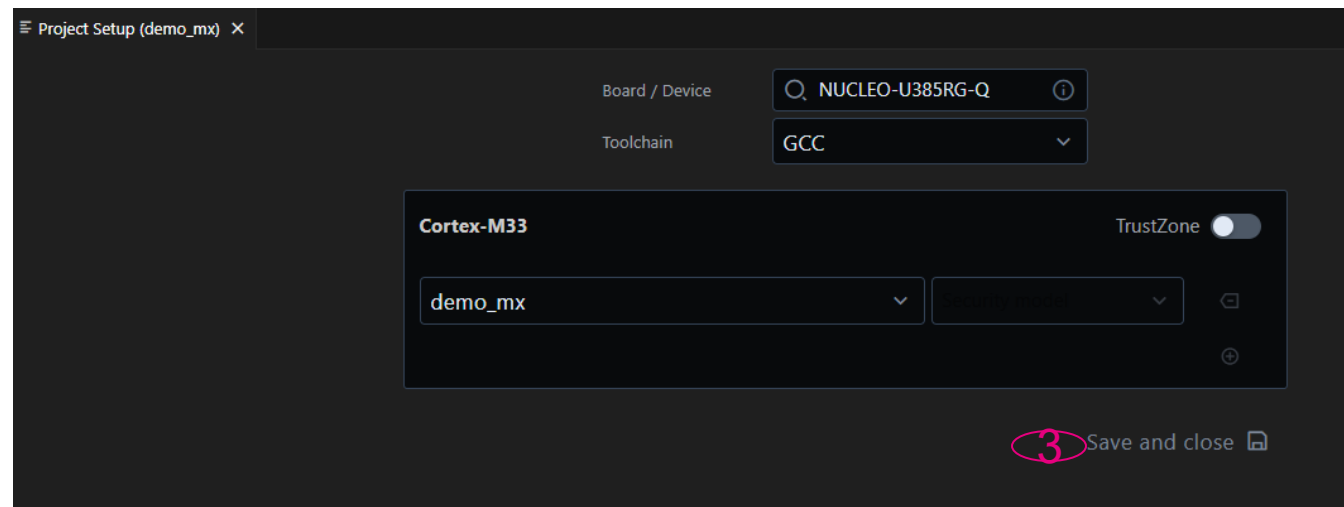
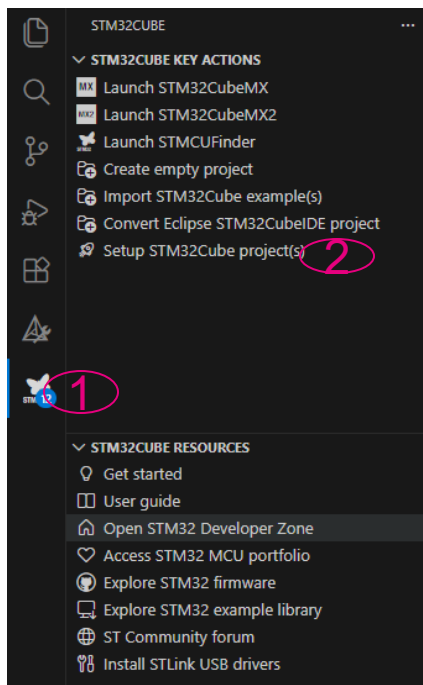
GENERATE CODE

打开工程

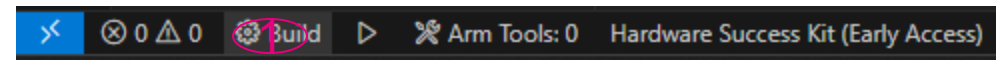
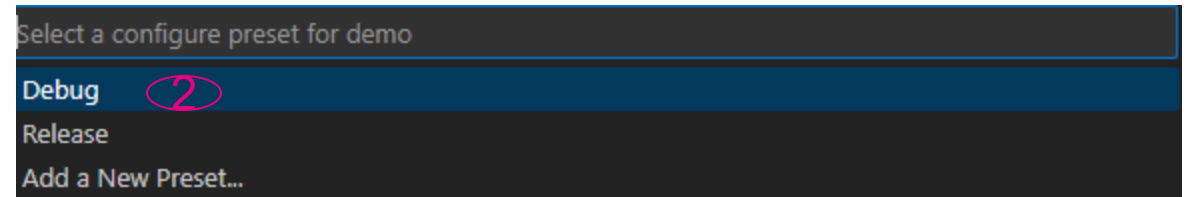
- 使用VSCode打开工程文件夹
 - 【File】 -> 【Open Folder】
 - 选择文件目录【demo_mx】



- 在STM32Cube界面选择【STM32Cube: Setup STM32Cube project(s)】
- 选择【CONFIGURE】
- 选择【Save and close】

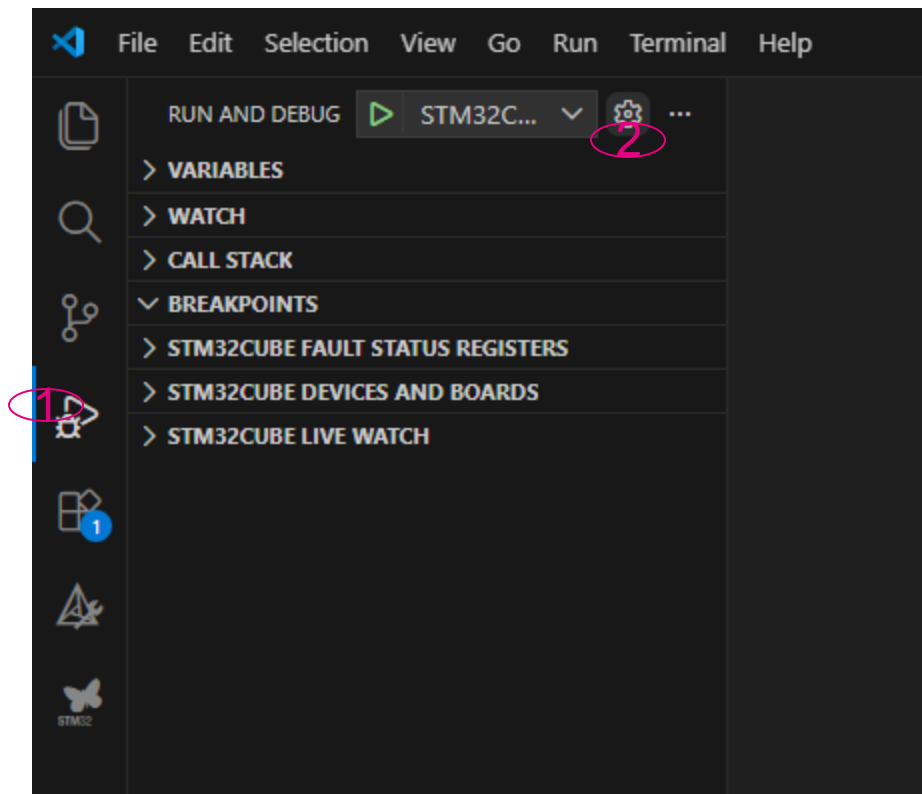


- 在配置过程中已经触发编译，则请选择【Debug】让其编译
- 如果需要再次编译
 - 在界面底部，选择【Build】
 - 如果弹出配置选项，可选择【Debug】



重新配置调试文件

- 配置调试文件支持RTOS



RTOS调试配置

- 在自动提示的功能支持下
 - 仅需要输入“,rt “，然后选择【serverRtos】就可以了。也可以全部输入serverRtos。
 - 在生成的配置里，对于【driver】的配置，删除一对双引号，再输入一个双引号，然后选择【freertos】
- 保存文件

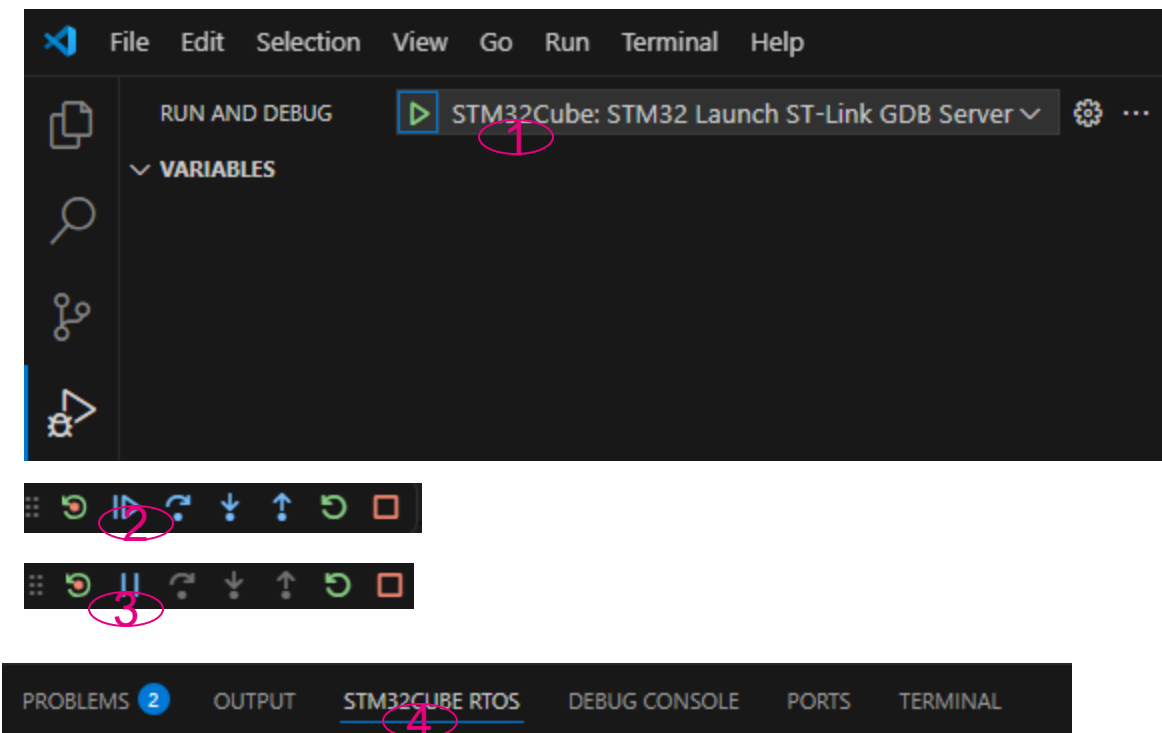
```
"configurations": [  
  {  
    "initCommands": [  
      ],  
    "serverApID": "${command:st-stm32-ide-debug-launch.get-access-port}",  
    "imagesAndSymbols": [  
      {  
        "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"  
      }  
    ],  
    "1"  
  }  
]
```

serverRtos
serverReset
serverReadyAction

```
"serverRtos": {  
  "enabled": true,  
  "port": "60000",  
  "driver": ""  
}  
"freertos"  
"threadx"
```

执行调试

- 点击【STM32Cube: STM32 Launch ST-Link GDB Server】启动调试
- 使用【continue】运行程序
- 使用【pause】暂停程序
- 点击【STM32Cube RTOS】可以看到基本的RTOS信息
- 点击【Call stack】和【STM32CUBE RTOS】可以看到STM32Cube带来的调试工具和详细信息



RTOS多线程调试界面

The screenshot displays the STM32Cube IDE interface for debugging an RTOS application. The top pane shows the source code for 'tasks.c' with the 'portTASK_FUNCTION' function. The middle pane shows the 'Tasks' tab with a table of running tasks. The bottom pane shows the 'BREAKPOINTS' section.

Tasks Table:

Task id	Name	Top of stack	Start of stack	Min free stack	Priority (Base/Actual)	Run-time	State	Event object
0x20000358	defaultTask	0x200002b8	0x20000150	>256B	24/24	N/A	DELAYED	
0x20002a34	Tmr Svc	0x20002c08	0x20002a90	>256B	2/2	N/A	BLOCKED	0x200027a0
0x200027d8	IDLE	0x200029c8	0x20002834	>256B	0/0	N/A	RUNNING	

工程定制



- 技巧

- 在工程根目录CMakeLists.txt为用户定制预留了空间
 - 修改此文件
- STM32CubeMX在cmake/stm32cubemx目录所生成的CMakeLists.txt
 - 此文件内容不需要修改
 - 只需要复制此文件里所包含相应的路径和文件名

添加自己的文件

- 同过去一样，加入自己的文件
 - 在./core/src目录创建自己的文件my.c
 - 在main.c中加入对函数my_hello的调用

```
C my.c 2 x ①
Core > Src > C my.c > my_hello
1  #include "stdio.h"
2
3  void my_hello()
4  {
5  printf("this is the file created by me");
6  }
```

```
Core > Src > C main.c > main
52  /* Private function prototypes
53  void SystemClock_Config(void);
54  void MX_FREERTOS_Init(void);
55  static void MX_GPIO_Init(void);
56  /* USER CODE BEGIN PFP */
57
58  /* USER CODE END PFP */
59
60  /* Private user code -----
61  /* USER CODE BEGIN 0 */
62  extern void my_hello(); ②
63  /* USER CODE END 0 */
```

```
/* -- Sample board code to switch on leds ---- */
BSP_LED_On(Led: LED_GREEN);
my_hello(); ③
/* USER CODE END BSP */
```

修改配置

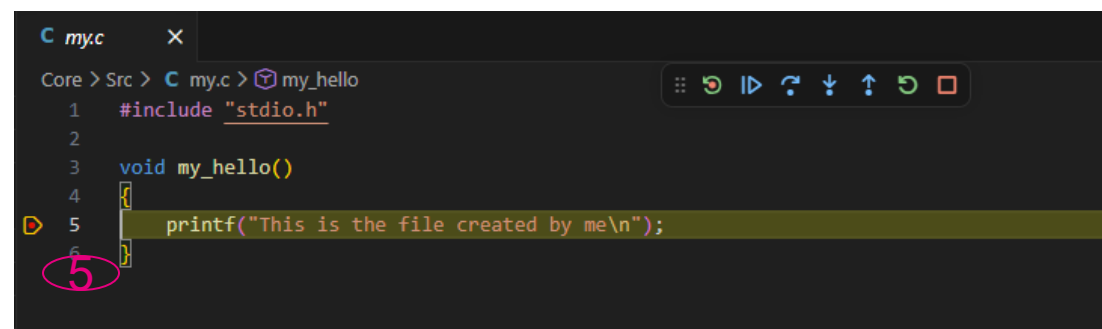
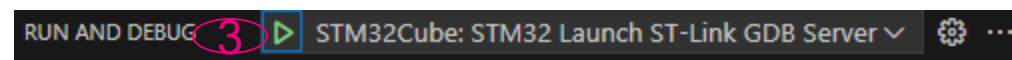
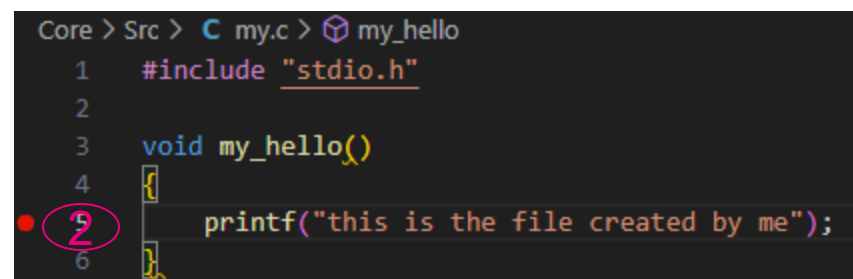
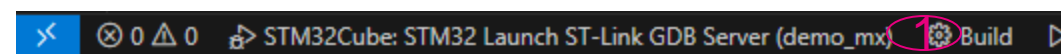
- 复制main.c所在的行到根目录的CMakeLists.txt的【Add sources to executable】中的【Add user sources here】之下
- 修改路径为
\${CMAKE_CURRENT_SOURCE_DIR}/Core/Src/my.c

```
# STM32CubeMX generated application sources
set(MX_Application_Src
    ${CMAKE_CURRENT_SOURCE_DIR}/../..../Core/Src/main.c
```

```
# Add sources to executable
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    # Add user sources here
    ${CMAKE_CURRENT_SOURCE_DIR}/Core/Src/my.c
)
```

编译/设置断点/调试

- 使用【Build】编译
- 在my.c左边单击设置断点
- 使用【start debugging】启动调试
- 使用【continue】执行程序
- 可以看到断点被触发，定制的代码已经被编译且被执行



自动收集目录下的所有文件

- 方法一：使用file
 - 收集所有custom目录下所有c文件到MY_SOURCES变量中

```
file(GLOB MY_SOURCES "./custom/*.c")
```

- 使用MY_SOURCES变量

```
# Add sources to executable
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
  # Add user sources here
  ${MY_SOURCES}
)
```

- 编译调试

- 方法二：使用aux_source_directory
 - 收集所有custom目录下文件到SOURCES中

```
aux_source_directory("./custom" SOURCES)
```

- 使用SOURCES变量

```
# Add sources to executable
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
  # Add user sources here
  ${SOURCES}
)
```

- 编译调试

全局编译选项修改

- 全局编译选项快速修改

- 直接修改cmake目录下的gcc-arm-none-eabi.cmake (或者starm-clang.cmake)

```
set(CMAKE_C_FLAGS_DEBUG "-O0 -g3")
set(CMAKE_C_FLAGS_RELEASE "-Os -g0")
set(CMAKE_CXX_FLAGS_DEBUG "-O0 -g3")
set(CMAKE_CXX_FLAGS_RELEASE "-Os -g0")
```

- 优雅修改全局编译选项

- 修改工程根目录下的CMakeLists.txt, 覆盖相应的定义
 - 该文件设计为用户定制
 - 在STM32CubeMX重新生成工程时, 该文件不会被重新覆盖

```
# Add linked libraries
target_link_libraries(${CMAKE_PROJECT_NAME}
    stm32cubemx

    # Add user defined libraries
)

set(CMAKE_C_FLAGS_DEBUG "-O2 -g3")
```

注意: 也可以在同样页面加入用户库

局部编译选项修改

- 特定文件

- 使用set_source_files_properties修改工程根目录下的CmakeLists.txt

```
set_source_files_properties(  
  ${CMAKE_CURRENT_SOURCE_DIR}/Core/Src/main.c  
  PROPERTIES  
  COMPILE_FLAGS "-Os -g0")
```

- 多个文件

- 使用file以及set_source_files_properties修改工程根目录下的CmakeLists.txt

```
file(GLOB MY_SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/Core/Src/*.c)  
  
set_source_files_properties(  
  ${MY_SOURCES}  
  PROPERTIES  
  COMPILE_FLAGS "-Os -g0")
```

加入前处理

- 使用命令add_custom_target以及add_dependencies
 - 添加在根目录的CMakeLists.txt中

```
add_custom_target(  
    PreBuildTarget  
    COMMAND STM32_Programmer_CLI.exe -l  
    COMMENT "Running pre-build command"  
)  
  
add_dependencies(${CMAKE_PROJECT_NAME} PreBuildTarget)
```

加入后处理

- 使用后处理将elf文件转化为hex和bin
 - 从demo工程的CMakeLists.txt的定制命令复制到demo_mx工程的CMakeLists.txt
 - 将PROJECT_ORIGINAL_NAME改为CMAKE_PROJECT_NAME
 - 重新编译
 - 观察输出
 - 在build/Debug查看demo_mx.bin和demo_mx.hex是否已经生成

```
# Execute post-build to print size, generate hex and bin
add_custom_command(TARGET ${CMAKE_PROJECT_NAME} POST_BUILD
  COMMAND ${CMAKE_SIZE} ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}}
  COMMAND ${CMAKE_OBJCOPY} -O ihex ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}} ${PROJECT_ORIGINAL_NAME}.hex
  COMMAND ${CMAKE_OBJCOPY} -O binary ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}} ${PROJECT_ORIGINAL_NAME}.bin
)
```

```
# Execute post-build to print size, generate hex and bin
add_custom_command(TARGET ${CMAKE_PROJECT_NAME} POST_BUILD
  COMMAND ${CMAKE_SIZE} ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}}
  COMMAND ${CMAKE_OBJCOPY} -O ihex ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}} ${CMAKE_PROJECT_NAME}.hex
  COMMAND ${CMAKE_OBJCOPY} -O binary ${CMAKE_TARGET_FILE:${CMAKE_PROJECT_NAME}} ${CMAKE_PROJECT_NAME}.bin
)
```

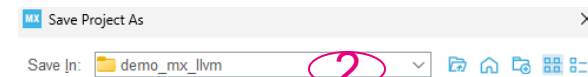
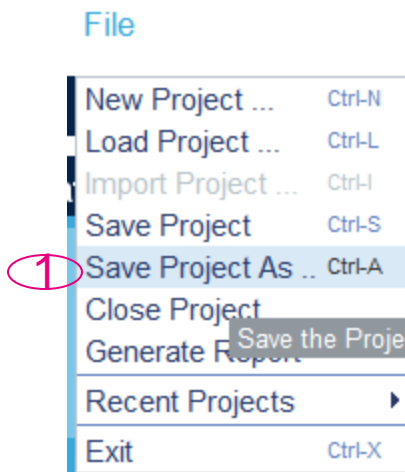
[build]	Memory region	Used Size	Region Size	%age Used
[build]	RAM:	13384 B	192 KB	6.81%
[build]	RAM2:	0 B	64 KB	0.00%
[build]	FLASH:	29568 B	1 MB	2.82%

demo_mx.bin	7/10/2025 1:35 PM	FTE Binary Export ...	29 KB
demo_mx.elf	7/10/2025 1:35 PM	ELF File	1,898 KB
demo_mx.hex	7/10/2025 1:35 PM	HEX File	82 KB

使用LLVM编译器

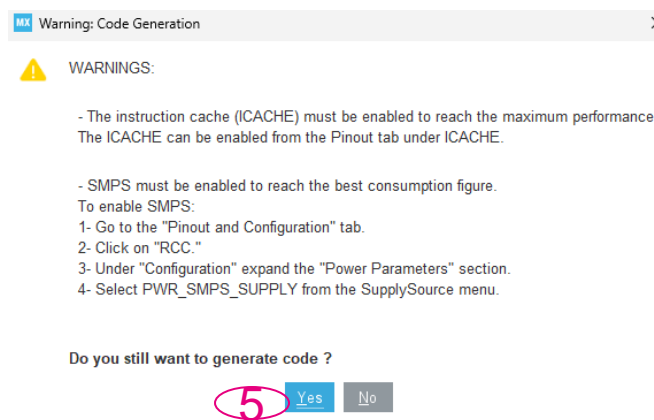
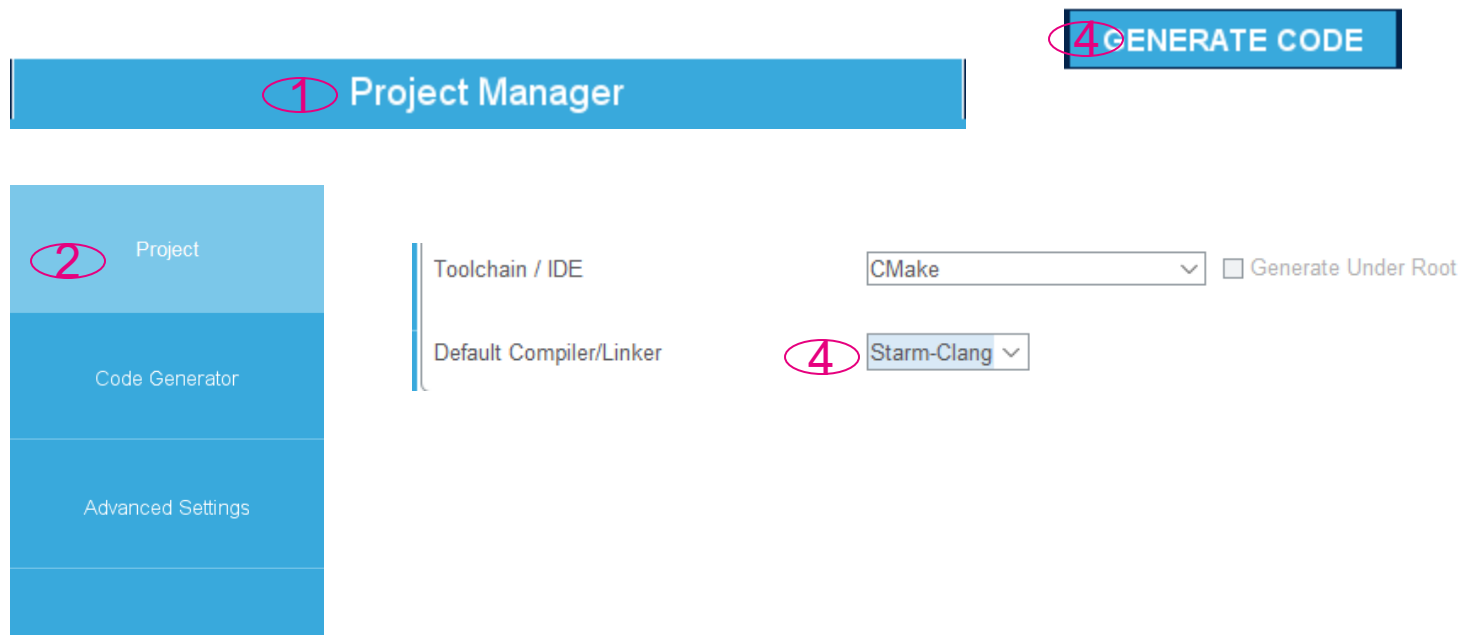
创建LLVM工程

- 新建目录demo_mx_llvm
- 将之前demo_mx.ioc另存到demo_mx_llvm目录



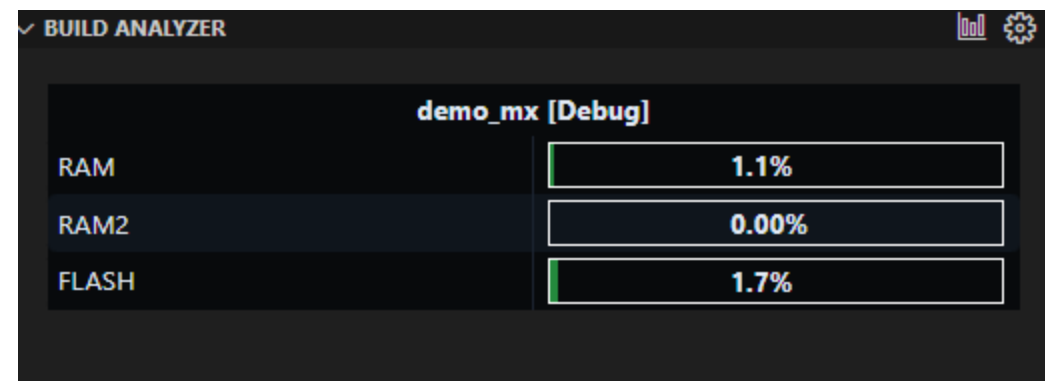
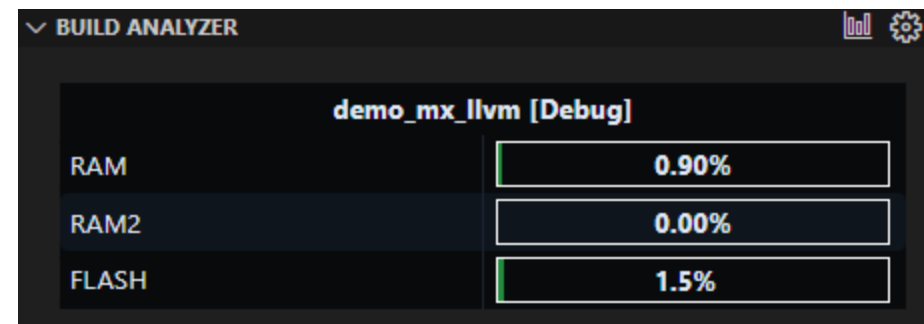
生成LLVM代码

- 选择CMake
 - 【Project Manager】->【Code Generator】->【Toolchain/IDE】
- 使用编译器【Starm-Clang】
- 保存工程
 - 【File】->【Save Project】
- 生成代码
 - 【Generate Code】
 - 关于Cache
 - 为简化这里不配置Cache。对于真实场景，一般需要打开这项功能



编译该工程

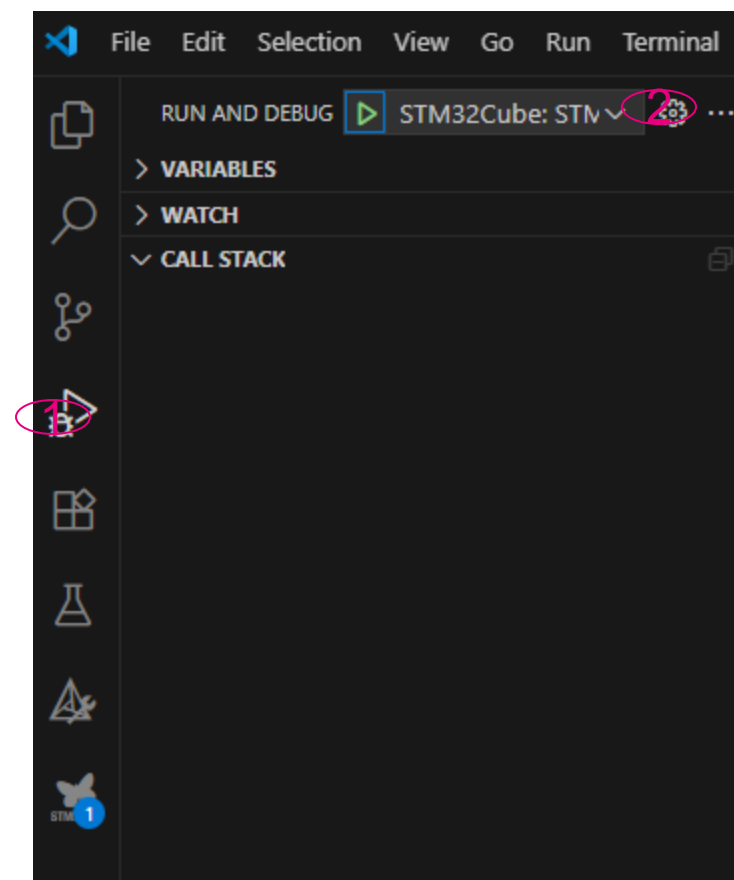
- 使用STM32CubeIDE For VSCode
 - 导入该工程
 - 配置该工程
 - 在starm-clang.cmake中的STARM_TOOLCHAIN_CONFIG手工选择改为STARM_PICOLIBC
 - 需要st-arm-clang 19.1
 - 编译该工程
- 查看编译后的二进制文件大小
 - LLVM编译器占用更小的Flash和RAM
 - demo_mx_llvm vs. demo_mx



Live watch

修改配置

- 选择【Run and Debug】
- 选择【Open 'launch.json'】
- 加入livewatch配置（如下页所示）
 - 输入“live”
 - 选择“livewatch”，自动完成所有配置



Live Watch调试配置

```
"configurations": [  
  {  
    "type": "stlinkgdbtarget",  
    "request": "launch",  
    "name": "STM32Cube: STM32 Launch ST-Link GDB Server",  
    "origin": "snippet",  
    "cwd": "${workspaceFolder}",  
    "preBuild": "${command:st-stm32-ide-debug-launch.build}",  
    "runEntry": "main",  
    "imagesAndSymbols": [  
      {  
        "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"  
      }  
    ],  
    "serverRtos": {  
      "enabled": true,  
      "port": "60000",  
      "driver": "freertos"  
    },  
    "3LiveWatch": {  
      "enabled": true,  
      "samplesPerSecond": "4"  
    }  
  }  
]
```

使用STM32CUBE LIVE WATCH

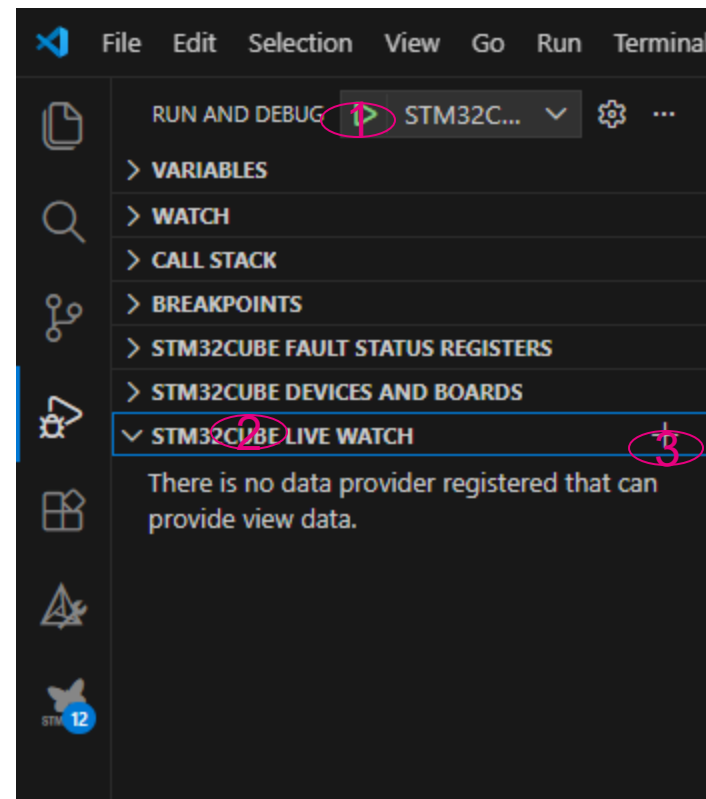
- 启动调试
- 选择【STM32CUBE LIVE WATCH】
- 选择【+】
- 在弹出的命令窗口输入uwTick
- 点击【Resume】，可看到uwTick持续变化

uwTick

Enter Live Watch Expression (Press 'Enter' to confirm or 'Escape' to cancel)

✓ STM32CUBE LIVE WATCH

uwTick = 41920



Hotplug 调试

环境准备

- 硬件准备

- 关闭调试连接
- 重新启动开发板
- 观察到串口有输出

- 配置准备

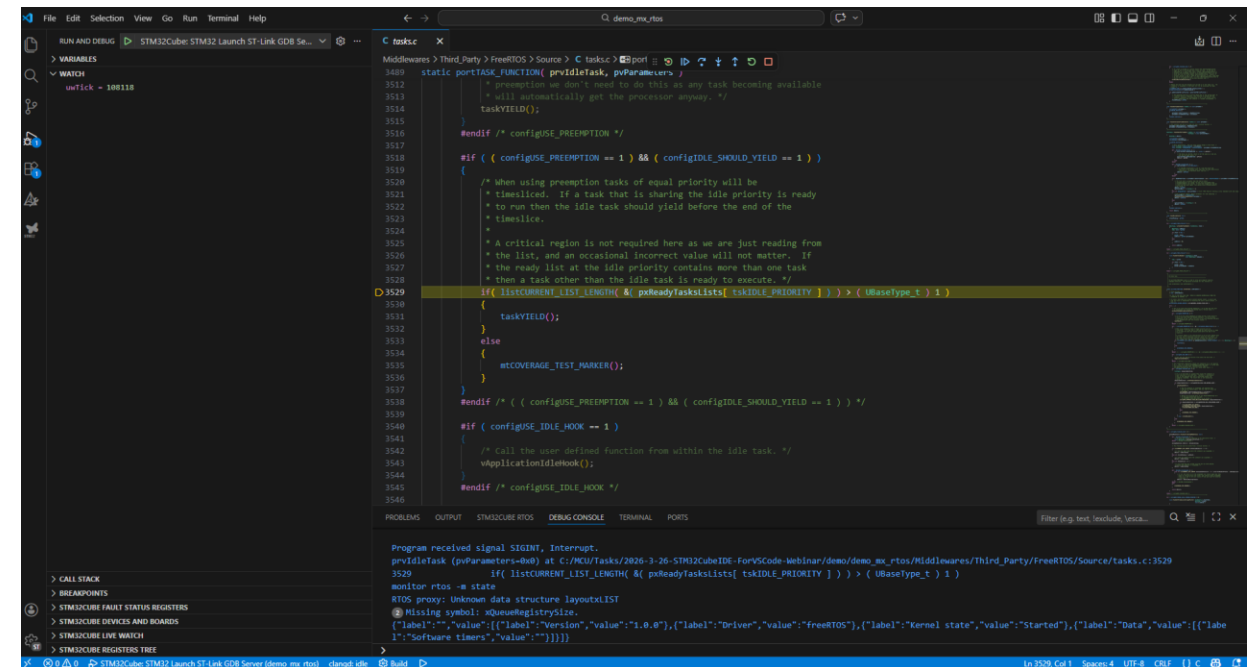
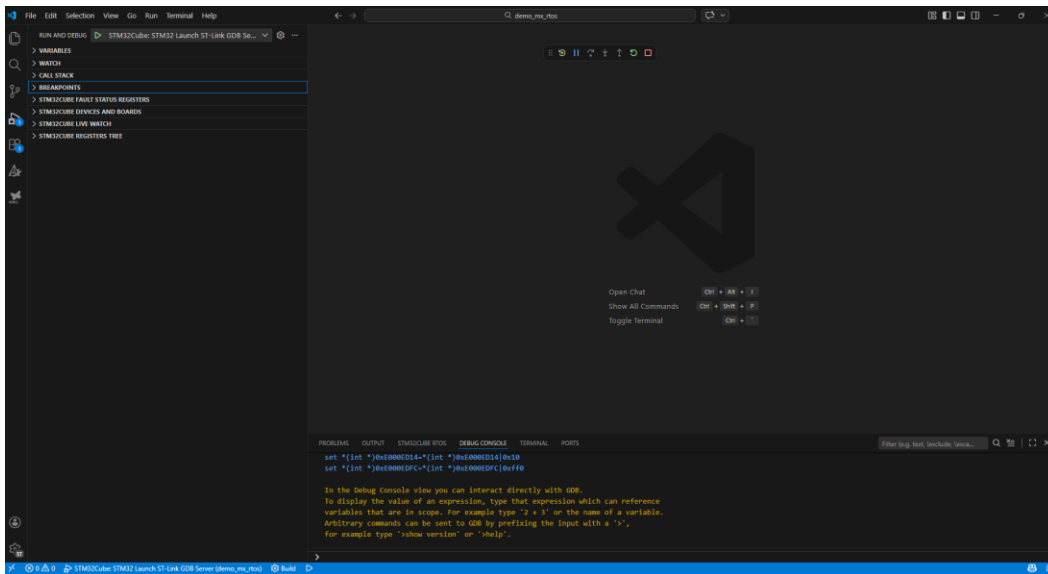
- 修改配置文件加入"serverReset": "None"
- 修改配置文件将"imageFileName"改为"symbolFileName"

```
vscode > {} launch.json > Launch Targets > {} STM32Cube: STM32 Launch ST-Link GDB Server
2 // Use IntelliSense to learn about possible attributes.
3 // Hover to view descriptions of existing attributes.
4 // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5 "version": "0.2.0",
6 "configurations": [
7
8   {
9     "type": "stlinkgdbtarget",
10    "request": "launch",
11    "name": "STM32Cube: STM32 Launch ST-Link GDB Server",
12    "origin": "snippet",
13    "cwd": "${workspaceFolder}",
14    "preBuild": "${command:st-stm32-ide-debug-launch.build}",
15    "runEntry": "main",
16    "imagesAndSymbols": [
17      {
18        "symbolFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
19      }
20    ],
21    "serverRtos": {
22      "enabled": true,
23      "port": "60000",
24      "driver": "freertos"
25    },
26    "liveWatch": {
27      "enabled": true,
28      "samplesPerSecond": "4"
29    },
30    "serverReset": "None"
31  }
32 ]
33 }
```



Hotplug调试效果

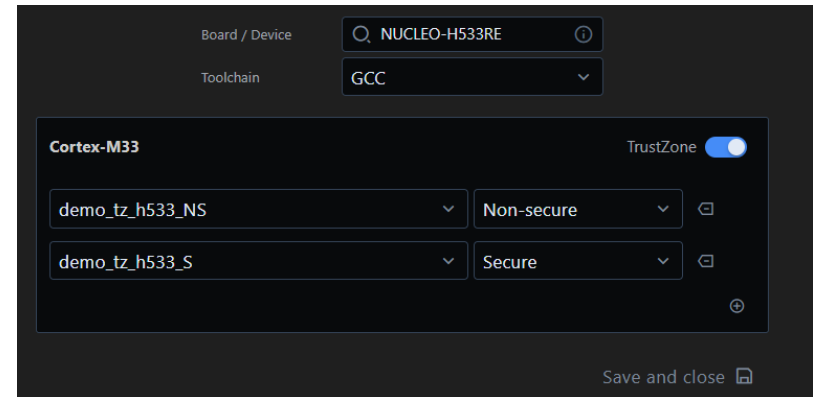
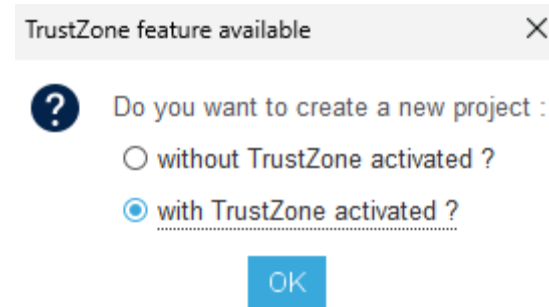
- 启动调试，可以观察到不会停在main函数
- 按【pause(F6)】查看未重启的当前状态，例如uwTick的当前值表示程序的运行时间



Trustzone调试

生成一个Trustzone工程

- 使用STM32CubeMX生成一个STM32H533 Trustzone工程，也就是【with TrustZone activated】
 - STM32H5上Trustzone应用比较多
- 其他和之前的非Trustzone选择一样
 - 选择从开发板触发
 - 选择生成例程
 - 导出Cmake工程
- 使用Setup STM32Cube project(s)使能Trustzone
- 正常编译



配置Trustzone相关选项字节

- 使能Trustzone
- 修改Securearea1为secure
 - 开始值为0，结束值为0x1f
- 修改Securearea2为non-secure
 - 开始值为1f，结束值为0

User Configuration 2

TZEN B4 SRAM2_ECC ☒ BKPRAM_ECC ☒ SRAM2_RST ☒ SRAM1_3_RST ☒

Boot Configuration

NSBOOTADD Value 0x80000 Address 0x08000000 NSBOOT_LOCK C3 SECBOT_LOCK C3 SECBOTADD Value 0xc0000 Address 0x0c000000

Bank1 - Flash watermark area definition

SECWM1_STRT Value 0x0 Address 0x08000000 SECWM1_END Value 0x1f Address 0x0803e000

Write sector group protection 1

WRPSGn1 0xff

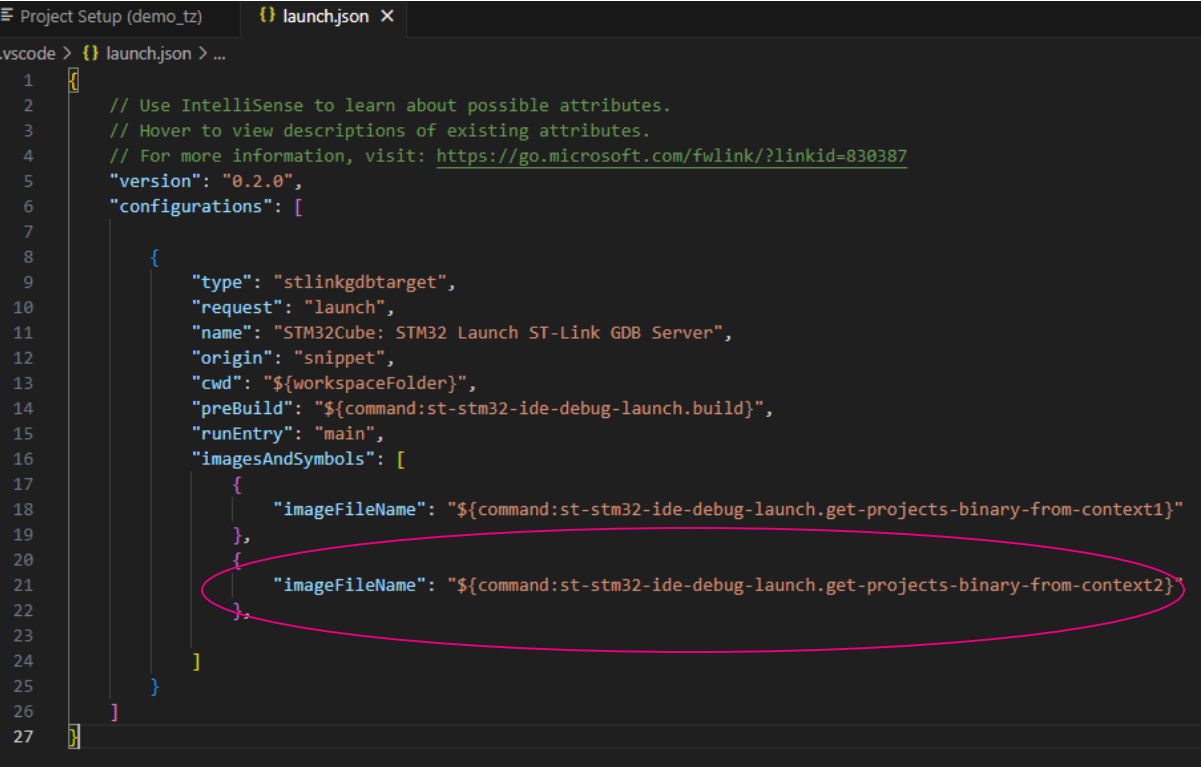
Bank2 - Flash watermark area definition

SECWM2_STRT Value 0x1f Address 0x0807e000 SECWM2_END Value 0x0 Address 0x08040000

配置STM32CubeIDE For VSCode调试配置

- 增加一个image:

```
{  
  "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context2}"  
},
```



```
Project Setup (demo_tz)  launch.json X  
vscode > launch.json > ...  
1  {  
2    // Use IntelliSense to learn about possible attributes.  
3    // Hover to view descriptions of existing attributes.  
4    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387  
5    "version": "0.2.0",  
6    "configurations": [  
7      {  
8        "type": "stlinkgdbtarget",  
9        "request": "launch",  
10       "name": "STM32Cube: STM32 Launch ST-Link GDB Server",  
11       "origin": "snippet",  
12       "cwd": "${workspaceFolder}",  
13       "preBuild": "${command:st-stm32-ide-debug-launch.build}",  
14       "runEntry": "main",  
15       "imagesAndSymbols": [  
16         {  
17           "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"  
18         },  
19         {  
20           "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context2}"  
21         },  
22       ],  
23     },  
24   ],  
25 },  
26  
27 }
```

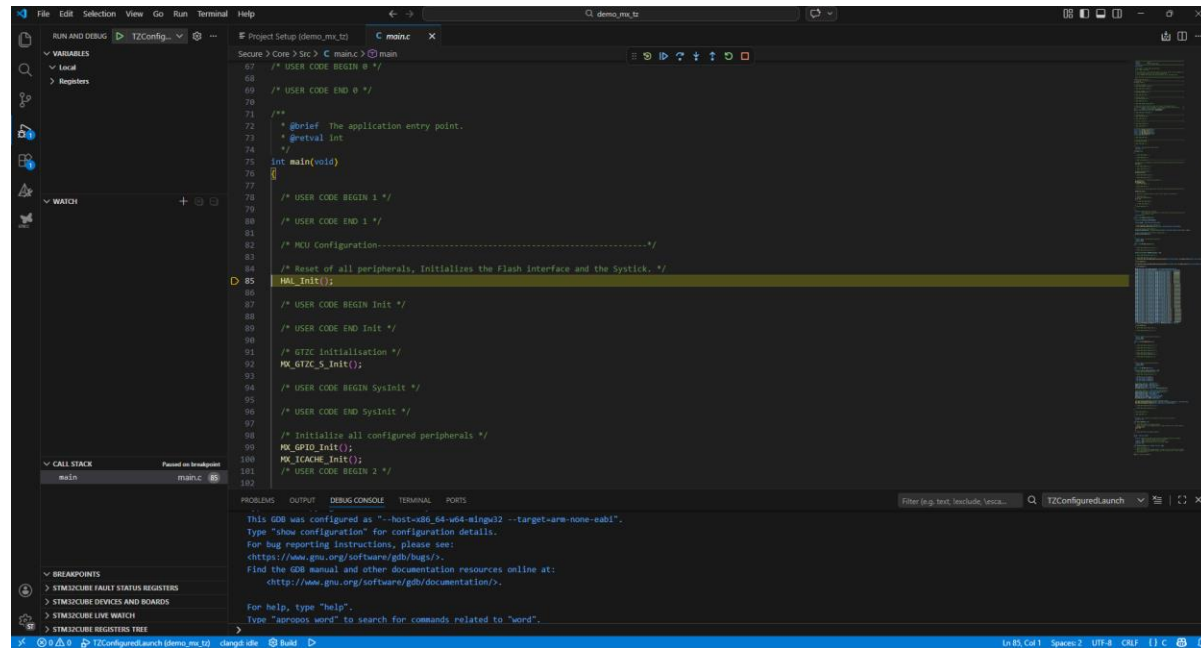
启动调试以完成配置

- 启动调试后，系统会提示选择相应的映像
 - Context1选择non secure elf
 - Context2选择secure elf
- 当询问是否保存设置时，保存为tz_debug
- 生成的整体调试配置例程如右

```
{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "type": "stlinkgdbtarget",
      "request": "launch",
      "name": "STM32Cube: STM32 Launch ST-Link GDB Server",
      "origin": "snippet",
      "cwd": "${workspaceFolder}",
      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
      "runEntry": "main",
      "imagesAndSymbols": [
        {
          "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
        },
        {
          "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context2}"
        }
      ]
    },
    {
      "name": "tz_debug",
      "type": "stlinkgdbtarget",
      "request": "launch",
      "cwd": "${workspaceFolder}",
      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
      "runEntry": "main",
      "imagesAndSymbols": [
        {
          "imageFileName": "${workspaceFolder}/NonSecure/build/demo_tz_NS.elf"
        },
        {
          "imageFileName": "${workspaceFolder}/Secure/build/demo_tz_S.elf"
        }
      ],
      "verbose": false
    }
  ]
}
```

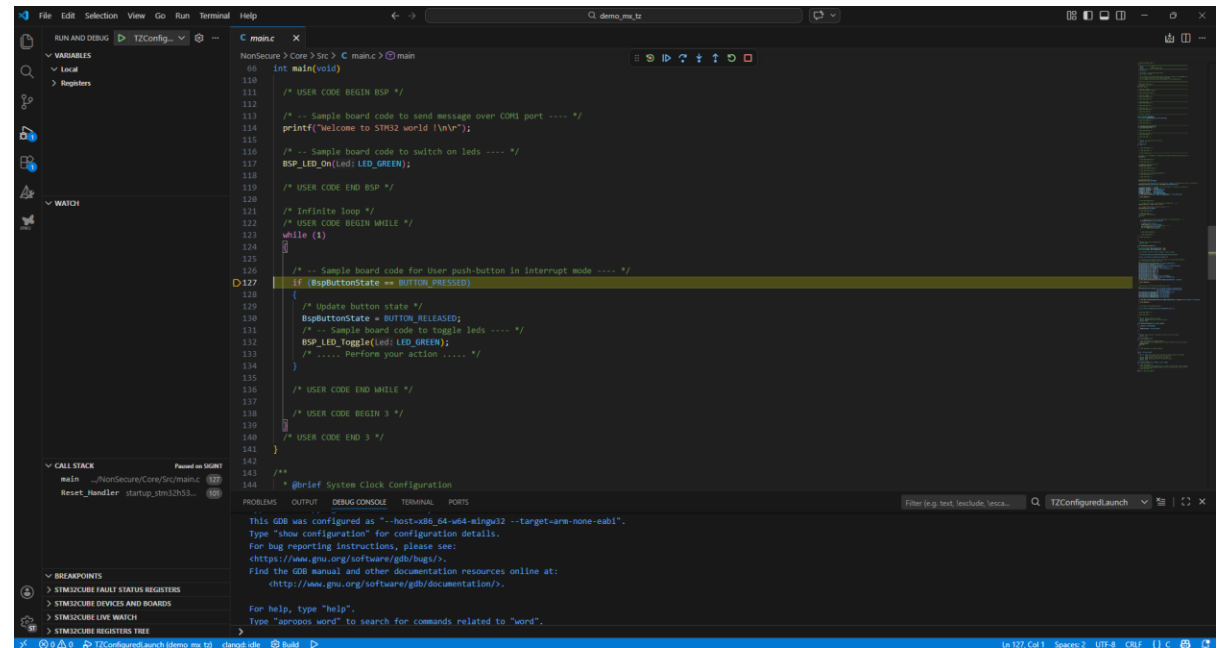
调试界面

安全世界(Secure world)



```
Secure > Core > Src > C main.c > main
67 /* USER CODE BEGIN 0 */
68
69 /* USER CODE END 0 */
70
71 /**
72  * @brief The application entry point.
73  * @retval int
74  */
75 int main(void)
76 {
77     /* USER CODE BEGIN 1 */
78     /* USER CODE END 1 */
79
80     /* MCU Configuration ----- */
81
82     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
83     HAL_Init();
84
85     /* USER CODE BEGIN Init */
86     /* USER CODE END Init */
87
88     /* USER CODE BEGIN SysInit */
89     /* USER CODE END SysInit */
90
91     /* RTC initialization */
92     MX_RTC_Init();
93
94     /* USER CODE BEGIN SysInit */
95     /* USER CODE END SysInit */
96
97     /* Initialize all configured peripherals */
98     MX_GPIO_Init();
99     MX_I2C_Init();
100
101     /* USER CODE BEGIN 2 */
102 }
```

非安全世界(None-secure world)

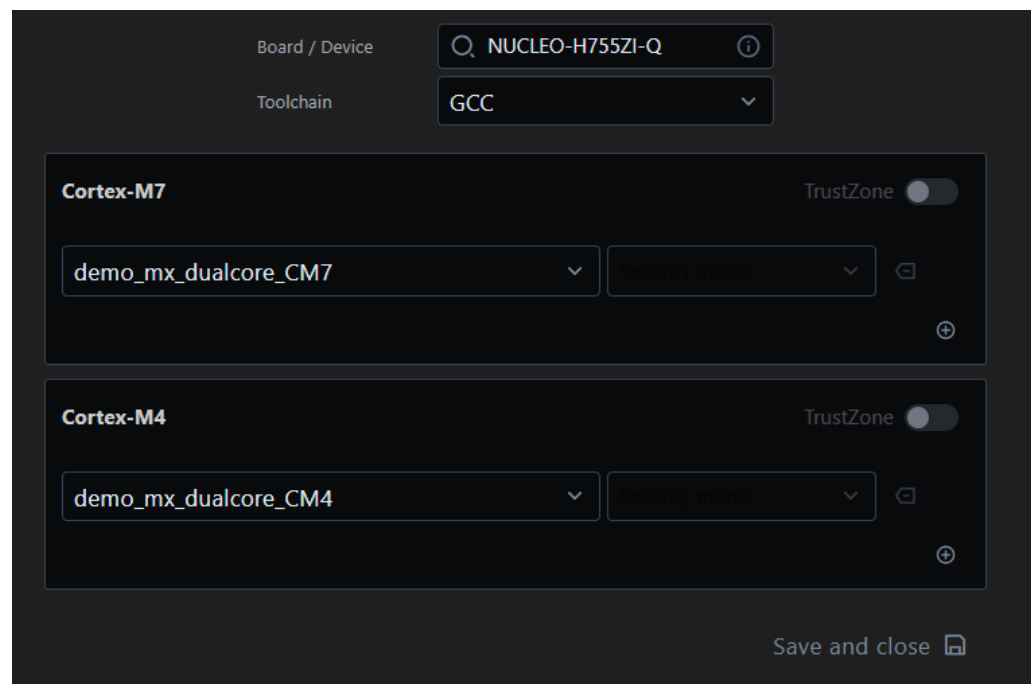


```
NonSecure > Core > Src > C main.c > main
66 int main(void)
67 {
68     /* USER CODE BEGIN BSP */
69
70     /* Sample board code to send message over COM1 port ----- */
71     printf("Welcome to STM32 world !\n\n");
72
73     /* Sample board code to switch on leds ----- */
74     BSP_LED_On(LED_GREEN);
75
76     /* USER CODE END BSP */
77
78     /* Infinite loop */
79     /* USER CODE BEGIN WHILE */
80     while (1)
81     {
82         /* Sample board code for User push-button in interrupt mode ----- */
83         if (BSPButtonState == BUTTON_PRESSED)
84         {
85             /* Update button state */
86             BSPButtonState = BUTTON_RELEASED;
87             /* Sample board code to toggle leds ----- */
88             BSP_LED_Toggle(LED_GREEN);
89             /* Perform your action ----- */
90
91         }
92     }
93     /* USER CODE END WHILE */
94
95     /* USER CODE BEGIN 3 */
96
97     /* USER CODE END 3 */
98 }
99
100 /**
101  * @brief System Clock Configuration
102  */
```

双核调试

创建双核工程

- 使用STM32CubeMX基于NUCLEO-H755ZI-Q创建双核工程
- 使用STM32CubeIDE For VSCode导入目录
- 使用【Setup STM32Cube project(s)】配置双核工程
- 正常创建【Launch.json】



配置CM7调试

- 加入"serverShared": true
- 加入"serverHaltAllCores": true
- 加入"deviceCore": "Cortex-M7",
- 加入另外一个image, 注意顺序

```
{  
  "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context2}"  
}
```

- 启动调试
 - 在调试时为context2选择CM4 elf, 为context1选择CM7 elf
 - 在调试时保存调试配置为CM7
 - 暂时关闭

```
{  
  "name": "CM7",  
  "type": "stlinkgdbtarget",  
  "request": "launch",  
  "deviceCore": "Cortex-M7",  
  "cwd": "${workspaceFolder}",  
  "preBuild": "${command:st-stm32-ide-debug-launch.build}",  
  "runEntry": "main",  
  "serverShared": true,  
  "serverHaltAllCores": true,  
  "imagesAndSymbols": [  
    {  
      "imageFileName": "${workspaceFolder}/CM4/build/demo_mx_dualcore_CM4.elf"  
    },  
    {  
      "imageFileName": "${workspaceFolder}/CM7/build/demo_mx_dualcore_CM7.elf"  
    }  
  ],  
  "verbose": false  
}
```

配置CM4调试

- 复制CM7配置
- 修改"name": "CM4",
- 修改 "deviceCore": "Cortex-M4",
- 加入"serverReset": "None",
- 添加"serverApID": "3"
- "serverPort": "61239",
- 修改imageFileName为
symbolFileName

```
{  
  "name": "CM4",  
  "type": "stlinkgdbtarget",  
  "request": "launch",  
  "deviceCore": "Cortex-M4",  
  "cwd": "${workspaceFolder}",  
  "preBuild": "${command:st-stm32-ide-debug-launch.build}",  
  "runEntry": "main",  
  "serverShared": true,  
  "serverHaltAllCores": true,  
  "serverApID": "3",  
  "serverPort": "61239",  
  "serverReset": "None",  
  "imagesAndSymbols": [  
    {  
      "symbolFileName": "${workspaceFolder}/CM4/build/demo_mx_dualcore_CM4.elf"  
    }  
  ],  
  "verbose": false  
},
```

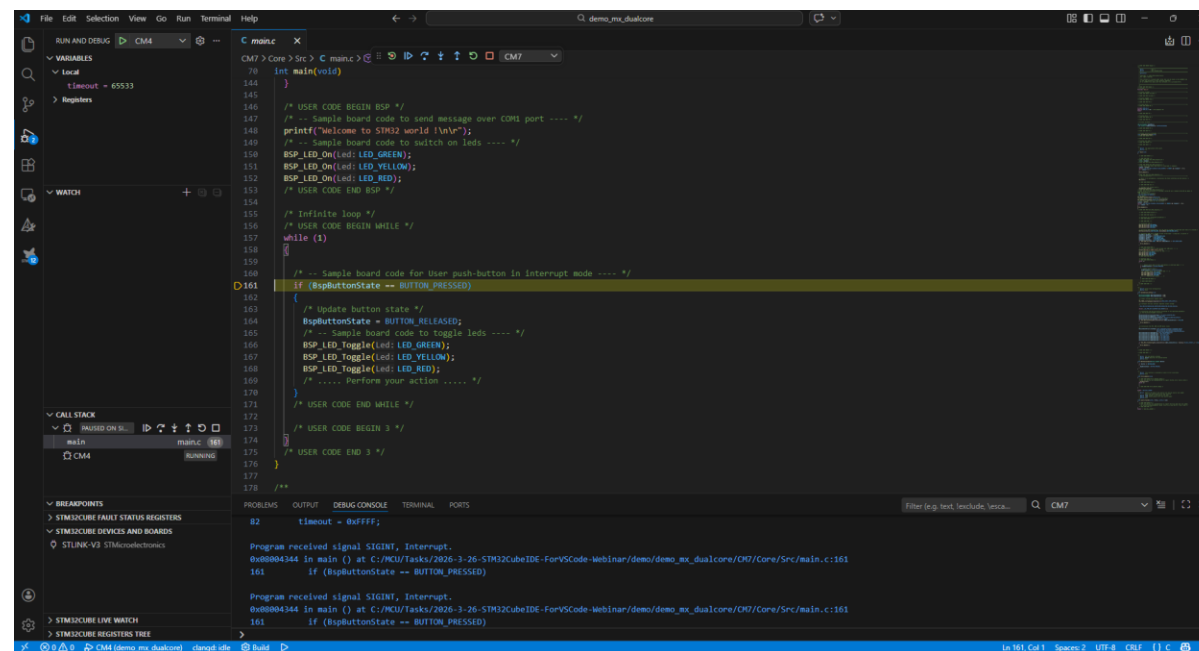
完整双核调试配置

```
.vscode > launch.json X
.vscode > launch.json > Launch Targets > {} STM32Cube

1 {
2   // Use IntelliSense to learn about possible attributes.
3   // Hover to view descriptions of existing attributes.
4   // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5   "version": "0.2.0",
6   "configurations": [
7
8     {
9       "type": "stlinkgdbtarget",
10      "request": "launch",
11      "name": "STM32Cube",
12      "deviceCore": "Cortex-M0",
13      "origin": "initpet",
14      "cwd": "${workspaceFolder}",
15      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
16      "runEntry": "main",
17      "serverShared": true,
18      "serverHaltAllCores": true,
19      "imagesAndSymbols": [
20        {
21          "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context2}"
22        },
23        {
24          "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
25        }
26      ],
27    },
28    {
29      "name": "O0",
30      "type": "stlinkgdbtarget",
31      "request": "launch",
32      "deviceCore": "Cortex-M0",
33      "cwd": "${workspaceFolder}",
34      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
35      "runEntry": "main",
36      "serverShared": true,
37      "serverHaltAllCores": true,
38      "imagesAndSymbols": [
39        {
40          "imageFileName": "${workspaceFolder}/O0/build/demo_mx_dualcore_O0.elf"
41        },
42        {
43          "imageFileName": "${workspaceFolder}/O0/build/demo_mx_dualcore_O0.elf"
44        }
45      ],
46      "verbose": false
47    },
48    {
49      "name": "O4",
50      "type": "stlinkgdbtarget",
51      "request": "launch",
52      "deviceCore": "Cortex-M4",
53      "cwd": "${workspaceFolder}",
54      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
55      "runEntry": "main",
56      "serverShared": true,
57      "serverHaltAllCores": true,
58      "serverApID": "2",
59      "serverPort": "61380",
60      "serverReset": "None",
61      "imagesAndSymbols": [
62        {
63          "symbolFileName": "${workspaceFolder}/O4/build/demo_mx_dualcore_O4.elf"
64        }
65      ],
66      "verbose": false
67    },
68  ],
69 }
70 }
```

启动调试

- 首先启动CM7调试，自动停在main函数
- 再启动CM4调试，自动停在main函数
- 让CM4全速执行进入等待信号量状态
- 让CM7全速执行唤醒CM4
- 根据需要中断任一内核CM7或者CM4进行调试



多ST-Link环境调试

多ST-LINK环境调试

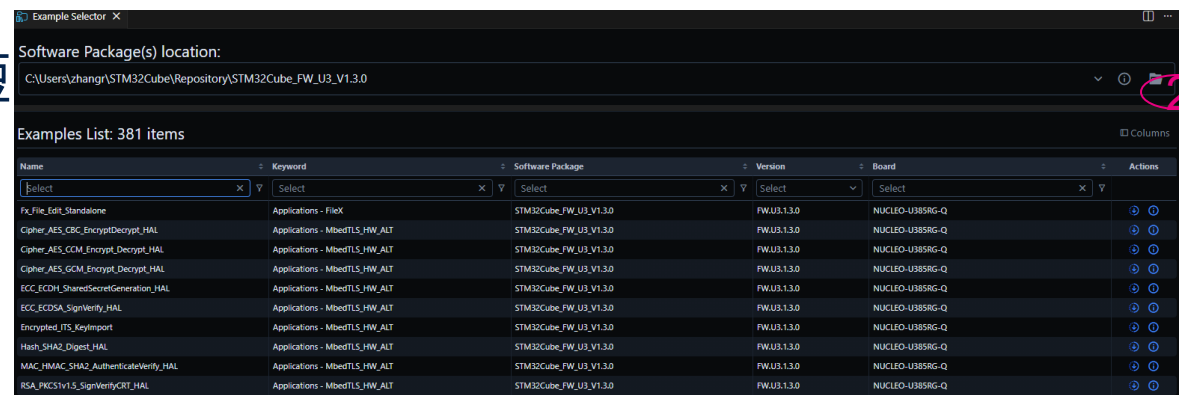
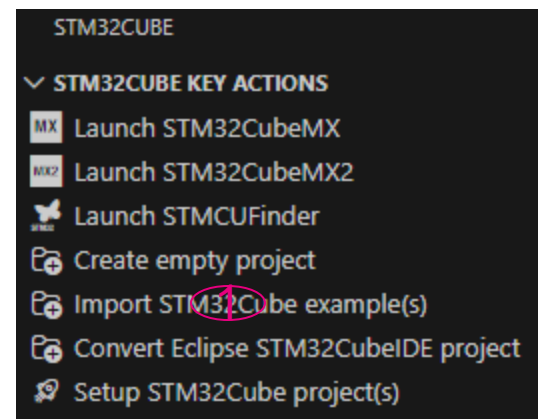
- 在多ST-LINK环境下需要使用serverSerialNumber手工指定ST-LINK串号
 - ST-LINK串口可以通过STM32CubeProgrammer获得，例如cube programmer -l，也可以参考之前的前处理所使用的命令

```
{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "type": "stlinkgdbtarget",
      "request": "launch",
      "name": "STM32Cube: STM32 Launch ST-Link GDB Server",
      "origin": "snippet",
      "cwd": "${workspaceFolder}",
      "initCommands": [
        "tbreak main"
      ],
      "serverSerialNumber": "0051003A3433511737363934",
      "serverReset": "none",
      "imagesAndSymbols": [
        {
          // "imageFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}",
          "symbolFileName": "${command:st-stm32-ide-debug-launch.get-projects-binary-from-context1}"
        }
      ]
    }
  ]
}
```

导入与转换例程

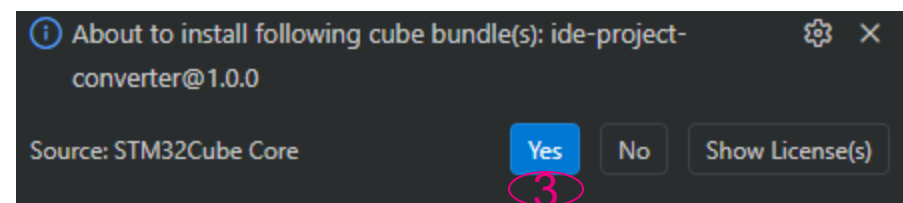
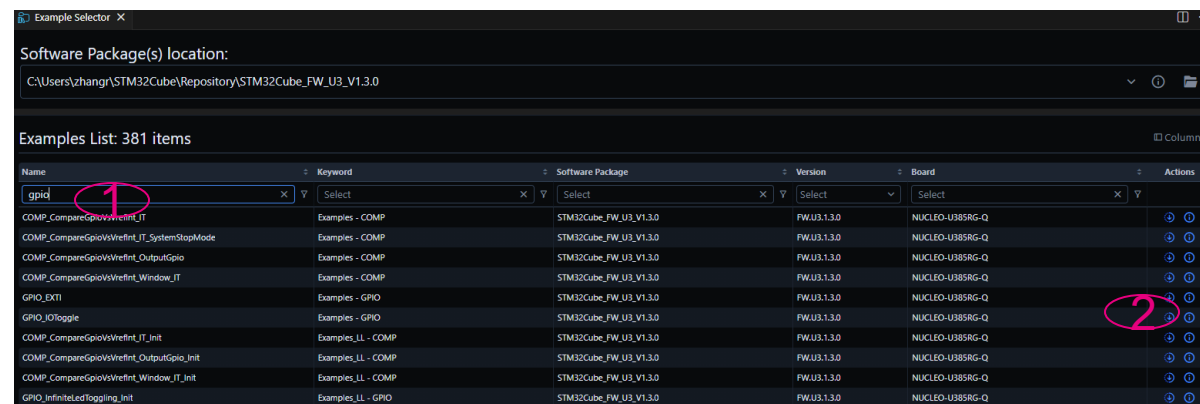
导入例程

- 新建并使用VSCode打开【demo_import】目录
 - 不然，可能后续无法安装ide-project-converter
- 选择【Import STM32Cube Example(s)】
- 自动出现【Example Selector】
 - 取决于本地目录的大小，默认仓库时间长短可能不一样
 - 建议选择指定的STM32Cube库目录，例如U3仓库



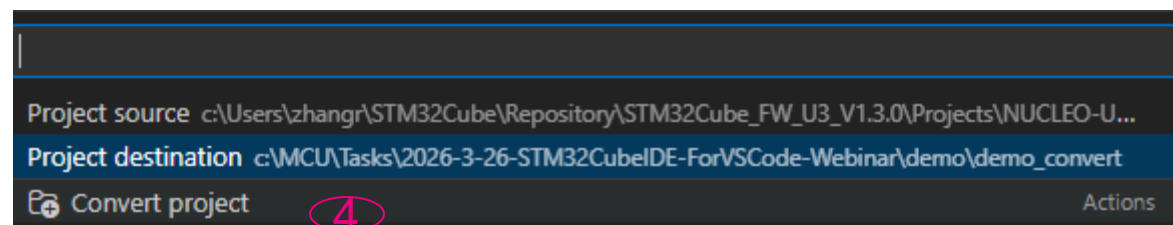
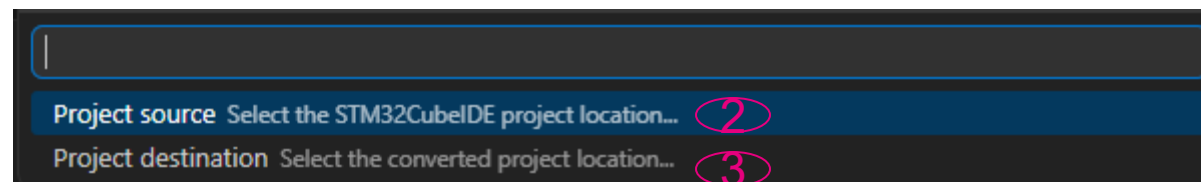
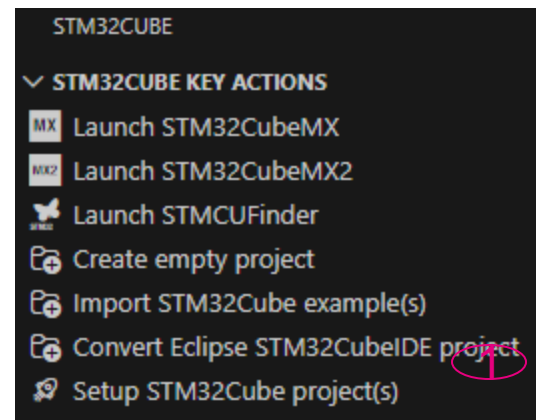
选择例程GPIO_IOToggle

- 在【Name】处输入gpio
- 选择【Import】
- 同意转换
- 选择当前目录【demo_import】
- 其他操作与之前一样
 - 工程自动打开
 - 工程自动配置以及相应Pack下载
 - 编译下载调试



转换STM32CubeIDE工程

- 我们已经看到STM32CubeIDE for VSCode能够自动转换STM32CubeIDE工程
- 我们也可以手工选择转换工程
 - 新建目录demo_convert
 - 使用VSCode打开
 - 选择【Convert Eclipse STM32CubeIDE project】
 - 选择【Project source】
 - 选择【Project destination】
 - 选择【Convert project】
- 其他操作与之前一样

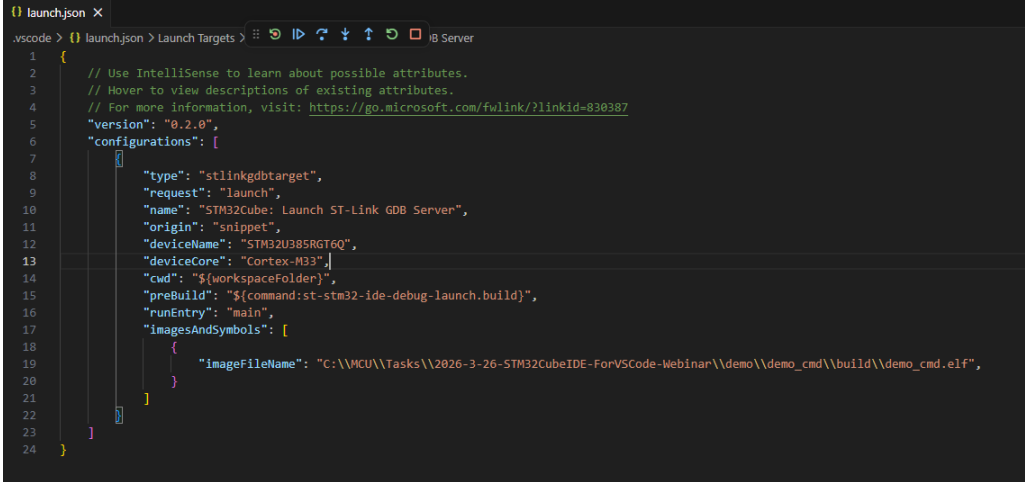


持续集成

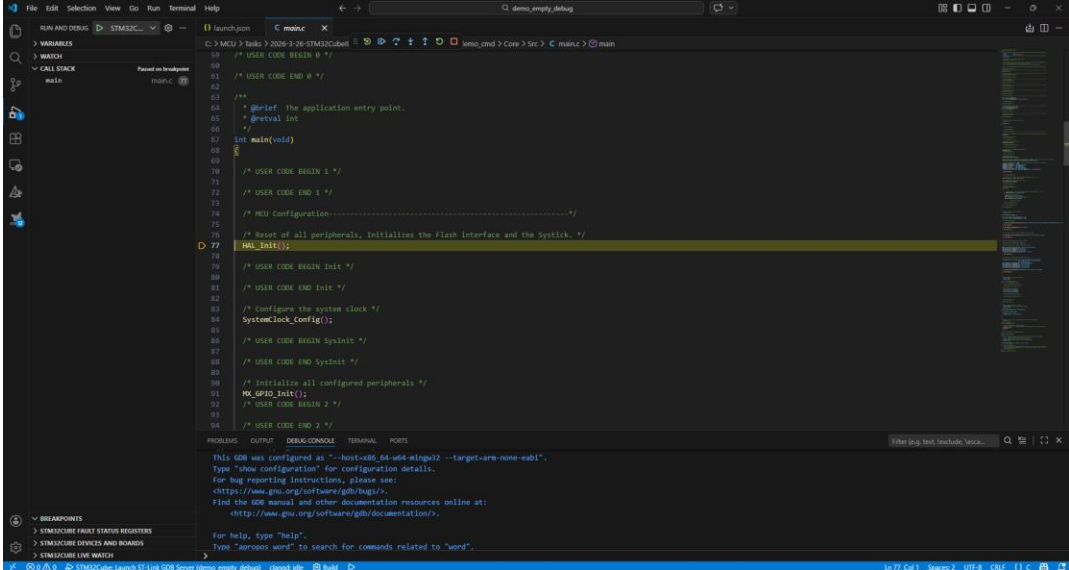
- 借用工具链
 - STM32CubeIDE For VSCode插件会自动安装所有必需的工具链
- 设置环境变量
 - Cmake
 - Ninja
 - Gcc
- 导出工程
 - 使用STM32CubeMX1导出一个Cmake工程, 不需要导入到VSCode, 例如
C:\MCU\Tasks\demo_cmd
- 命令行
 - cd "C:\MCU\Tasks\demo_cmd"
 - mkdir build
 - cd build
 - cmake --toolchain ..\cmake\gcc-arm-none-eabi.cmake ..
 - ninja

使用图形界面调试命令行编译的文件

- 创建一个空工程demo_empty_debug, 使用之前的方法保证能编译调试
- 直接修改“imageFileName”配置为前面命令行所编译出的elf文件
- 执行调试
- 注意:
 - 如果调试时报告发现device和core缺失, 则需手工加入:
 - "deviceName": "STM32U385RGT6Q",
 - "deviceCore": "Cortex-M33",



```
1 {
2   // Use IntelliSense to learn about possible attributes.
3   // Hover to view descriptions of existing attributes.
4   // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5   "version": "0.2.0",
6   "configurations": [
7     {
8       "type": "stlinkgdbtarget",
9       "request": "launch",
10      "name": "STM32Cube: Launch ST-Link GDB Server",
11      "origin": "snippet",
12      "deviceName": "STM32U385RGT6Q",
13      "deviceCore": "Cortex-M33",
14      "cwd": "${workspaceFolder}",
15      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
16      "runEntry": "main",
17      "imagesAndSymbols": [
18        {
19          "imageFileName": "C:\\VCU\\Tasks\\2026-3-26-STM32CubeIDE-ForVSCode-Webinar\\demo\\demo_cmd\\build\\demo_cmd.elf",
20        }
21      ]
22    }
23  ]
24 }
```



```
1 // USER CODE BEGIN 0
2
3 // USER CODE END 0
4
5 /**
6  * @brief The application entry point.
7  * @retval int
8  */
9 int main(void)
10 {
11   /* USER CODE BEGIN 1 */
12   /* USER CODE END 1 */
13
14   /* MCU Configuration ----- */
15
16   /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
17   HAL_Init();
18
19   /* USER CODE BEGIN Init */
20   /* USER CODE END Init */
21
22   /* Configure the system clock */
23   SystemClock_Config();
24
25   /* USER CODE BEGIN SysInit */
26   /* USER CODE END SysInit */
27
28   /* Initialize all configured peripherals */
29   MX_GPIO_Init();
30   /* USER CODE BEGIN 2 */
31   /* USER CODE END 2 */
32 }
```



使用命令行调试

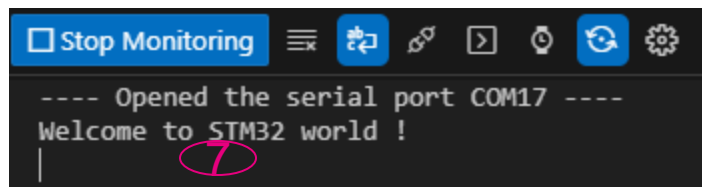
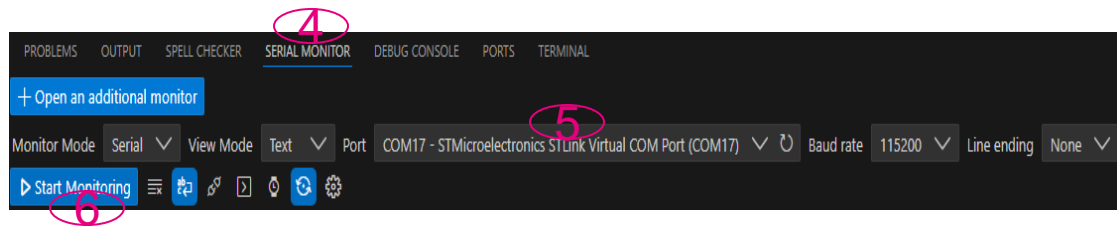
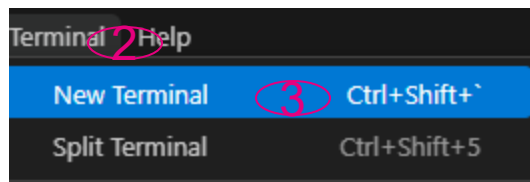
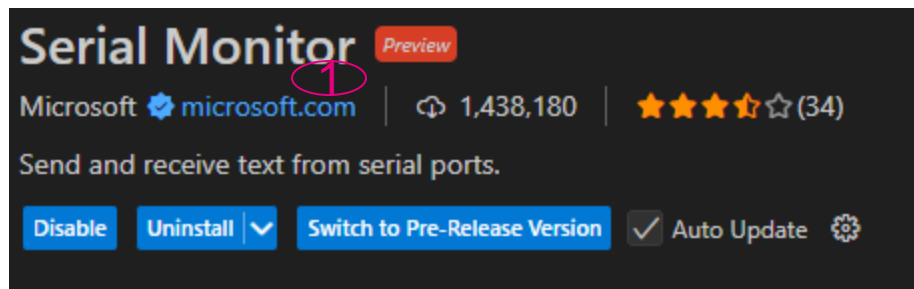
- 设置好环境变量
 - gnu-gdb-for-stm32
 - stlink-gdbserver
- 启动服务器
 - 例如:
 - ST-LINK_gdbserver -p 61234 -d -s -cp "C:\Users\zhangr\AppData\Local\stm32cube\bin\stlink-gdbserver\programmer\2.20.0\bin"
 - 可参考UM2576
- 在另外一个终端里启动调试器
 - arm-none-eabi-gdb.exe .\build\demo_cmd.elf
 - 在(gdb)提示符下输入:
 - target remote :61234
 - load
 - c

丰富生态



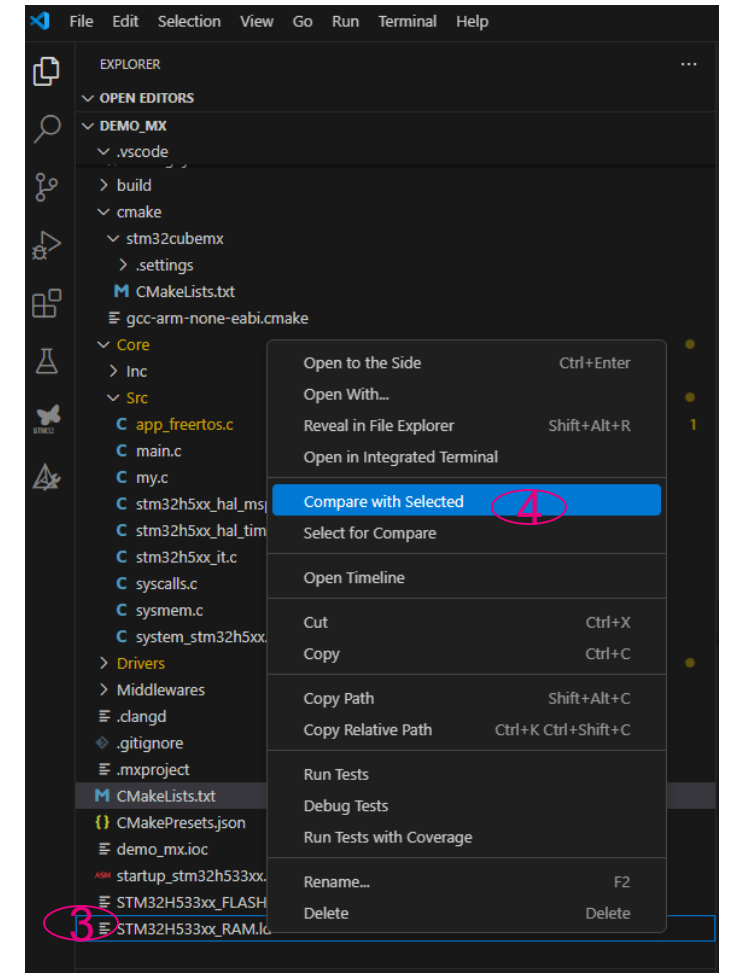
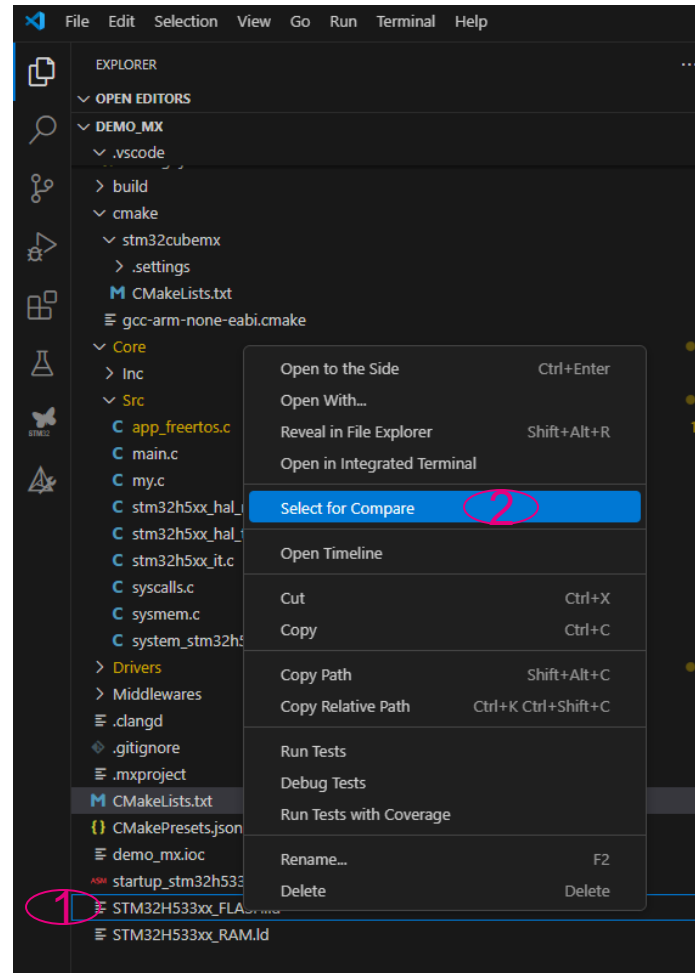
VSCode串口工具

- 安装Serial Monitor扩展
- 点击【Terminal】->【New Terminal】
- 点击【SERIAL MONITOR】
- 选择STLink串口
- 选择【Start Monitoring】
- 重启开发板
- 在屏幕上观察输出

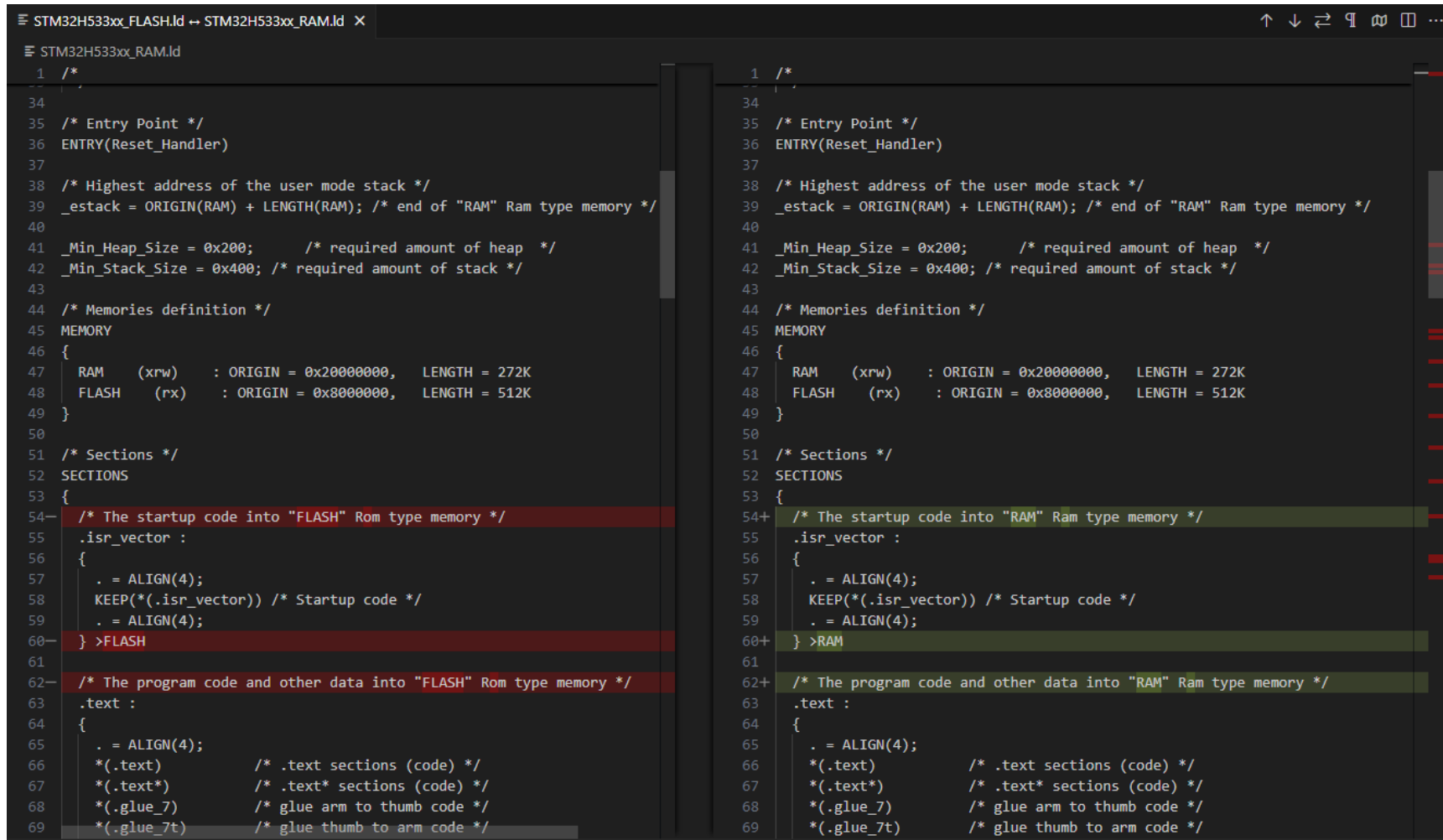


VSCode文件比较

- 右键点击
STM32H533xx_FLASH.Id
- 选择【Select for Compare】
- 右键点击
STM32H533xx_RAM.Id
- 选择【Compare with Selected】



VSCode文件比较



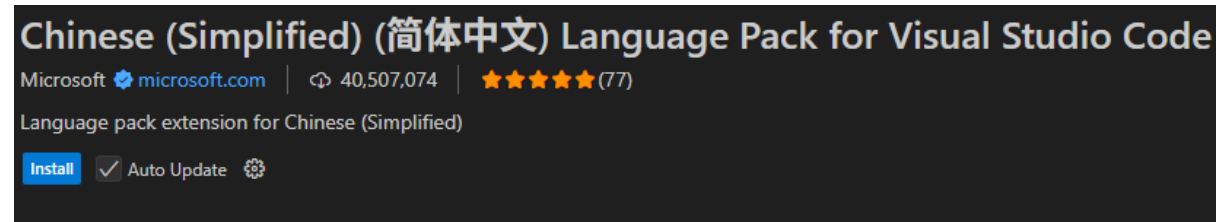
```
STM32H533xx_FLASH.ld ↔ STM32H533xx_RAM.ld X
STM32H533xx_RAM.ld
1  /*
34
35  /* Entry Point */
36  ENTRY(Reset_Handler)
37
38  /* Highest address of the user mode stack */
39  _estack = ORIGIN(RAM) + LENGTH(RAM); /* end of "RAM" Ram type memory */
40
41  _Min_Heap_Size = 0x200; /* required amount of heap */
42  _Min_Stack_Size = 0x400; /* required amount of stack */
43
44  /* Memories definition */
45  MEMORY
46  {
47    RAM    (xrw)  : ORIGIN = 0x20000000, LENGTH = 272K
48    FLASH  (rx)   : ORIGIN = 0x80000000, LENGTH = 512K
49  }
50
51  /* Sections */
52  SECTIONS
53  {
54-  /* The startup code into "FLASH" Rom type memory */
55  .isr_vector :
56  {
57    . = ALIGN(4);
58    KEEP(*(.isr_vector)) /* Startup code */
59    . = ALIGN(4);
60-  } >FLASH
61
62-  /* The program code and other data into "FLASH" Rom type memory */
63  .text :
64  {
65    . = ALIGN(4);
66    *(.text)           /* .text sections (code) */
67    *(.text*)          /* .text* sections (code) */
68    *(.glue_7)         /* glue arm to thumb code */
69    *(.glue_7t)        /* glue thumb to arm code */

```

```
1  /*
34
35  /* Entry Point */
36  ENTRY(Reset_Handler)
37
38  /* Highest address of the user mode stack */
39  _estack = ORIGIN(RAM) + LENGTH(RAM); /* end of "RAM" Ram type memory */
40
41  _Min_Heap_Size = 0x200; /* required amount of heap */
42  _Min_Stack_Size = 0x400; /* required amount of stack */
43
44  /* Memories definition */
45  MEMORY
46  {
47    RAM    (xrw)  : ORIGIN = 0x20000000, LENGTH = 272K
48    FLASH  (rx)   : ORIGIN = 0x80000000, LENGTH = 512K
49  }
50
51  /* Sections */
52  SECTIONS
53  {
54+  /* The startup code into "RAM" Ram type memory */
55  .isr_vector :
56  {
57    . = ALIGN(4);
58    KEEP(*(.isr_vector)) /* Startup code */
59    . = ALIGN(4);
60+  } >RAM
61
62+  /* The program code and other data into "RAM" Ram type memory */
63  .text :
64  {
65    . = ALIGN(4);
66    *(.text)           /* .text sections (code) */
67    *(.text*)          /* .text* sections (code) */
68    *(.glue_7)         /* glue arm to thumb code */
69    *(.glue_7t)        /* glue thumb to arm code */

```

- VSCode支持中文
 - 安装中文扩展包
 - 在命令面板输入【display】
 - 选择【Configure Display Language】
 - 选择 “中文简体”



问与答

VSCode重要的快捷键

- 快捷键
 - 命令【CTRL】+【SHIFT】+【P】
 - 搜索文件【CTRL】+【SHIFT】+【F】
 - 保存文件【CTRL】+【S】

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.

